A COMPLETE COURSE IN ONE VOLUME

System Design

From first principles to planet scale — the concepts, patterns and trade-offs behind systems that serve millions.



FIRST EDITION · 2026
8 parts · 25 chapters · 4 case studies

FIELD GUIDE SERIES

How to use this book

Read it front to back the first time. After that, use it as a reference.

This PDF is interactive

Everything in the **Contents** is a link. Your PDF viewer's **bookmarks / outline panel** shows the full structure for quick jumps. Every chapter has a ↩ **Contents** button. Every quiz question has a **See answer** → button that jumps to the Answer Key, and every answer has a button that jumps back.

Each chapter follows the same rhythm

Problem → mechanism → diagram → trade-offs → a quiz. The boxes in the margins of the argument mean:

KEY IDEA
The one thing to remember from a section.

TRADE-OFF
What you give up when you choose this.

WORKED EXAMPLE
Numbers, step by step.

INTERVIEW TIP
How to say it under pressure.

COMMON PITFALL
Where real systems go wrong.

GOING DEEPER
Optional detail for the curious.

All text and diagrams in this volume are original. The chapter sequence follows the standard system-design curriculum: fundamentals, networking, APIs, communication, caching, databases, architecture, distributed systems, operations and security.

Contents

Preface 7
How the book is organised 7
How to study with it 8

PART I FOUNDATIONS 9

1 What System Design Is 10
1.1 Two kinds of requirements 10
1.2 The anatomy of a typical web system 11
1.3 A repeatable design process 11
1.4 Trade-offs are the whole game 12

2 Scalability and Performance 14
2.1 Describing load 14
2.2 Latency, throughput and percentiles 14
2.3 Vertical vs. horizontal scaling 15
2.4 Statelessness: the key enabler 16
2.5 Finding the bottleneck 16
2.6 Performance techniques at a glance 17

3 Availability, Reliability and Fault Tolerance 18
3.1 Defining the terms 18
3.2 The nines 18
3.3 Availability arithmetic 19
3.4 SLIs, SLOs and SLAs 19
3.5 Eliminating single points of failure 20
3.6 Failover: active-passive and active-active 20
3.7 Designing for failure 21
3.8 Measuring recovery: MTBF, MTTR, RPO, RTO 22

4 Consistency, CAP and PACELC 23
4.1 Why copies disagree 23
4.2 The consistency spectrum 23
4.3 The CAP theorem 24
4.4 PACELC: the everyday trade-off 25
4.5 Tunable consistency with quorums 26
4.6 Choosing consistency per feature 26

5 Back-of-the-Envelope Estimation 28
5.1 Numbers worth memorising 28
5.2 Handy conversions 29
5.3 The estimation recipe 29
5.4 Sanity checks and common mistakes 30

PART II MOVING REQUESTS 32

6 Networking Essentials 33
6.1 The layers that matter 33
6.2 IP addresses and ports 33
6.3 DNS: the internet's phone book 34
6.4 TCP vs. UDP 34
6.5 HTTP: the language of the web 35
6.6 TLS and HTTPS 36
6.7 Proxies: forward and reverse 36

7 Load Balancing 38
7.1 What a load balancer does 38
7.2 Layer 4 vs. layer 7 38
7.3 Load balancing algorithms 39
7.4 Health checks 39
7.5 Session persistence 40
7.6 Avoiding the load balancer as a SPOF 40
7.7 Global load balancing 40

8 Designing APIs 42
8.1 REST 42
8.2 RPC and gRPC 42
8.3 GraphQL 43
8.4 Choosing a style 43
8.5 Pagination 44
8.6 Idempotency keys 44
8.7 Versioning and evolution 44
8.8 API gateways 45

9 Rate Limiting and Authentication 47
9.1 Why rate limit? 47
9.2 Rate limiting algorithms 47
9.3 Distributed rate limiting 49
9.4 Authentication vs. authorisation 49
9.5 Ways to authenticate API calls 50
9.6 OAuth 2.0 and OpenID Connect 50
9.7 Authorisation models 51

PART III MOVING DATA 53

10 Real-Time Communication 54
10.1 Short polling 54
10.2 Long polling 54
10.3 Server-Sent Events (SSE) 54
10.4 WebSockets 54
10.5 WebRTC 55
10.6 Scaling persistent connections 56

11 Message Queues and Publish/Subscribe 58

11.1 Why asynchronous messaging? 58
11.2 Point-to-point queues vs. publish/subscribe 59
11.3 Two families of brokers 59
11.4 Delivery guarantees 60
11.5 Dead-letter queues and poison messages 61
11.6 Backpressure 61

12 Caching 63
12.1 Why caching works 63
12.2 Where caches live 63
12.3 Caching strategies 64
12.4 Eviction policies 65
12.5 Invalidation: the hard part 66
12.6 Cache failure modes 66
12.7 Distributed caching 67

PART IV STORING DATA 69

13 Choosing a Database 70
13.1 Relational databases 70
13.2 NoSQL families 71
13.3 SQL vs. NoSQL: how to decide 72
13.4 Normalisation vs. denormalisation 73

14 Indexing and Replication 74
14.1 Why indexes? 74
14.2 B-tree indexes 74
14.3 LSM trees 75
14.4 Other index types 75
14.5 Replication: why keep copies? 76
14.6 Single-leader replication 76
14.7 Multi-leader replication 77
14.8 Leaderless replication 77

15 Partitioning and Sharding 79
15.1 Vertical vs. horizontal partitioning 79
15.2 Choosing a shard key 79
15.3 Partitioning strategies 80
15.4 Consistent hashing 80
15.5 Rebalancing 81
15.6 The cost of sharding 81

16 Storage Systems and CDNs 84
16.1 Block, file and object storage 84
16.2 Uploading large files directly 85
16.3 Durability: replication vs. erasure coding 85
16.4 Distributed file systems 86
16.5 Content Delivery Networks 86
16.6 Video streaming: a storage case in miniature 87

PART V ARCHITECTURE 88

17 Architectural Styles 89
17.1 The monolith 89
17.2 Microservices 89
17.3 Serverless 91
17.4 Event-driven architecture 91
17.5 Event sourcing and CQRS 92
17.6 Choosing a style 93

18 Microservices Patterns 94
18.1 Service discovery 94
18.2 Circuit breaker 95
18.3 Bulkhead 95
18.4 Timeouts, retries and backoff (revisited) 95
18.5 Sidecar and service mesh 96
18.6 Other essential patterns 97

PART VI DISTRIBUTED SYSTEMS 99

19 Distributed Systems Fundamentals 100
19.1 The fallacies of distributed computing 100
19.2 Partial failure and the unreliable network 100
19.3 Time and clocks 101
19.4 Failure detection and membership 102
19.5 Leader election 102
19.6 Consensus 103
19.7 Distributed locks 104

20 Distributed Transactions 106
20.1 The problem 106
20.2 Two-phase commit (2PC) 106
20.3 The Saga pattern 107
20.4 The dual-write problem and the transactional outbox 109
20.5 Other techniques 109

21 Data Structures for Scale 111
21.1 Bloom filters: "definitely not" or "probably yes" 111
21.2 Count-Min Sketch: frequencies in fixed memory 112
21.3 HyperLogLog: counting distinct items 112
21.4 Merkle trees: comparing huge datasets cheaply 112
21.5 Geospatial indexing: geohash, quadtrees, S2 113
21.6 Skip lists and sorted sets 114
21.7 Unique ID generation at scale 114

PART VII OPERATING AT SCALE 116

22 Big Data Processing 117
22.1 OLTP vs. OLAP 117
22.2 Batch processing and MapReduce 117

22.3 Data warehouses, lakes and lakehouses 118
22.4 Stream processing 119
22.5 Lambda and Kappa architectures 120

23 Deployment and Release Engineering 121
23.1 CI/CD pipelines 121
23.2 Deployment strategies 122
23.3 Feature flags 123
23.4 Database migrations without downtime 124

24 Observability 125
24.1 Logs 125
24.2 Metrics 125
24.3 Distributed tracing 126
24.4 Alerting 127
24.5 Incidents and learning 127

25 Security at Scale 129
25.1 Principles 129
25.2 Encryption in transit: TLS and mTLS 129
25.3 Encryption at rest 130
25.4 Secrets management 130
25.5 Identity and access control 130
25.6 Common attacks and defences 131
25.7 DDoS protection 131
25.8 Auditing, privacy and compliance 132

PART VIII CASE STUDIES 133

26 Case Study: A URL Shortener 134
26.1 Step 1: Requirements 134
26.2 Step 2: Estimates 134
26.3 Step 3: API 135
26.4 Step 4: Generating short codes 135
26.5 Step 5: Data model and storage 135
26.6 Step 6: High-level design 136
26.7 Step 7: Deep dives 136

27 Case Study: A Social News Feed 138
27.1 Step 1: Requirements 138
27.2 Step 2: Estimates 138
27.3 Step 3: Fan-out on write vs. fan-out on read 138
27.4 Step 4: Data model 139
27.5 Step 5: High-level design 140
27.6 Step 6: Deep dives 140

28 Case Study: A Chat System 142
28.1 Step 1: Requirements 142
28.2 Step 2: Estimates 142
28.3 Step 3: Connection layer 142

28.4 Step 4: Sending a message 143
28.5 Step 5: Ordering and delivery guarantees 143
28.6 Step 6: Storage 144
28.7 Step 7: Group chats 144
28.8 Step 8: Presence 144
28.9 Step 9: End-to-end encryption 144

29 Case Study: A Ride-Hailing Platform 146
29.1 Step 1: Requirements 146
29.2 Step 2: Estimates 146
29.3 Step 3: High-level design 147
29.4 Step 4: Location ingestion and geospatial index 147
29.5 Step 5: Matching 147
29.6 Step 6: The trip state machine 148
29.7 Step 7: Surge pricing and ETAs 148
29.8 Step 8: Reliability and scale 148

The Design Interview Playbook 150
Questions to ask in every design 150
Phrases that show judgement 151

Reference Numbers 152
Latency 152
Throughput rules of thumb (single node, order of magnitude) 152
Time and availability 153

Glossary 154

Answer Key 157

Preface

↩ Contents

Every large system you use today, whether a search engine, a ride-hailing app or a video platform, started as a single program on a single machine. At some point the load outgrew that machine. Someone had to decide how to split the work, where to keep the data, what to do when a disk died at 3 a.m., and which guarantees to give up for the sake of speed. **System design is the discipline of making those decisions on purpose.**

This book teaches that discipline from the ground up. It assumes you can write code and know roughly what a database and an HTTP request are. It does not assume you have run anything at scale. Each chapter starts from a concrete problem, builds the mechanism that solves it, draws it, and then spends as much time on what the mechanism *costs* as on what it buys. That last part matters most. Nobody hires a senior engineer to name every component. They hire one to know when *not* to use it.

How the book is organised

The book has eight parts, and each one depends on the parts before it.

Part	What you will learn
I. Foundations	The vocabulary: scalability, latency, availability, reliability, consistency, and the arithmetic of estimation.
II. Moving Requests	How a request travels: DNS, TCP, HTTP, load balancers, API design, gateways, rate limiting and authentication.
III. Moving Data	How services talk to each other in real time and asynchronously, and how caching makes everything faster.
IV. Storing Data	Choosing databases, indexing, replication, sharding, and storing files and objects at scale.
V. Architecture	Monoliths, microservices, serverless and event-driven systems, and the patterns that keep them standing.
VI. Distributed Systems	Time, ordering, consensus and distributed transactions, plus clever data structures for huge data.
VII. Operating at Scale	Batch and stream processing, safe deployment, observability and security.
VIII. Case Studies	Four complete designs that put it all together, worked as you would in an interview.

How to study with it

1. **Read actively.** Before each diagram, try to predict what it will show.
2. **Do the quizzes.** Every chapter ends with questions. Answer them in writing before you click *See answer*. Struggling to recall something is what moves it into long-term memory.
3. **Redraw the figures from memory.** If you can redraw a figure and explain each arrow, you understand it.
4. **Work the case studies twice.** First on your own with a blank page, then with the book.

The road from "it works on my laptop" to "it works for a hundred million people" is long, but it is not mysterious. Let's begin.

PART I

Foundations

The vocabulary every design conversation is built on: what we are optimising for, how we measure it, and the trade-offs we can never escape.

CHAPTER 1

↩ Contents

What System Design Is

System design is the craft of choosing **components**, the **connections** between them, and the **guarantees** they provide, so that a system meets its requirements at an acceptable cost. A design is never simply "correct". A good design is one where every trade-off was chosen deliberately and can be explained.

1.1 Two kinds of requirements

Every design starts from requirements, and they come in two kinds.

Functional requirements describe *what* the system does: "users can upload a photo", "a driver is matched to a rider", "a message is delivered to everyone in a group". They define features.

Non-functional requirements describe *how well* the system does it: how many users, how fast, how often it may be down, how fresh the data must be, how much it may cost. They define the *shape* of the architecture. Two systems with identical features can look nothing alike because one serves a thousand users and the other a billion.

Non-functional property	The question it answers	Typical target
Scalability	Can we handle 10x the load by adding resources?	Linear-ish growth
Latency	How long does one request take?	p99 < 200 ms
Throughput	How many requests per second?	50k req/s
Availability	What fraction of time does it work?	99.95%
Durability	Once saved, will data survive failures?	11 nines
Consistency	Do all readers see the same, latest data?	Strong / eventual
Cost	What does each request cost us?	$ per million req

KEY IDEA

The architecture follows from the non-functional requirements

Features tell you *which* boxes exist. Scale, latency, availability and consistency targets tell you *how many* boxes there are, *where* they live, and *how* they are wired together. Pin these numbers down early.

1.2 The anatomy of a typical web system

Most systems you will design are variations on one skeleton. Learn it well, because nearly every later chapter zooms into one of its boxes.

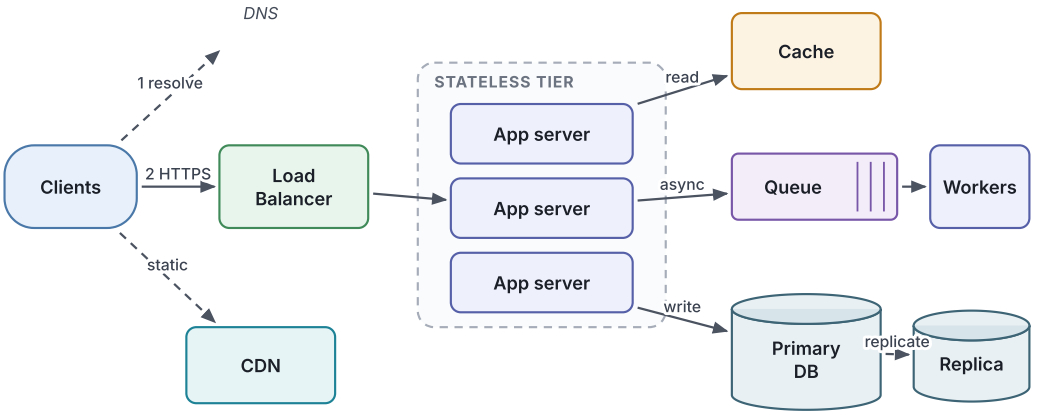


Figure 1.1 The canonical web architecture. Nearly every chapter in this book zooms into one of these boxes.

A client resolves a name through **DNS**, fetches static assets from a **CDN** close to it, and sends dynamic requests to a **load balancer**. The load balancer spreads them across **stateless application servers**. Those servers read hot data from a **cache**, store durable data in a **database** (often a primary with replicas), and hand slow work to a **queue** for background **workers** to do later.

1.3 A repeatable design process

Whether you are in an interview or a design review, the same seven steps keep you from going in circles.

1. **Clarify the requirements.** Who are the users? What are the three most important features? What is explicitly *out of scope*?

2. **Estimate the scale.** Daily users, requests per second at peak, read/write ratio, storage per year, bandwidth (Chapter 5).
3. **Define the API.** List the endpoints or messages, with their inputs and outputs. This forces precision.
4. **Model the data.** Which entities exist, how they relate, and above all *how they will be queried*.
5. **Sketch the high-level design.** Draw the boxes and arrows, and walk one request end to end.
6. **Deep-dive the bottlenecks.** Ask what breaks first at 10x load, then fix that: cache it, shard it, queue it, or replicate it.
7. **Discuss trade-offs and failures.** What happens when each box dies? What would you monitor? What would you build differently with more time?

INTERVIEW TIP

Drive the conversation

Interviewers grade the *process* as much as the final picture. State your assumptions out loud ("I'll assume 100M daily users and a 10:1 read/write ratio"), check them, and move on. Silence while you draw is wasted time.

1.4 Trade-offs are the whole game

You will meet the same tensions in every chapter:

- **Consistency vs. availability.** During a failure, do you return possibly-stale data or an error?
- **Latency vs. throughput.** Batching raises throughput but makes individual requests wait.
- **Simplicity vs. flexibility.** A monolith is easy to reason about. Microservices are easier to scale separately.
- **Cost vs. everything.** Each extra nine of availability or millisecond saved has a price.
- **Read speed vs. write speed.** Indexes, caches and denormalised copies make reads fast and writes slow.

TRADE-OFF

Every "best practice" is a trade-off in disguise

When someone says "always use a cache" or "never use joins", ask: *what does it cost, and when is that cost worth paying?* By the end of this book you should be able to answer that question for every pattern it contains.

CHECK YOUR UNDERSTANDING

1. Classify each as functional or non-functional: (a) users can follow other users; (b) the feed loads in under 300 ms; (c) data survives the loss of a data centre; (d) users can search posts by hashtag.

   See answer →

2. Why is it important for application servers to be stateless in the canonical architecture?

   See answer →

3. In a design interview, why should you estimate scale *before* drawing the architecture?

   See answer →

CHAPTER 2

↩ Contents

Scalability and Performance

A system is **scalable** if it can handle more load by adding resources, without being redesigned, and with performance staying acceptable. Before we can scale anything, we need to say precisely what "load" and "performance" mean.

2.1 Describing load

Load is described by a few **load parameters** chosen for your system. For a web API it might be requests per second. For a chat system, concurrent connections. For a database, the read/write ratio and the size of the working set. For a social network, a famous example is *fan-out*: how many followers' feeds a single post must reach.

Pick the load parameters that actually stress your design. "Ten thousand users" is not a load parameter. "Ten thousand writes per second, each fanning out to an average of 200 followers" is.

2.2 Latency, throughput and percentiles

Latency is the time to handle one request. **Throughput** is how many requests are completed per unit time. They are related but not interchangeable. A highway can carry many cars per hour (throughput) while each car still takes forty minutes to cross the city (latency).

Latency is never one number. Each request takes a different amount of time because of garbage collection, cache misses, network jitter, lock contention and so on. So we report **percentiles**:

- **p50 (median):** half of requests are faster than this. It describes the typical experience.
- **p95 / p99:** 95% or 99% of requests are faster. These describe the *tail*, the unhappy users.
- **p99.9:** often your most valuable customers, because the heaviest users make the most requests and so are the most likely to hit the slow ones.

COMMON PITFALL

Averages hide pain

An average latency of 80 ms can hide a p99 of 2 seconds. Averages blend fast and slow requests into a number that no actual user experiences. Always ask for percentiles.

GOING DEEPER

Tail latency amplification

If a page calls 50 backend services in parallel and must wait for all of them, the page is as slow as the *slowest* call. If each service is slow on 1% of requests, the chance that at least one of the 50 is slow is $1 - 0.99^{50} \approx$ **39%**. Rare backend slowness becomes common user slowness. This is why large systems obsess over p99 and use techniques like *hedged requests*: send a duplicate request after a short delay and take whichever reply arrives first.

2.3 Vertical vs. horizontal scaling

There are two fundamental ways to add capacity.

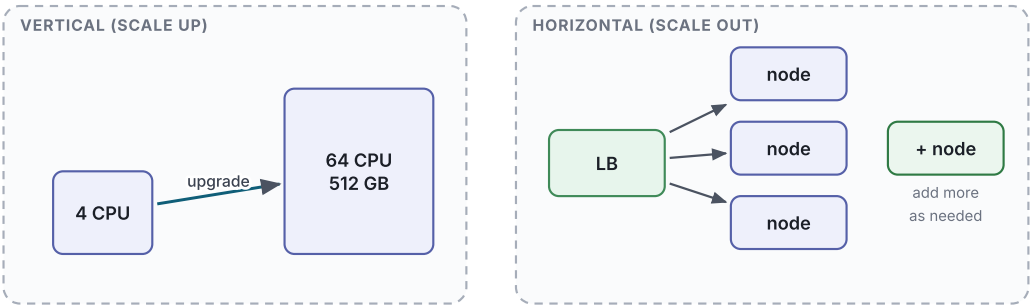


Figure 2.1 Vertical scaling grows one machine. Horizontal scaling adds machines behind a load balancer.

Vertical scaling (scaling up) means buying a bigger machine: more CPU, RAM and faster disks. It needs no code changes and keeps everything simple. But hardware has a ceiling, the price of the top-end machines rises much faster than their capacity, and you still have one machine whose failure takes everything down.

Horizontal scaling (scaling out) means adding more machines and spreading the load across them. In principle it has no ceiling, uses cheap commodity hardware and survives individual failures. The price is complexity: you now need load balancing, a way to divide data, and answers to the hard questions of distributed systems.

TRADE-OFF

Start vertical, design for horizontal

A single well-tuned modern server can handle far more than most people expect. Scale up while that is cheap. But keep the application tier stateless from day one, so that scaling out later is a configuration change rather than a rewrite.

2.4 Statelessness: the key enabler

A service is **stateless** if it keeps no client-specific data between requests. Every request carries, or can look up, everything needed to handle it. Stateless servers are interchangeable: the load balancer can send any request anywhere, you can add ten servers during a traffic spike and remove them afterwards, and a crash loses nothing.

State has to live *somewhere*, so we push it into components built for it: databases, distributed caches and session stores. The scaling problem doesn't disappear. It moves to the data tier, where we deal with it carefully, using the replication and partitioning techniques of Part IV.

COMMON PITFALL

Sticky sessions

Pinning a user to one server ("session affinity") is a common shortcut that quietly brings state back. If that server dies, the user's session is lost. If one server gets a cluster of heavy users, it overheats while the others sit idle. Prefer external session storage.

2.5 Finding the bottleneck

Throughput is always limited by the slowest stage, which is the **bottleneck**. Adding web servers does nothing if the database is saturated. Before scaling anything, measure where time is actually spent: CPU, memory, disk I/O, network, lock contention, or a downstream dependency.

Little's Law links the three quantities you will measure most: $L = \lambda \times W$. The average number of requests in the system (L) equals the arrival rate (λ) times the average time each request spends in it (W). If a service handles 2,000 requests per second with 50 ms latency, about 100 requests are in flight at any moment. You need enough threads or connections for that, plus headroom.

WORKED EXAMPLE

Sizing a thread pool with Little's Law

An API receives 1,200 req/s at peak. Each request waits ~150 ms on a downstream call. In-flight requests = 1,200 × 0.15 = **180**. With synchronous, thread-per-request servers, the fleet needs at least 180 worker threads in total, plus about 50% headroom for bursts, so roughly **270**. Spread over 6 servers, that is about 45 threads each.

2.6 Performance techniques at a glance

Technique	Improves	Costs	Chapter
Caching	Latency, DB load	Staleness, invalidation	12
Load balancing	Throughput, availability	Another hop	7
Async processing / queues	Latency of user path	Eventual completion	11
Read replicas	Read throughput	Replication lag	14
Sharding	Write throughput, storage	Cross-shard queries	15
CDN	Latency for static content	Purge complexity	16
Batching	Throughput	Per-item latency	22

CHECK YOUR UNDERSTANDING

1. A service reports an average latency of 90 ms. Why is that not enough information to judge user experience?

   See answer →

2. A page makes 20 parallel backend calls and waits for all of them. Each call independently exceeds 500 ms 1% of the time. Roughly how often is the page slower than 500 ms?

   See answer →

3. Give two advantages and two disadvantages of horizontal scaling compared with vertical scaling.

   See answer →

4. A service handles 5,000 req/s and each request takes 40 ms. How many requests are in flight on average?

   See answer →

CHAPTER 3

↩ Contents

Availability, Reliability and Fault Tolerance

Machines fail. Disks wear out, power supplies die, someone pushes a bad config, a backhoe cuts a fibre line. In a fleet of ten thousand servers, something is *always* broken. This chapter is about building systems that keep working anyway.

3.1 Defining the terms

- **Availability** is the fraction of time a system is able to serve requests successfully.
- **Reliability** is the probability that the system does the *correct* thing over a period of time. A system can be available but unreliable, for example up and responding, but with corrupted data.
- **Fault tolerance** is the ability to keep operating, possibly with reduced capability, when components fail.
- A **fault** is one component going wrong. A **failure** is the system as a whole stopping its service to users. The goal of fault tolerance is to keep faults from becoming failures.

3.2 The nines

Availability is usually quoted in "nines":

Availability	Downtime / year	Downtime / month	Typical for
99% (two nines)	3.65 days	7.3 hours	Internal tools
99.9% (three nines)	8.77 hours	43.8 minutes	Many SaaS products
99.95%	4.38 hours	21.9 minutes	Mature web services
99.99% (four nines)	52.6 minutes	4.4 minutes	Payments, core infra
99.999% (five nines)	5.26 minutes	26 seconds	Telecom, critical systems

Each extra nine costs roughly ten times the engineering effort. Five nines leaves 26 seconds of downtime a month, which is far less time than it takes a human to notice an alert. At that level, *everything* must recover automatically.

3.3 Availability arithmetic

Components in series (all must work) multiply their availabilities, so every dependency lowers the total:

$$A(total) = A_1 \times A_2 \times \ldots \times A_n$$

Redundant components in parallel (any one is enough) fail only if *all* fail:

$$A(total) = 1 - (1 - A_1) \times (1 - A_2) \times \ldots \times (1 - A_n)$$

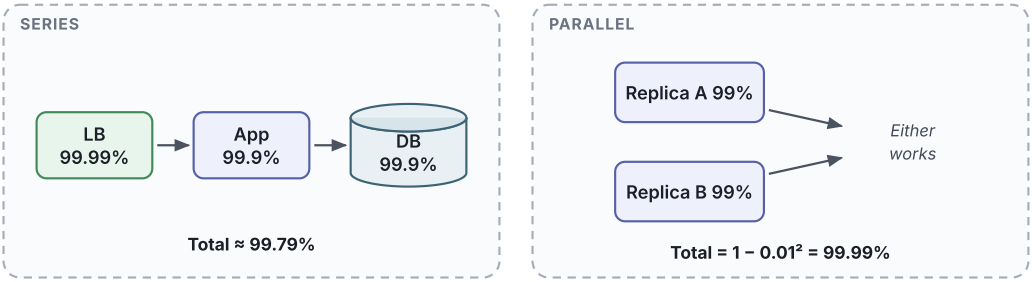


Figure 3.1 Dependencies in series lower availability. Redundancy in parallel raises it.

WORKED EXAMPLE

The cost of a long call chain

A request passes through an API gateway (99.99%), an auth service (99.95%), an order service (99.9%) and a database (99.95%). Total = 0.9999 × 0.9995 × 0.999 × 0.9995 ≈ **99.79%**, which is about 18 hours of downtime a year, even though each part looks excellent on its own. Removing a hop or making a dependency optional (degrading gracefully if the recommendation service is down, say) raises the ceiling.

3.4 SLIs, SLOs and SLAs

- An **SLI** (Service Level *Indicator*) is a measured number, for example "the fraction of HTTP requests that returned a non-5xx response within 300 ms".
- An **SLO** (Service Level *Objective*) is an internal target for an SLI, for example "99.9% over a rolling 30 days".

- An **SLA** (Service Level *Agreement*) is a contract with customers, with consequences (usually refunds) if it is missed. SLAs are set *looser* than SLOs so that you notice trouble before it costs money.

The gap between 100% and the SLO is the **error budget**. If the SLO is 99.9%, you may "spend" 0.1% of requests on failures each month. While budget remains, ship features fast. When it runs out, slow down and invest in reliability. This turns an endless argument between product and operations teams into a number both can see.

3.5 Eliminating single points of failure

A **single point of failure (SPOF)** is any component whose failure brings down the whole system. Finding them is a systematic exercise. Walk the request path and, for each box, ask: *"If this disappears right now, what happens?"*

The cure is **redundancy**: more than one of everything, ideally in independent failure domains.

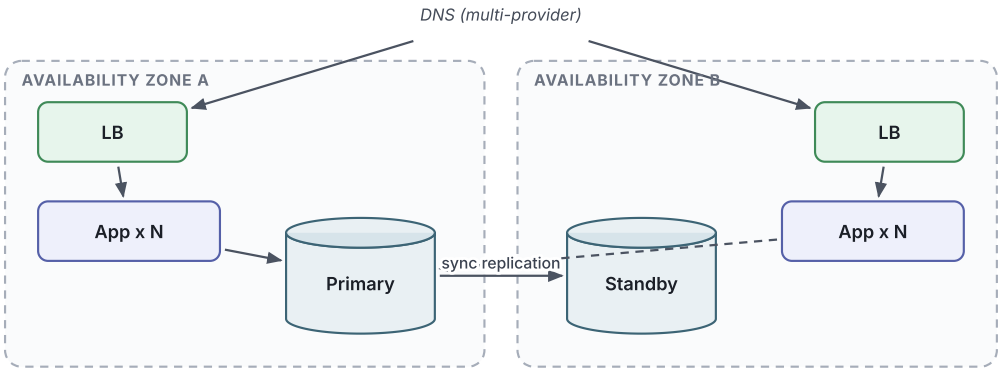


Figure 3.2 Redundancy across availability zones. Losing any single box, or a whole zone, leaves the system serving.

Common redundancy layers include multiple servers behind a load balancer, multiple load balancers sharing a floating IP or DNS entries, database replicas with automatic failover, multiple availability zones (separate data centres in one region), and, for the most critical systems, multiple regions.

3.6 Failover: active-passive and active-active

- **Active-passive.** One node serves while a standby waits. A health check detects failure and promotes the standby. It is simple, but the standby's capacity sits idle, and failover takes seconds to minutes.

- **Active-active.** All nodes serve traffic at once. Losing one shifts its load to the others. Capacity isn't wasted and recovery is near-instant, but for stateful systems you must handle concurrent writes in several places (see Chapter 14).

COMMON PITFALL

Split brain

If the primary is merely *slow* rather than dead, a naive failover promotes the standby while the old primary keeps accepting writes. Now two nodes both think they are in charge, and their data diverges. Real systems prevent this with **fencing** (cutting the old leader off from storage or the network) and with **quorum**-based leader election (Chapter 19).

3.7 Designing for failure

Redundancy is only half the job. Every remote call can fail, hang or return garbage, so the code needs defensive habits too.

1. **Timeouts on everything.** A call without a timeout can hang forever and take a thread with it. Set timeouts from the dependency's measured p99.
2. **Retries with exponential backoff and jitter.** Retry transient errors, waiting longer each time (100 ms, 200 ms, 400 ms…) with randomness added, so that thousands of clients don't retry in lockstep.
3. **Idempotency.** Retries create duplicates. Make operations safe to repeat, for example by attaching an *idempotency key* to each payment request so the server can recognise and ignore repeats.
4. **Circuit breakers.** Stop calling a dependency that keeps failing, so that it can recover and your threads aren't wasted (Chapter 18).
5. **Bulkheads.** Isolate resources (thread pools, connection pools) per dependency, so that one slow dependency can't sink the whole service.
6. **Graceful degradation.** Serve a reduced experience rather than an error: hide recommendations, show a cached price, disable non-essential features.

COMMON PITFALL

Retry storms

When a service slows down, clients retry. That multiplies load on the struggling service, so more requests fail, so clients retry more. Unbounded retries can turn a brief blip into a full outage. Cap retries, use backoff with jitter, and consider a *retry budget* (for example, retries may never exceed 10% of normal traffic).

3.8 Measuring recovery: MTBF, MTTR, RPO, RTO

- **MTBF** (mean time between failures) measures how often things break.
- **MTTR** (mean time to recovery) measures how fast you fix them. Availability ≈ MTBF / (MTBF + MTTR), so cutting MTTR is often the cheapest way to add a nine.
- **RPO** (recovery point objective): how much data you can afford to lose, measured in time. With an RPO of 5 minutes, backups or replication must be no more than 5 minutes behind.
- **RTO** (recovery time objective): how long you can afford to be down before service is restored.

CHECK YOUR UNDERSTANDING

1. A system has four dependencies in series, each 99.9% available. What is the best-case total availability, and roughly how much downtime is that per year?

   See answer →

2. Two independent replicas are each 99.5% available, and either can serve traffic. What is the combined availability?

   See answer →

3. What is an error budget, and how does it change team behaviour?

   See answer →

4. Explain why retries without backoff can cause an outage.

   See answer →

5. What is split brain and how is it prevented?

   See answer →

CHAPTER 4

↩ Contents

Consistency, CAP and PACELC

The moment you keep more than one copy of data, and you will, for availability and performance, you must decide what happens when the copies disagree. That decision is one of the most consequential in any design.

4.1 Why copies disagree

Suppose a user updates their profile photo. The write lands on replica A. Replica B, on another continent, receives the change a few hundred milliseconds later. During that window, a reader hitting B sees the old photo. Whether that matters depends on the data. For a profile photo, nobody cares. For a bank balance, it matters a great deal.

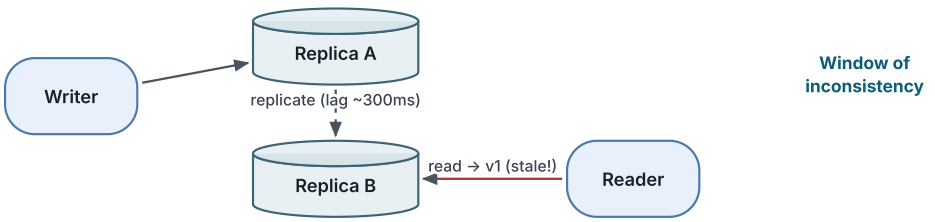


Figure 4.1 Replication lag: a reader hitting the lagging replica sees stale data for a short window.

4.2 The consistency spectrum

Consistency models are promises about what a reader may observe. From strongest to weakest:

- **Linearizability (strong consistency).** The system behaves as if there were a single copy and every operation happens instantly at some point between its start and end. Once a write completes, every later read sees it. This is the easiest model to program against and the most expensive to provide.
- **Sequential consistency.** All nodes see operations in the same order, though not necessarily in real-time order.

- **Causal consistency.** Operations that are causally related (a reply and the post it answers) are seen in order by everyone. Unrelated operations may be seen in different orders.
- **Read-your-writes.** A user always sees their own updates, even if other users see them a little later.
- **Monotonic reads.** Once you have seen a value, you never see an older one. Time doesn't run backwards for you.
- **Eventual consistency.** If writes stop, all replicas will *eventually* converge to the same value. No promise is made about when, or what you see in the meantime.

KEY IDEA

Weaker consistency buys latency and availability

Strong consistency requires coordination: replicas must agree before answering, which costs round trips and becomes impossible when they can't communicate. Weaker models let each replica answer on its own, which is faster and survives partitions, at the price of sometimes returning stale data.

4.3 The CAP theorem

The CAP theorem concerns three properties of a replicated data store:

- **C, Consistency:** in CAP this means linearizability. Every read sees the most recent write.
- **A, Availability:** every request to a non-failed node gets a non-error response.
- **P, Partition tolerance:** the system keeps working even if the network drops messages between nodes.

The theorem says that **when a network partition happens, you must choose between consistency and availability.** You cannot "choose not to have partitions", because networks fail. So the real choice is what to do *during* one:

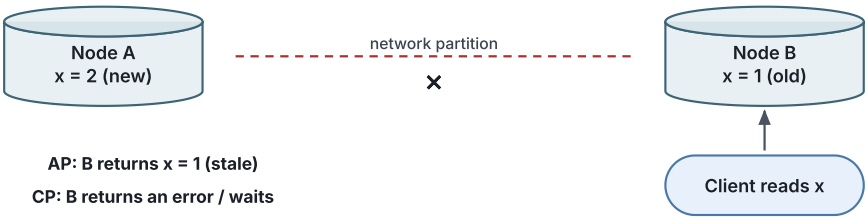


Figure 4.2 During a partition, node B cannot learn about writes on A. It must either refuse to answer (CP) or answer with possibly stale data (AP).

- A **CP system** refuses some requests (returns errors or times out) on the side of the partition that can't confirm it has the latest data. Examples: ZooKeeper, etcd, and HBase-style stores. Bank balances and leader election belong here.
- An **AP system** keeps answering everywhere and reconciles afterwards. Examples: Cassandra and DynamoDB in their default configurations, and DNS. Shopping carts, like counts and social feeds belong here.

COMMON PITFALL

"Pick two out of three" is misleading

The slogan suggests you could pick "CA" and give up partition tolerance. In a distributed system you can't, because partitions simply happen. CAP only forces a choice *while* a partition is happening. The rest of the time a system can be both consistent and available.

4.4 PACELC: the everyday trade-off

CAP says nothing about normal operation, which is where systems spend 99.99% of their time. **PACELC** fills the gap:

> If there is a **P**artition, choose **A** or **C**; **E**lse (normal operation), choose **L**atency or **C**onsistency.

Even with a perfect network, strong consistency costs time, because a write must reach a quorum or a leader before it is acknowledged. Systems like DynamoDB and Cassandra are **PA/EL**: available during partitions, low-latency otherwise. Traditional single-leader relational setups with synchronous replicas are **PC/EC**. Many databases let you tune this per request.

4.5 Tunable consistency with quorums

Leaderless databases keep N replicas of each item and let you choose how many must respond to a write (W) and to a read (R).

If R + W > N, every read overlaps with the latest write on at least one replica, so reads see the latest value (given careful conflict handling).

N	W	R	Behaviour
3	3	1	Fast reads, slow writes; any one node down blocks writes
3	1	3	Fast writes, slow reads
3	2	2	Balanced; tolerates one node down for both reads and writes
3	1	1	Fastest, but reads may be stale (eventual)

4.6 Choosing consistency per feature

You rarely choose a single model for a whole system. Choose per kind of data:

Data	Model	Why
Account balance, inventory count	Strong	Overselling or double-spending costs money
Usernames, uniqueness constraints	Strong	Two users must not get the same name
User's own posts and settings	Read-your-writes	Users panic if their edit "disappears"
Comments thread	Causal	Replies must not appear before their parent
Like counts, view counts	Eventual	Off by a few for a second is harmless
Recommendations, analytics	Eventual	Stale by minutes is fine

INTERVIEW TIP

How to talk about CAP in an interview

Don't recite the definition. Apply it: "For the payment ledger I'll choose consistency, so during a partition we reject writes rather than risk a double charge. For the activity feed I'll choose availability and eventual consistency, because showing a feed that is slightly behind is better than showing an error."

CHECK YOUR UNDERSTANDING

1. In the CAP theorem, why is "CA" not a realistic choice for a distributed system?

   See answer →

2. What extra trade-off does PACELC describe that CAP does not?

   See answer →

3. With N = 5 replicas, give a choice of R and W that guarantees reads see the latest write, and one that favours write latency.

   See answer →

4. Which consistency model fits each: (a) a user editing their bio; (b) a global "trending topics" list; (c) the remaining seats on a flight?

   See answer →

CHAPTER 5

↩ Contents

Back-of-the-Envelope Estimation

Good engineers can tell, within a minute and on a napkin, whether a design needs one server or a thousand. Estimation isn't about precision. It is about getting the **order of magnitude** right, so that you neither over-build nor under-build.

5.1 Numbers worth memorising

Operation	Approximate latency
L1 cache reference	1 ns
Main memory reference	100 ns
Compress 1 KB (fast codec)	2 µs
Read 1 MB sequentially from memory	3 µs
SSD random read (4 KB)	16–100 µs
Read 1 MB sequentially from SSD	50 µs
Round trip within a data centre	0.5 ms
Read 1 MB sequentially from HDD	1 ms
HDD seek	10 ms
Round trip US East ↔ US West	70 ms
Round trip US ↔ Europe	80–150 ms

Things to take away: memory is roughly **1000x faster than SSD random access**, the network inside a data centre is fast, the network between continents is slow (light only goes so fast), and spinning-disk seeks are terrible. Design so that the hot path touches memory and avoids cross-region round trips.

5.2 Handy conversions

Quantity	Value
Seconds per day	86,400 ≈ **10^5**
Seconds per month	~2.6 million ≈ 2.5×10^6
1 million requests/day	≈ **12 req/s**
1 billion requests/day	≈ **12,000 req/s**
1 KB × 1 million	1 GB
1 MB × 1 million	1 TB
1 KB × 1 billion	1 TB

Powers of two: 2^{10} ≈ thousand (KB), 2^{20} ≈ million (MB), 2^{30} ≈ billion (GB), 2^{40} ≈ trillion (TB), 2^{50} ≈ quadrillion (PB).

Rough single-machine capacities (they vary a lot, so treat them as starting points): a web/app server handles a few thousand simple req/s; a relational database a few thousand to tens of thousands of simple queries per second; an in-memory cache like Redis around 100k+ ops/s per node; a single Kafka partition tens of MB/s.

5.3 The estimation recipe

1. **Users → requests.** Daily active users (DAU) × actions per user per day ÷ 10^5 gives the average requests per second.
2. **Average → peak.** Multiply by a peak factor (2–3x for smooth global traffic, up to 10x for spiky events).
3. **Split reads and writes** using the read/write ratio.
4. **Storage** = writes per day × object size × retention (× replication factor).
5. **Bandwidth** = requests per second × response size.
6. **Memory for caching:** the 80/20 rule suggests caching the hottest ~20% of daily read data.
7. **Servers** = peak requests per second ÷ capacity per server, plus headroom.

WORKED EXAMPLE

Estimating a photo-sharing service

Assumptions: 50M DAU. Each user views 30 photos and uploads 0.2 photos per day. Average photo: 2 MB original, 200 KB in display size. Retain forever, with 3x replication.

Requests - Views: 50M × 30 = 1.5B/day ÷ 10^5 ≈ **15,000 reads/s** average, ~45,000/s at 3x peak. - Uploads: 50M × 0.2 = 10M/day ÷ 10^5 ≈ **100 writes/s**, ~300/s at peak. The read/write ratio is ~150:1, heavily read-dominated, so caching and CDNs are essential.

Storage - 10M photos/day × (2 MB + 200 KB) ≈ 22 TB/day ≈ **8 PB/year**, × 3 replicas ≈ **24 PB/year**. This rules out a database for the image bytes. You need object storage, with metadata in a database.

Bandwidth - Egress: 45,000 views/s × 200 KB ≈ **9 GB/s at peak** (~72 Gbps). A CDN must absorb this.

Metadata - ~500 bytes per photo × 10M/day ≈ 5 GB/day ≈ 1.8 TB/year. That is manageable in a sharded database.

5.4 Sanity checks and common mistakes

- **Keep the units straight.** Bits vs. bytes (network speeds are quoted in bits: 1 Gbps ≈ 125 MB/s).
- **Round aggressively.** 86,400 becomes 100,000. 3.7 becomes 4. You are after the order of magnitude.
- **Don't forget replication and overhead.** Replicas, indexes, backups and metadata often double or triple raw storage.
- **Distinguish average from peak.** Systems fall over at peak, not on average.
- **State your assumptions.** A wrong assumption that is out in the open can be corrected. A hidden one can't.

INTERVIEW TIP

Narrate the arithmetic

In an interview, write the formula before the numbers: "views per second = DAU × views per user ÷ 10^5". If you slip on the arithmetic, the interviewer can still see your reasoning, and your conclusion ("so we need a CDN") is what they really care about.

CHECK YOUR UNDERSTANDING

1. A service gets 300 million requests per day, with peak at 4x average. Estimate peak requests per second.

   See answer →

2. A messaging app stores 2 billion messages per day at 200 bytes each and keeps them for 5 years with 3x replication. Estimate the storage.

   See answer →

3. Why is the ~1000x difference between memory and SSD access worth remembering?

   See answer →

4. Your API returns 50 KB responses at 20,000 req/s. What egress bandwidth is that, in Gbps?

   See answer →

PART II

Moving Requests

How a request finds its way from a phone to a server and back: naming, transport, load balancing, interfaces, and how APIs are protected.

CHAPTER 6

↩ Contents

Networking Essentials

You can't design distributed systems without understanding the network that connects their parts. This chapter covers the networking every system designer needs: how names become addresses, how bytes travel reliably, and what HTTP actually does.

6.1 The layers that matter

Networking is organised in layers, each building on the one below it. For system design you need four of them:

Layer	Job	Examples
Application (L7)	Meaning of messages	HTTP, gRPC, WebSocket, DNS, SMTP
Transport (L4)	End-to-end delivery between processes	TCP, UDP, QUIC
Network (L3)	Routing packets between machines	IP (v4 / v6)
Link (L2)	Moving frames on one physical network	Ethernet, Wi-Fi

When engineers say a load balancer works at "L4" or "L7", they mean whether it makes decisions from transport information (IP addresses and ports) or from application content (URLs, headers, cookies).

6.2 IP addresses and ports

Every host on a network has an **IP address**. IPv4 addresses are 32 bits (`203.0.113.7`), and they ran out long ago. IPv6 addresses are 128 bits. Private ranges (`10.0.0.0/8`, `192.168.0.0/16`) are used inside data centres and homes, and **NAT** (network address translation) lets many private hosts share one public address.

A **port** (0–65535) identifies a particular process on a host: 443 for HTTPS, 5432 for PostgreSQL, 6379 for Redis. A connection is identified by the 4-tuple *(source IP, source port, destination IP, destination port)*.

6.3 DNS: the internet's phone book

Humans use names. Machines route by IP. The **Domain Name System** translates between them, and it is itself a large, distributed, heavily cached system.

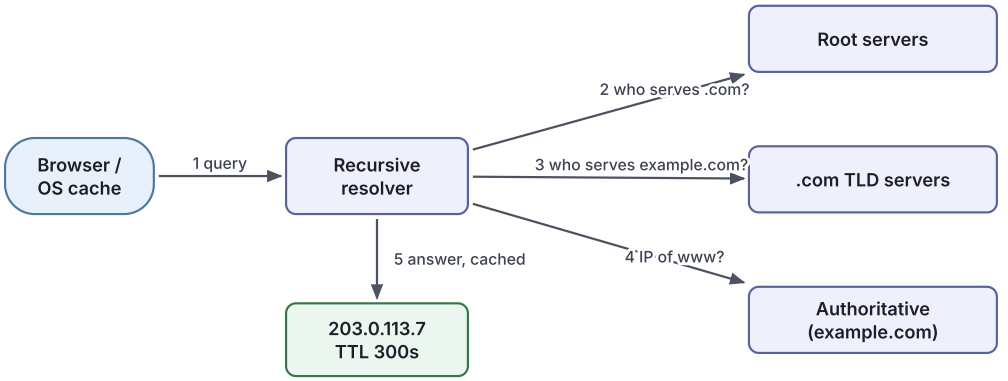


Figure 6.1 Resolving www.example.com. Every level caches answers for their TTL, so most lookups never leave the local resolver.

1. The browser checks its own cache, then the operating system's.
2. On a miss, it asks a **recursive resolver** (from your ISP, or a public one).
3. The resolver asks a **root server**, which points to the **TLD servers** for `.com`.
4. The TLD server points to the domain's **authoritative name servers**.
5. The authoritative server returns the record, and every level caches it for its **TTL**.

Common record types: **A** (name → IPv4), **AAAA** (→ IPv6), **CNAME** (alias to another name), **MX** (mail servers), **NS** (delegation), **TXT** (arbitrary text, used for verification).

> KEY IDEA
>
> **DNS is a load balancing and failover tool**
>
> Returning several A records spreads clients across servers (round-robin DNS). **GeoDNS** returns the nearest data centre's address. Health-checked DNS removes dead endpoints. The catch is the TTL: clients cache answers, so changes take effect only as caches expire. Short TTLs (30–60 s) allow faster failover but put more load on the DNS servers.

6.4 TCP vs. UDP

TCP gives you a reliable, ordered byte stream. It sets up a connection with a **three-way handshake** (SYN, SYN-ACK, ACK), numbers every byte, retransmits lost segments, slows down when the network is congested, and delivers data in order. Most application protocols

use it: HTTP/1.1, HTTP/2, database protocols.

UDP sends independent datagrams with no handshake, no ordering, no retransmission and no congestion control. It is fast and simple, and the application handles any loss. It suits real-time media, games, DNS queries and, increasingly, protocols that build their own reliability on top, such as **QUIC**.

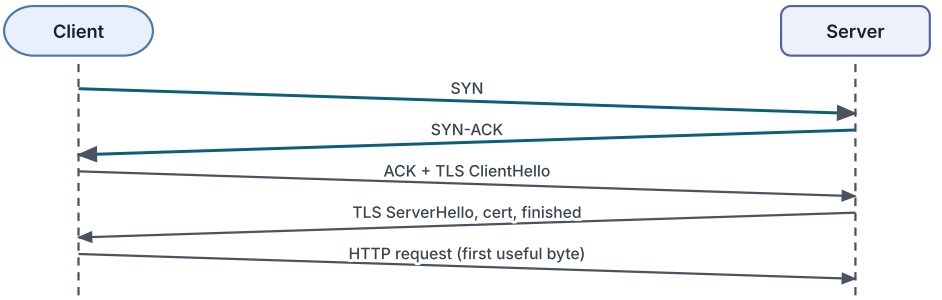


Figure 6.2 TCP needs one round trip before any data flows, and TLS 1.3 adds another. This is why connection reuse matters so much.

	TCP	UDP
Connection	Handshake required	None
Reliability	Retransmits lost data	Fire and forget
Ordering	Guaranteed	None
Overhead	Higher (headers, ACKs, state)	Minimal
Use cases	Web, APIs, databases, file transfer	Video calls, gaming, DNS, metrics

COMMON PITFALL

Connection setup is expensive

A fresh TCP + TLS connection to a server 100 ms away costs about 200 ms *before the first request byte is sent*. Systems that open a new connection per request waste huge amounts of time. Use **connection pooling** and **keep-alive**, and put servers close to users.

6.5 HTTP: the language of the web

HTTP is a request/response protocol. A request has a **method**, a **path**, **headers** and an optional **body**. A response has a **status code**, headers and a body.

Method	Meaning	Safe?	Idempotent?
GET	Read a resource	Yes	Yes
POST	Create / trigger an action	No	No
PUT	Replace a resource entirely	No	Yes
PATCH	Partially update	No	Not necessarily
DELETE	Remove a resource	No	Yes

Status code families: `2xx` success (200 OK, 201 Created, 204 No Content); `3xx` redirection (301 Moved Permanently, 304 Not Modified); `4xx` client error (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 429 Too Many Requests); `5xx` server error (500, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout).

HTTP versions

- **HTTP/1.1** sends one request at a time per connection, so browsers open about six connections per host. It suffers from *head-of-line blocking*: a slow response holds up the ones behind it.
- **HTTP/2** multiplexes many concurrent streams over one TCP connection, with binary framing and header compression. But one lost TCP packet still stalls *all* the streams.
- **HTTP/3** runs over **QUIC** (on UDP). Streams are independent, so a lost packet only stalls its own stream. Connection setup is faster (often 1 round trip, or 0 when resuming), and connections survive network changes such as Wi-Fi to cellular.

6.6 TLS and HTTPS

TLS encrypts the connection and authenticates the server. During the handshake, the server presents a **certificate** signed by a trusted **certificate authority**. Client and server use asymmetric cryptography to agree on a shared session key, then switch to fast symmetric encryption for the data. TLS 1.3 cut the handshake to one round trip. We return to TLS, mTLS and certificates in Chapter 25.

6.7 Proxies: forward and reverse

- A **forward proxy** sits in front of *clients* and makes requests on their behalf. It is used for corporate filtering, anonymity and caching.
- A **reverse proxy** sits in front of *servers* and receives requests on their behalf. It handles TLS termination, compression, caching, routing and load balancing. Nginx, Envoy,

HAProxy and cloud load balancers are all reverse proxies. Clients never see the servers behind them.

CHECK YOUR UNDERSTANDING

1. Why does DNS caching make failover slower, and what is the trade-off in choosing a TTL?

   See answer →

2. Name two situations where UDP is a better choice than TCP, and explain why.

   See answer →

3. What problem does HTTP/2 solve compared with HTTP/1.1, and what problem remains that HTTP/3 fixes?

   See answer →

4. Which HTTP methods are idempotent, and why does that matter for retries?

   See answer →

CHAPTER 7

↩ Contents

Load Balancing

A load balancer spreads incoming requests across a pool of servers. It is what makes horizontal scaling and redundancy possible, and it is usually the first box after DNS in any design.

7.1 What a load balancer does

1. **Distributes traffic** so that no single server is overwhelmed.
2. **Health-checks** backends and stops sending traffic to failed ones.
3. **Allows elastic scaling**: servers can be added and removed without clients noticing.
4. Often **terminates TLS**, compresses responses, adds headers, and enforces timeouts and limits.

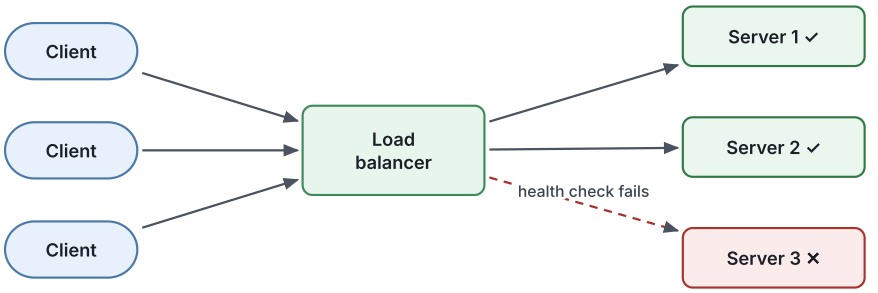


Figure 7.1 A load balancer with health checks. The failed server is removed from rotation automatically.

7.2 Layer 4 vs. layer 7

- An **L4 load balancer** routes on IP and port, without looking inside the packets. It is extremely fast and protocol-agnostic, but it can't route by URL or header.
- An **L7 load balancer** understands HTTP (or gRPC and so on). It can route `/api/*` to one pool and `/images/*` to another, split traffic for canary releases, add authentication, rewrite headers and retry failed requests. It does more work per request.

Large systems often stack them: an L4 tier spreads traffic over a fleet of L7 proxies, which then route to services.

7.3 Load balancing algorithms

Algorithm	How it works	Good for	Weakness
Round robin	Each server in turn	Identical servers, uniform requests	Ignores current load
Weighted round robin	Bigger servers get more turns	Mixed hardware	Static weights
Least connections	Server with fewest active connections	Long or variable requests	Needs connection tracking
Least response time	Fastest recent responses	Latency-sensitive services	Can oscillate
IP / consistent hash	hash(key) picks the server	Cache affinity, sticky routing	Uneven load with hot keys
Random / power of two choices	Pick 2 at random, use the less loaded	Large fleets	Slight randomness

> GOING DEEPER
>
> **The power of two random choices**
>
> Picking one server at random leads to surprisingly uneven load. Picking **two** at random and sending the request to the less loaded one cuts the worst-case load dramatically, with almost no coordination. Many modern proxies use this because it scales to huge fleets without a central view of every server's load.

7.4 Health checks

- **Passive** checks watch real traffic and eject servers that return errors or time out.
- **Active** checks probe an endpoint such as `/healthz` every few seconds.
- Distinguish **liveness** ("the process is running") from **readiness** ("it can serve traffic right now", with database connected and caches warm). Send traffic only to *ready* instances.

COMMON PITFALL

Deep health checks can cause cascading failure

If every server's health check also checks the shared database, a brief database hiccup makes *every* server fail its check at once. The load balancer then removes the entire fleet, turning a degraded situation into a total outage. Keep health checks focused on the instance itself, and make sure the load balancer "fails open" if every backend looks unhealthy.

7.5 Session persistence

Sometimes requests from one user must reach the same server, for example when a server holds an in-memory session or a WebSocket. Load balancers support this with cookie-based or IP-based **sticky sessions**. As Chapter 2 argued, prefer externalising state so you don't need this. When you do need affinity (as with WebSockets), use consistent hashing, which limits disruption when servers change.

7.6 Avoiding the load balancer as a SPOF

A single load balancer is itself a single point of failure. Common fixes:

- **Active-passive pair** with a floating virtual IP (VRRP / keepalived). The standby takes over the IP if the active one dies.
- **Active-active** load balancers behind DNS with several A records, or behind **anycast**, where the same IP is announced from many locations and the network routes each client to the nearest one.
- **Managed cloud load balancers**, which are internally distributed and redundant.

7.7 Global load balancing

For multi-region systems, traffic is steered *between* regions with GeoDNS or anycast, by geography (lowest latency), by health (avoid a failed region) and by capacity. Inside each region, local load balancers take over.

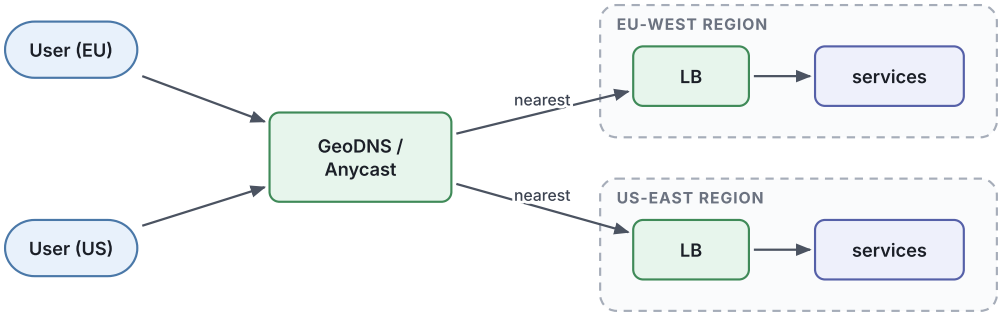


Figure 7.2 Global traffic management. GeoDNS or anycast picks a region, and regional load balancers pick a server.

CHECK YOUR UNDERSTANDING

1. When would you choose an L7 load balancer over an L4 one?

   See answer →

2. Requests to your service vary from 5 ms to 30 seconds. Why is round robin a poor choice, and what would you use?

   See answer →

3. Explain the difference between liveness and readiness checks.

   See answer →

4. How can the load balancer itself be made highly available?

   See answer →

CHAPTER 8

↩ Contents

Designing APIs

An API is a contract between a service and its clients. Internal APIs connect services. External APIs connect you to customers and partners. A good API is predictable, hard to misuse, efficient, and able to change without breaking the clients that depend on it.

8.1 REST

REST models a system as **resources** identified by URLs and manipulated with standard HTTP methods.

```
GET    /v1/users/42              -> fetch user 42
POST   /v1/users                 -> create a user (returns 201 + Location)
PATCH  /v1/users/42              -> update some fields
DELETE /v1/users/42              -> delete
GET    /v1/users/42/orders?status=shipped&limit=20
```

Good REST design means: plural nouns for collections, nesting only for true ownership, HTTP status codes used correctly, **statelessness** (each request carries its own auth and context), and support for caching through `ETag` / `Cache-Control`.

8.2 RPC and gRPC

RPC (remote procedure call) models the API as *functions*: `getUser(id)`, `chargeCard(amount)`. **gRPC** is the dominant modern framework. You define services and messages in **Protocol Buffers**, generate typed clients and servers in many languages, and communicate over HTTP/2 with compact binary encoding. It supports unary calls and client, server and bidirectional **streaming**.

```
service OrderService {
  rpc GetOrder (GetOrderRequest) returns (Order);
  rpc WatchOrder (GetOrderRequest) returns (stream OrderEvent);
}
```

8.3 GraphQL

GraphQL exposes a single endpoint and a typed schema. Clients send a query describing exactly the fields they want, possibly across several related entities, and get back exactly that shape. This avoids **over-fetching** (receiving fields you don't need) and **under-fetching** (needing several round trips).

```
query {
  user(id: 42) {
    name
    orders(last: 3) { id total items { name } }
  }
}
```

The costs: HTTP caching is harder (everything is a POST to one URL), a single query can be very expensive (you need query cost limits), and naive resolvers cause the **N+1 query problem**, usually fixed with batching (DataLoader).

8.4 Choosing a style

	REST	gRPC	GraphQL
Model	Resources	Procedures	Typed graph query
Encoding	JSON (text)	Protobuf (binary)	JSON
Transport	HTTP/1.1+	HTTP/2	HTTP
Strengths	Simple, cacheable, universal	Fast, typed, streaming	Flexible client-driven queries
Weaknesses	Over/under-fetching	Browser support needs a proxy; less human-readable	Caching, query cost control
Best for	Public APIs, CRUD	Internal service-to-service calls	Client apps with varied data needs

INTERVIEW TIP

A common, defensible answer

"Public API: REST, because it's universal and cacheable. Internal service-to-service calls: gRPC, for performance and strict contracts. If many different front-ends need different slices of the data, a GraphQL layer (or backend-for-frontend) on top of the services."

8.5 Pagination

Never return unbounded lists. Two main strategies:

- **Offset pagination**: `?limit=20&offset=40`. It is simple and lets you jump to a page, but it gets slow for deep offsets (the database scans and discards rows) and skips or duplicates items when data changes between pages.
- **Cursor (keyset) pagination**: `?limit=20&after=eyJpZCI6MTIzNH0`. The cursor encodes the position, for example the last seen `(created_at, id)`. It is fast at any depth and stable under inserts. It is the right default for feeds and large collections.

8.6 Idempotency keys

Networks fail *after* the server has done the work but *before* the client hears back. If the client retries a `POST /payments`, the customer may be charged twice. The fix is an **idempotency key**: the client generates a unique ID per logical operation and sends it in a header. The server stores the key with the result. If the same key comes again, it returns the stored result instead of repeating the action.

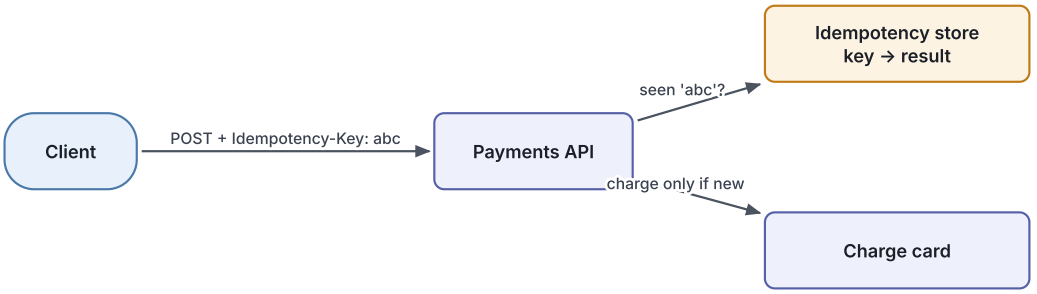


Figure 8.1 An idempotency key turns an unsafe retry into a harmless one.

8.7 Versioning and evolution

APIs outlive the code that first implemented them. Strategies include putting the version in the URL (`/v1/…`, the most visible), in a header (`Accept: application/vnd.example.v2+json`), or evolving *without* versions by making only **backward-compatible** changes: add optional fields, never remove or rename fields, never change their meaning. Deprecate with warnings and timelines before removing anything.

8.8 API gateways

An **API gateway** is a reverse proxy that acts as the single entry point for clients in front of many backend services. It centralises cross-cutting concerns:

- Routing requests to the right service
- Authentication and authorisation checks
- Rate limiting and quotas
- Request and response transformation, protocol translation (REST ↔ gRPC)
- Aggregation (one client call fans out to several services)
- Observability: logging, metrics, tracing
- Caching, compression, TLS termination

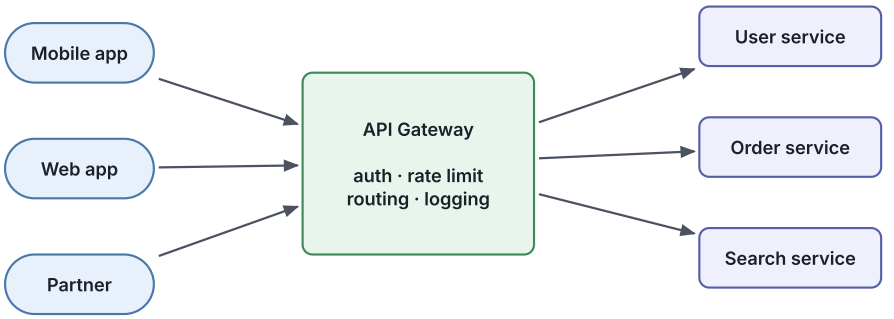


Figure 8.2 An API gateway gives clients one entry point and keeps cross-cutting concerns out of each service.

A related pattern, the **Backend for Frontend (BFF)**, gives each client type its own thin gateway (one for mobile, one for web), shaped to that client's needs.

COMMON PITFALL

The gateway that ate the business logic

Keep gateways thin. Once business rules creep into the gateway, it becomes a shared monolith that every team must change and deploy, which defeats the point of separate services.

CHECK YOUR UNDERSTANDING

1. Why is cursor-based pagination generally preferred over offset pagination for feeds?

See answer →

2. A client sends `POST /orders`, the server creates the order, but the response is lost. How do you make the client's retry safe?

   See answer →

3. What is the N+1 problem in GraphQL and how is it usually solved?

   See answer →

4. List four responsibilities an API gateway commonly takes on.

   See answer →

CHAPTER 9

↩ Contents

Rate Limiting and Authentication

Every public API needs to know two things about each request: **who is calling** (authentication), **what they may do** (authorisation), and whether they are **asking too often** (rate limiting). These controls protect both your users and your infrastructure.

9.1 Why rate limit?

- **Protect availability.** Stop a single client, whether buggy, abusive or malicious, from overwhelming shared resources.
- **Enforce fairness and business tiers.** Free users get 100 requests/minute, paid users 10,000.
- **Control cost,** especially for expensive operations or calls to paid third-party APIs.
- **Reduce abuse:** credential stuffing, scraping, spam.

A rejected request should receive **HTTP 429 Too Many Requests**, ideally with a `Retry-After` header and headers showing the remaining quota.

9.2 Rate limiting algorithms

Token bucket

A bucket holds up to *B* tokens and is refilled at *r* tokens per second. Each request takes one token. If the bucket is empty, the request is rejected. The bucket allows **bursts** up to *B* while enforcing an average rate of *r*. It is simple, memory-light (two numbers per client) and the most widely used algorithm.

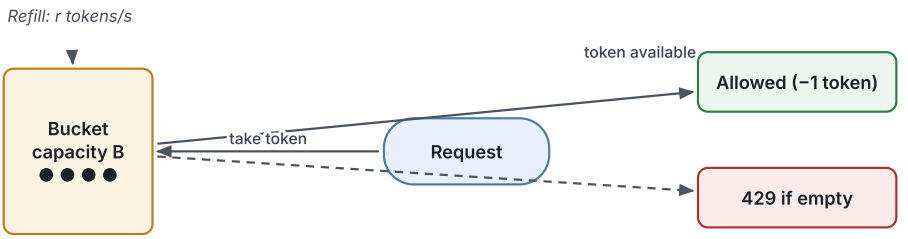


Figure 9.1 Token bucket: tokens drip in at a steady rate; each request spends one; bursts are allowed up to the bucket's capacity.

Leaky bucket

Requests enter a fixed-size queue that drains at a constant rate, like water leaking from a bucket. It smooths traffic into a perfectly steady flow, which protects downstream systems, but it adds queueing delay and doesn't allow bursts.

Fixed window counter

Count requests per client in fixed windows (for example, per calendar minute) and reject once the count passes the limit. It is trivial to implement with a counter that expires. Its flaw: a client can send the full limit at 12:00:59 and again at 12:01:00, which is **double the rate** across the window boundary.

Sliding window log

Store the timestamp of each request and count those in the last N seconds. It is exact, but the memory cost grows with the request rate.

Sliding window counter

A practical compromise: keep counters for the current and previous fixed windows, and weight the previous one by how much it overlaps the sliding window.

$$\text{estimate} = \text{current_count} + \text{previous_count} \times (1 - \text{elapsed_fraction_of_current_window})$$

It is accurate enough for nearly every use, with O(1) memory.

Algorithm	Bursts	Memory	Accuracy	Notes
Token bucket	Allowed up to B	O(1)	Good	Industry default
Leaky bucket	Smoothed out	O(queue)	Good	Steady output rate
Fixed window	2x at boundaries	O(1)	Weak	Simplest
Sliding log	Exact	O(requests)	Exact	Expensive at scale
Sliding window counter	Mostly prevented	O(1)	Very good	Great compromise

9.3 Distributed rate limiting

With 50 gateway instances, each needs a shared view of a client's usage. Common approaches:

- **Centralised counters** in Redis, updated atomically (a Lua script or `INCR` with expiry). This is accurate, but adds a network hop and makes Redis critical, so decide whether to fail open or closed if it is unreachable.
- **Local limits with periodic sync.** Each node enforces roughly limit ÷ N and reconciles occasionally. It is fast but approximate.
- **Sticky routing** of each client to a particular limiter node by consistent hashing.

TRADE-OFF

Fail open or fail closed?

If the rate-limit store is down, **failing open** (allowing requests) keeps the product working but drops protection. **Failing closed** (rejecting requests) keeps protection but causes an outage. Most consumer APIs fail open with a conservative local fallback limit. Security-critical endpoints such as login may fail closed.

9.4 Authentication vs. authorisation

- **Authentication (AuthN):** *who are you?* Verifying identity with passwords, keys, tokens, certificates or biometrics.
- **Authorisation (AuthZ):** *what are you allowed to do?* Checking permissions on a resource. A user authenticated as Alice may still not delete Bob's document.

9.5 Ways to authenticate API calls

API keys are long random secrets identifying a calling application. They are simple and good for server-to-server and developer APIs, but they identify an *application* rather than a user, and they are dangerous if leaked. Always send them over TLS, scope them, and make them easy to rotate.

Session cookies: after login, the server creates a session record and gives the browser a random session ID in a cookie (`HttpOnly`, `Secure`, `SameSite`). Each request looks the session up. They are easy to revoke (delete the record), but every request needs a lookup in a shared session store.

Tokens (JWT): a JSON Web Token is a signed, self-contained statement such as "user 42, role editor, expires 14:05". Any service holding the verification key can check it **without a database lookup**, which suits distributed systems. The catch is revocation: a stolen token stays valid until it expires. Hence the standard pattern of **short-lived access tokens** (5–15 minutes) plus **long-lived refresh tokens** that are stored server-side and can be revoked.

```
header.payload.signature
{"alg":"RS256"} . {"sub":"42","role":"editor","exp":1767225600} . <signature>
```

COMMON PITFALL

JWTs are signed, not encrypted

Anyone can base64-decode a JWT and read its payload. Never put secrets in it. Always verify the signature and the `exp`, `aud` and `iss` claims, and never accept `alg: none`.

9.6 OAuth 2.0 and OpenID Connect

OAuth 2.0 is a framework for *delegated authorisation*. It lets an application act on a user's behalf at another service ("Allow PhotoPrinter to read your photos?") without ever seeing the user's password. **OpenID Connect (OIDC)** adds an *identity* layer on top ("Sign in with…"), returning an ID token that describes who the user is.

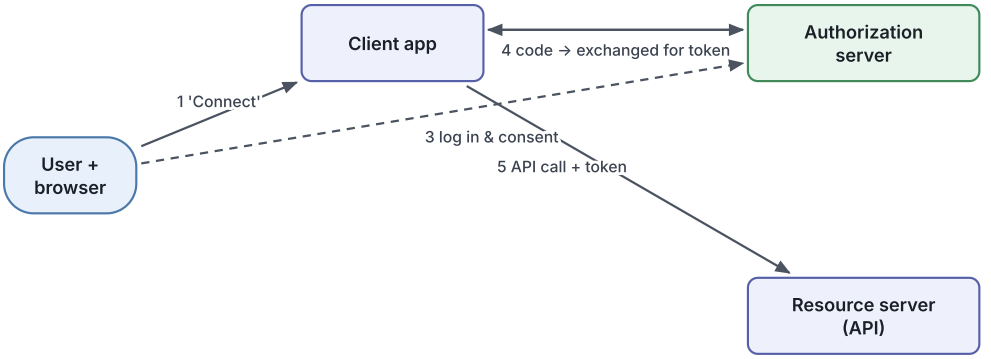


Figure 9.2 OAuth 2.0 authorisation-code flow (with PKCE). The app never sees the user's password, only a scoped token.

9.7 Authorisation models

- **RBAC (role-based):** permissions are attached to roles (viewer, editor, admin), and users get roles. Simple and auditable.
- **ABAC (attribute-based):** policies evaluate attributes of the user, resource and context ("managers can approve expenses under $5,000 in their own department during business hours"). Flexible, but more complex.
- **ReBAC (relationship-based):** permissions follow relationships in a graph ("can edit if a member of a team that owns the folder that contains the document"). This is how document-sharing systems work.

INTERVIEW TIP

Where to enforce

Authenticate at the edge (gateway) so that services receive a verified identity. But enforce **authorisation inside the service that owns the data**, because only it knows who owns which record. Missing object-level authorisation checks ("can user 42 read order 9001?") are among the most common API vulnerabilities.

CHECK YOUR UNDERSTANDING

1. A limit is 100 requests per minute using a fixed window counter. What is the maximum a client can send within any 60-second span, and why?

   See answer →

2. Why is a token bucket a popular choice for API rate limiting?

   See answer →

3. What is the main drawback of JWT access tokens, and how is it usually mitigated?

See answer →

4. Explain the difference between authentication and authorisation with an example.

See answer →

PART III

Moving Data

How services push updates to users in real time, how they talk to each other without waiting, and how caching makes almost everything faster.

CHAPTER 10

↩ Contents

Real-Time Communication

Plain HTTP is *client-initiated*: the server can only speak when spoken to. But chat messages, live scores, collaborative cursors and ride-tracking maps need the server to **push** updates as they happen. This chapter compares the techniques for doing that.

10.1 Short polling

The client asks every few seconds: "anything new?" It is trivial to build and works everywhere. But most responses are empty, so bandwidth and server capacity are wasted. Latency averages half the polling interval, and making the interval shorter multiplies the load.

10.2 Long polling

The client sends a request, and the server **holds it open** until there is data or a timeout (say 30 s) passes. Then the client immediately sends a new request. Updates arrive almost instantly, and far fewer requests come back empty. It still pays HTTP overhead per message, ties up a connection per waiting client, and needs care to avoid losing messages between requests.

10.3 Server-Sent Events (SSE)

The client opens one HTTP connection, and the server streams text events down it indefinitely (`Content-Type: text/event-stream`). Browsers reconnect automatically and resume from the last event ID. SSE is simple, works through most proxies and runs on plain HTTP, but it is **one-directional** (server → client) and text-only. It is ideal for notifications, live feeds, dashboards and streaming LLM output.

10.4 WebSockets

A WebSocket starts as an HTTP request with an `Upgrade` header. After that, the connection becomes a persistent, **full-duplex** channel: both sides can send messages at any time with minimal framing overhead. It is the standard choice for chat, multiplayer games, collaborative

editing and trading screens.

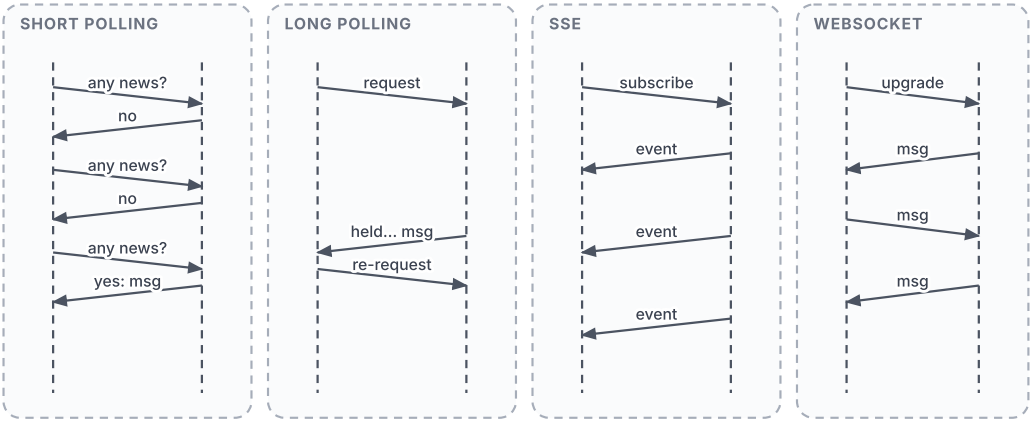


Figure 10.1 Four ways to get server updates to a client, from the most wasteful to the most interactive.

	Short polling	Long polling	SSE	WebSocket
Direction	Client pull	Client pull (held)	Server → client	Bidirectional
Latency	Up to interval	Near real-time	Real-time	Real-time
Overhead	High	Medium	Low	Lowest
Complexity	Trivial	Moderate	Low	Higher (stateful)
Best for	Rare updates	Fallback, simple push	Feeds, notifications	Chat, games, collaboration

10.5 WebRTC

For **peer-to-peer** audio, video and data (video calls, screen sharing), **WebRTC** lets browsers connect directly, usually over UDP. A *signalling server* (often using WebSockets) helps peers exchange connection details. **STUN** servers help peers discover their public addresses through NAT, and **TURN** servers relay traffic when a direct path is impossible. Group calls usually go through an **SFU** (selective forwarding unit), which receives each participant's stream and forwards it to the others.

10.6 Scaling persistent connections

WebSockets and SSE make servers **stateful**: each connection lives on one specific server. A million concurrent users means a million open connections, spread over a fleet of *connection servers* (a single tuned server can hold hundreds of thousands of mostly idle connections).

The central problem: user A is connected to server 3, and user B to server 17. How does A's message reach B?

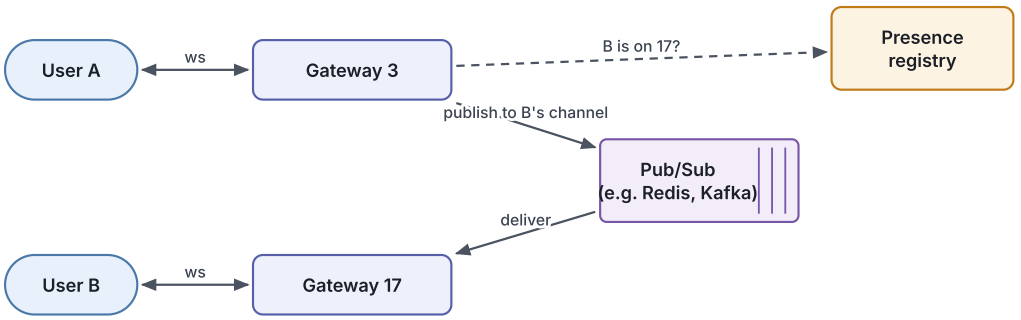


Figure 10.2 Routing messages between users on different connection servers through a pub/sub backbone, with a registry recording who is connected where.

The usual answer is a **pub/sub backbone**. Each connection server subscribes to channels for the users (or rooms) connected to it, and a sender's server publishes to the recipient's channel. Alternatively, a **presence registry** maps each user to their server, so messages can be routed directly.

COMMON PITFALL

Reconnect storms

When a connection server restarts or a deploy rolls out, hundreds of thousands of clients reconnect at once and can overwhelm the remaining servers or the auth service. Use randomised reconnect backoff on clients, drain connections gradually during deploys, and size for reconnect spikes, not just steady state.

INTERVIEW TIP

Choose the simplest technique that meets the need

"Notifications only flow server → client, so I'll use SSE: plain HTTP, automatic reconnects. For chat, where clients also send frequently, I'll use WebSockets, with long polling as a fallback for restrictive networks."

CHECK YOUR UNDERSTANDING

1. Why is SSE often a better choice than WebSockets for a live news feed?

   See answer →

2. In a chat system with 200 WebSocket servers, how does a message from a user on server 12 reach a recipient on server 140?

   See answer →

3. What are STUN and TURN servers used for in WebRTC?

   See answer →

CHAPTER 11

↩ Contents

Message Queues and Publish/Subscribe

Not every piece of work needs to finish before you reply to the user. When someone uploads a video, you can confirm the upload in milliseconds and transcode it over the next few minutes. **Asynchronous messaging** separates the producers of work from its consumers, in time, in speed and in deployment.

11.1 Why asynchronous messaging?

- **Faster responses:** slow work moves off the user's request path.
- **Load levelling:** a queue absorbs traffic spikes, and consumers work through the backlog at their own pace instead of being flattened.
- **Decoupling:** producers don't know or care who consumes their messages, or whether consumers are up right now.
- **Reliability:** if a consumer crashes, messages wait safely in the queue and are retried.
- **Fan-out:** one event can trigger many independent reactions.

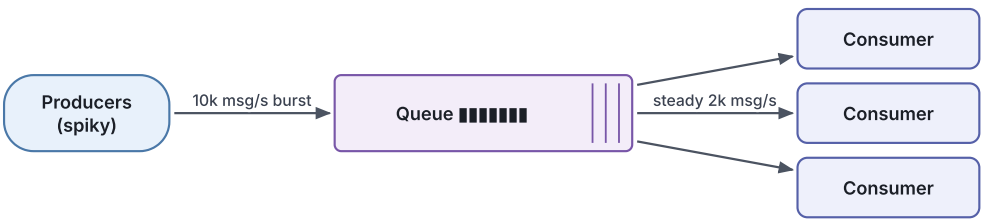


Figure 11.1 A queue absorbs a traffic spike and lets consumers work through the backlog at a steady rate.

11.2 Point-to-point queues vs. publish/subscribe

Point-to-point (work queue): each message is handled by **exactly one** of the competing consumers. You add consumers to increase throughput. Typical uses: sending emails, processing images, running jobs.

Publish/subscribe: each message is delivered to **every subscriber** (each subscription gets its own copy). An `OrderPlaced` event can at once update inventory, send a confirmation email, notify analytics and start fraud checks, and none of those services know about each other.

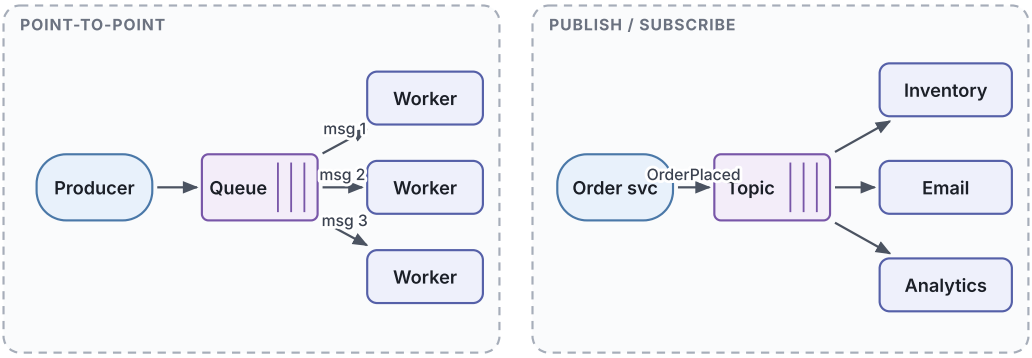


Figure 11.2 Left: competing consumers share the work. Right: every subscriber group receives every event.

11.3 Two families of brokers

Traditional message brokers (RabbitMQ, ActiveMQ, Amazon SQS) track the delivery of each message. A message is removed once a consumer acknowledges it. They offer rich routing (exchanges, topics, priorities) and per-message acknowledgement, and are well suited to task queues.

Log-based brokers (Apache Kafka, Pulsar, Kinesis) append messages to a durable, **partitioned, ordered log** and keep them for a retention period (days or forever). Consumers track their own **offset** (position) in each partition. Several consumer groups can read the same log independently, and a consumer can **replay** history by rewinding its offset.

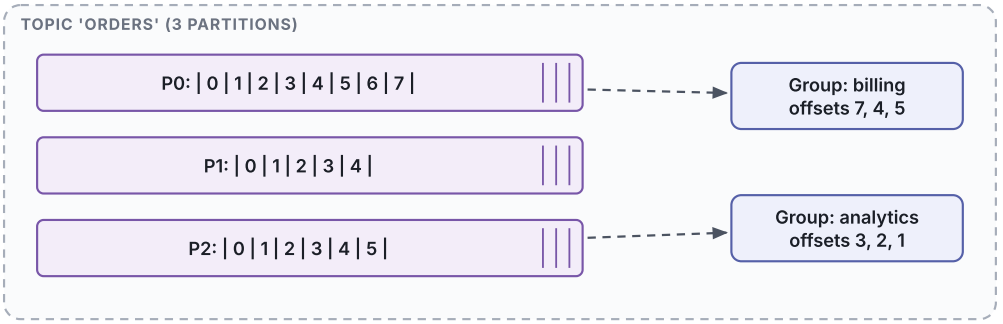


Figure 11.3 A partitioned log. Each consumer group tracks its own offset per partition, and messages stay after they are read.

Ordering in Kafka is guaranteed *within a partition*, not across them. Messages with the same **key** (for example, `order_id` or `user_id`) always go to the same partition, so all events for one entity stay in order. Parallelism equals the number of partitions: within one consumer group, each partition is read by at most one consumer.

	Message broker (RabbitMQ / SQS)	Log (Kafka)
Message after consumption	Deleted	Retained for a set period
Replay	No	Yes, rewind the offset
Ordering	Per queue (weak with many consumers)	Per partition (strong)
Throughput	High	Very high (sequential disk I/O)
Consumer state	Tracked by broker	Consumer tracks its offset
Best for	Task queues, complex routing	Event streams, event sourcing, pipelines

11.4 Delivery guarantees

- **At-most-once:** a message may be lost but is never duplicated (acknowledge before processing). Acceptable for metrics where a gap doesn't matter.
- **At-least-once:** a message is never lost but may be **delivered more than once** (acknowledge after processing; a crash before the ack causes redelivery). This is the practical default.
- **Exactly-once:** each message affects the result exactly once. True exactly-once delivery across arbitrary systems is impossible in general. What systems provide is *exactly-once*

processing within a closed system (Kafka transactions) or *effectively-once* behaviour through idempotent consumers.

KEY IDEA

At-least-once delivery + idempotent consumers = effectively once

Design every consumer so that handling the same message twice has the same effect as handling it once. Track processed message IDs, use upserts instead of inserts, or use conditional updates ("set status=SHIPPED where status=PACKED"). This is how real systems get correct results from imperfect delivery.

11.5 Dead-letter queues and poison messages

A **poison message** is one that always fails, perhaps because it is malformed or hits a bug. Without protection, it is retried forever and blocks the queue. After *N* failed attempts, move it to a **dead-letter queue (DLQ)** for inspection and later replay. Always monitor DLQ depth.

11.6 Backpressure

If producers outpace consumers indefinitely, queues grow without limit until memory, disk or latency targets give out. **Backpressure** is any way of pushing back: bounded queues that reject or block producers, consumers that scale automatically on queue depth, or shedding low-priority work. *Queue depth* and *consumer lag* are the two most important metrics for any asynchronous system.

COMMON PITFALL

A queue is not a magic scalability wand

A queue smooths bursts, but if the *average* arrival rate exceeds the *average* processing rate, the backlog grows forever. Queues buy time. They don't buy capacity.

CHECK YOUR UNDERSTANDING

1. When would you choose publish/subscribe over a point-to-point queue?

   See answer →

2. How does Kafka keep all events for a single order in order while still processing in parallel?

   See answer →

3. Your consumer receives the message "charge customer $50" twice. What delivery guarantee causes this, and how do you prevent a double charge?

See answer →

4. What is a dead-letter queue for?

See answer →

CHAPTER 12

↩ Contents

Caching

A cache is a small, fast store that keeps copies of data that is expensive to fetch or compute. Caching is the most powerful performance tool in system design, and the source of some of its most subtle bugs. As the old joke says, there are only two hard things in computer science: cache invalidation and naming things.

12.1 Why caching works

Data access is rarely uniform. A small fraction of items (a celebrity's profile, today's front page, a popular product) receives most of the traffic. This **skew** means a cache holding just the hottest few percent of the data can serve the large majority of reads.

Effective latency = hit_rate × cache_latency + (1 − hit_rate) × source_latency

With a 1 ms cache, a 50 ms database and a 95% hit rate: 0.95 × 1 + 0.05 × 50 ≈ **3.5 ms** on average, and the database sees just 5% of the read traffic.

12.2 Where caches live

Caches appear at every layer, and a single request may pass through several:

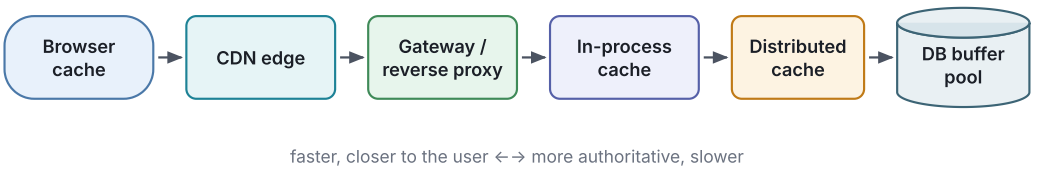


Figure 12.1 Caching layers from the user's device to the database. Each hit saves every hop to its right.

- **Client / browser:** HTTP caching via `Cache-Control`, `ETag`. It costs nothing, but you can't purge it.
- **CDN:** caches content at edge locations around the world (Chapter 16).

- **Reverse proxy / gateway:** caches whole responses.
- **In-process (local) cache:** a hash map in the application's memory. The fastest option, but each instance has its own copy, so instances can disagree and the cache is lost on restart.
- **Distributed cache:** Redis or Memcached, a shared cluster. Consistent across app instances, a network hop away.
- **Database:** buffer pools and query caches keep hot pages in memory.

12.3 Caching strategies

Cache-aside (lazy loading)

The application manages the cache explicitly. This is the most common pattern.

```
def get_user(id):
    user = cache.get(f"user:{id}")
    if user is None:                        # miss
        user = db.query("SELECT ... WHERE id = ?", id)
        cache.set(f"user:{id}", user, ttl=300)
    return user

def update_user(id, fields):
    db.update(id, fields)
    cache.delete(f"user:{id}")              # invalidate; next read refills
```

Only requested data is cached, and a cache outage degrades performance without breaking correctness. The first read of each item is slow (a miss), and stale data is possible in the window between a database write and the cache invalidation.

Read-through

The cache itself loads missing data from the database, so the application only talks to the cache. It is cleaner application code, but needs a cache layer that supports loaders.

Write-through

Every write goes to the cache **and** the database synchronously before it is acknowledged. The cache is always fresh, but writes are slower, and you cache data that may never be read.

Write-behind (write-back)

Writes go to the cache and are acknowledged immediately. The cache flushes them to the database asynchronously, often in batches. Write latency is very low and write-heavy workloads are absorbed well, but **data is lost if the cache fails before flushing**, and the logic is more complex.

Write-around

Writes go straight to the database and skip the cache, and reads fill it on a miss. This avoids filling the cache with data that is written once and rarely read.

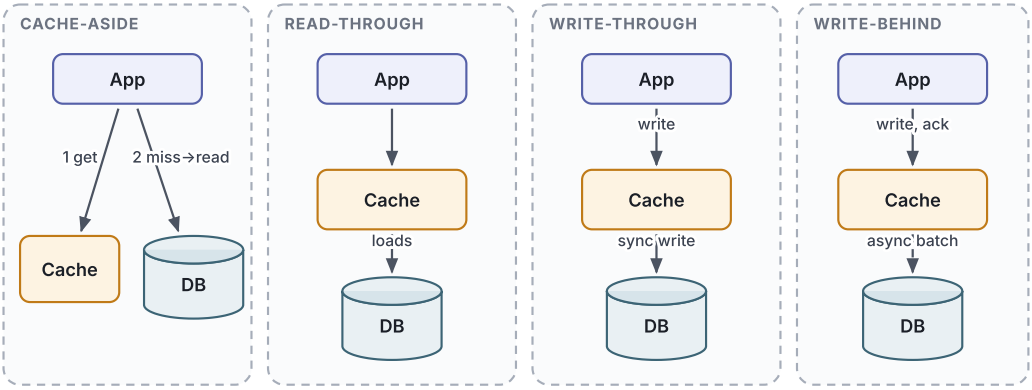


Figure 12.2 The four main caching strategies, side by side.

Strategy	Read latency	Write latency	Freshness	Risk
Cache-aside	Miss penalty	DB speed	Stale window	Low
Read-through	Miss penalty	DB speed	Stale window	Low
Write-through	Fast (hot)	Slower (two writes)	Fresh	Low
Write-behind	Fast	Very fast	Fresh in cache	Data loss
Write-around	Miss after write	DB speed	Fresh on refill	Low

12.4 Eviction policies

A cache is smaller than its source, so when it is full something must go:

- **LRU (least recently used):** evict whatever hasn't been touched longest. The sensible default.
- **LFU (least frequently used):** evict the least popular items. Better for stable popularity, slower to adapt to changes.
- **FIFO:** evict the oldest insertion. Simple, but often poor.
- **TTL-based:** items expire after a fixed time, whatever their use.
- Modern variants (W-TinyLFU, ARC) combine recency and frequency and resist one-off scans that would otherwise flush the cache.

12.5 Invalidation: the hard part

When the source data changes, cached copies become stale. The options:

1. **TTL expiry.** Accept staleness for up to *T* seconds. Simple and robust. Choose *T* by how stale the data may be.
2. **Explicit invalidation on write.** Delete (not update) the cache entry when the data changes.
3. **Event-driven invalidation.** Publish change events (or tail the database's change log, CDC) and let cache nodes evict affected keys.
4. **Versioned keys.** Embed a version in the key (`product:42:v17`), so a change produces a new key and old entries simply age out.

> **COMMON PITFALL**
>
> **Update-then-invalidate race**
>
> Thread A reads the old value from the database (a miss). Thread B updates the database and deletes the cache key. Thread A then writes its *old* value into the cache, which now holds stale data until the TTL expires. Mitigations: always set a TTL as a safety net, prefer delete over set on writes, use versioned or compare-and-set writes, or delay a second invalidation slightly ("delayed double delete").

12.6 Cache failure modes

Cache stampede (thundering herd)

A hot key expires, and thousands of concurrent requests miss at once and all hit the database to recompute the same value.

Fixes: *request coalescing* or a per-key lock, so only one request recomputes while the others wait; *early probabilistic refresh* before expiry; serving the stale value while one background job refreshes it (*stale-while-revalidate*); and adding **jitter** to TTLs so that keys don't expire in synchronised waves.

Cache penetration

Requests for keys that **don't exist** (for example, random IDs from an attacker) always miss and always reach the database. **Fixes:** cache negative results ("not found") with a short TTL, validate inputs, and put a **Bloom filter** of valid keys in front (Chapter 21).

Cache avalanche

Many keys expire together, or the whole cache cluster restarts, and the full read load lands on the database. **Fixes:** TTL jitter, warming the cache before taking traffic, replicated or highly available cache clusters, and rate limiting or circuit breakers in front of the database.

Hot keys

One key (a viral post) receives so much traffic that the single cache node holding it saturates. **Fixes:** replicate hot keys across nodes (read from a random replica with a suffixed key like `post:9:r3`), add a small in-process cache in front of the distributed cache, or split the value.

12.7 Distributed caching

Redis and Memcached clusters partition keys across nodes, usually with **consistent hashing** (Chapter 15), so that adding or removing a node remaps only a small fraction of keys. Redis adds rich data structures (lists, sorted sets for leaderboards, hashes, streams), optional persistence and replication. Memcached is a simpler, multi-threaded, pure key-value cache.

INTERVIEW TIP

What to say about a cache in a design

Cover five points: **what** is cached (and its key format), **where** (layer), **strategy** (cache-aside and so on), **invalidation/TTL**, and **failure handling** (stampedes, a cold start). "I'll cache rendered user profiles in Redis with cache-aside, keyed by `profile:{id}`, TTL 10 minutes with jitter, invalidated on profile update, and with request coalescing to prevent stampedes on celebrity profiles."

CHECK YOUR UNDERSTANDING

1. A cache has a 90% hit rate, 2 ms cache latency and 80 ms database latency. What is the average read latency? What if the hit rate rises to 99%?

   See answer →

2. Why do many systems *delete* the cache entry on update instead of writing the new value into the cache?

   See answer →

3. What is a cache stampede and name two mitigations.

   See answer →

4. When is write-behind caching appropriate, and what is its main risk?

See answer →

5. Attackers request millions of random product IDs that don't exist, and your database is overloaded. What is this called and how do you defend against it?

See answer →

PART IV

Storing Data

Choosing the right database, making queries fast, keeping copies in sync, splitting data across machines, and storing files at petabyte scale.

CHAPTER 13

↩ Contents

Choosing a Database

The database is usually the hardest part of a system to change later. Application servers are interchangeable and caches can be rebuilt, but data lasts, and migrating it is slow and risky. Choose with care.

13.1 Relational databases

Relational databases (PostgreSQL, MySQL, SQL Server, Oracle) store data in **tables** of rows and columns with a fixed **schema**, link tables through **foreign keys**, and query them with **SQL**, including **joins** across tables.

Their great strength is **ACID transactions**:

- **Atomicity:** a transaction's changes all happen or none do. A transfer never debits one account without crediting the other.
- **Consistency:** a transaction moves the database from one valid state to another, respecting constraints such as uniqueness and foreign keys. (This is a different "C" from CAP.)
- **Isolation:** concurrent transactions don't see each other's partial work. They behave as if run in some order, to a degree set by the *isolation level*.
- **Durability:** once committed, changes survive crashes. They are written to a write-ahead log on disk before the commit is acknowledged.

Isolation levels and anomalies

Level	Prevents	Still allows
Read uncommitted	(almost nothing)	Dirty reads
Read committed	Dirty reads	Non-repeatable reads, lost updates
Repeatable read / snapshot	Non-repeatable reads	Write skew (in snapshot isolation)
Serializable	All of the above	(none: behaves like serial execution)

Higher isolation is safer but has lower concurrency. Many databases default to *read committed*, so know what your database actually guarantees.

WORKED EXAMPLE

A lost update

Two requests read a counter (value 10) at the same time, each add 1 and each write 11. One increment is lost. Fixes: an atomic update (`UPDATE … SET n = n + 1`), a row lock (`SELECT … FOR UPDATE`), optimistic concurrency (`UPDATE … WHERE version = 7` , retrying if no row matched), or serializable isolation.

13.2 NoSQL families

"NoSQL" covers several data models, each giving up some relational features to gain scale, flexibility or performance for a particular access pattern.

Key-value stores (Redis, DynamoDB, Riak) work like a giant hash map: `get(key)` and `put(key, value)` . They are extremely fast and scale horizontally by partitioning on the key. They suit sessions, caches, carts, feature flags, and anything you look up by ID.

Document stores (MongoDB, Couchbase, Firestore) hold JSON-like documents with nested structure and flexible schemas. A whole aggregate (a user with their addresses and preferences) is stored and fetched together, so no joins are needed. They suit content, catalogues and user profiles.

Wide-column stores (Cassandra, HBase, Bigtable, ScyllaDB) hold rows identified by a partition key, with a sorted, sparse set of columns within each partition. They are built for massive write throughput and huge datasets spread across many nodes. They suit time series, messaging, activity logs and IoT data, where you query by partition key and a range within it.

Graph databases (Neo4j, Neptune) store nodes and edges as first-class citizens, so following relationships is cheap. They suit social graphs, recommendations, fraud rings and knowledge graphs: queries like "friends of friends who like X".

Other specialists: **time-series databases** (InfluxDB, TimescaleDB) for metrics; **search engines** (Elasticsearch, OpenSearch) for full-text search with inverted indexes; **vector databases** for similarity search on embeddings; **NewSQL** (Spanner, CockroachDB) for SQL and ACID with horizontal scaling.

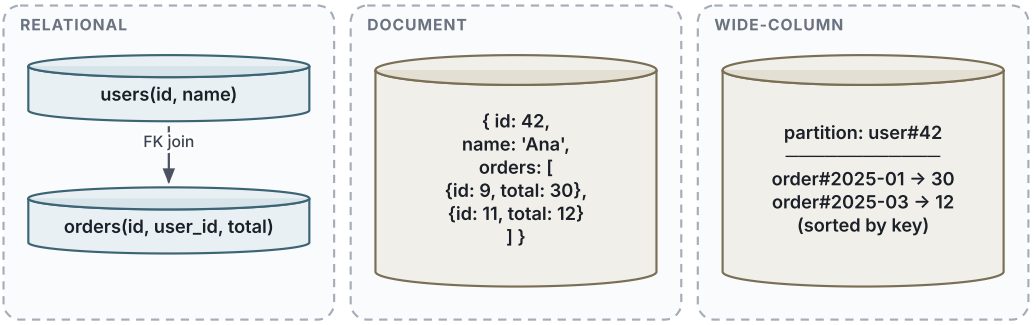


Figure 13.1 The same "user with orders" data in three models.

13.3 SQL vs. NoSQL: how to decide

Choose relational when…	Choose NoSQL when…
Data is relational, with many-to-many links	Access is by key or a simple pattern
You need multi-row ACID transactions	You need huge horizontal write scale
Queries are ad hoc and change over time	Queries are known and fixed in advance
Strong consistency and constraints matter	Flexible or rapidly evolving schemas
Data fits on one large node (or a few shards)	Data is many TB to PB across regions

KEY IDEA

Model around your queries

In relational design, you normalise the data and let queries join it. In NoSQL design, especially wide-column and key-value, you start from the **access patterns** and shape (often duplicate) the data so that each query reads one partition. List your queries *before* choosing a store.

INTERVIEW TIP

The default answer

"I'll start with PostgreSQL: it's mature, gives me transactions and flexible queries, and scales vertically a long way, plus read replicas. If a specific workload outgrows it, such as a 1M-writes/sec activity log, I'll move that workload to a store designed for it, like Cassandra." Using several stores in one system, each suited to its job, is called **polyglot persistence**.

13.4 Normalisation vs. denormalisation

Normalisation stores each fact once (user names live only in `users`), which avoids update anomalies but needs joins. **Denormalisation** copies data to where it is read (each `order` row also stores `user_name`), making reads fast and join-free at the cost of extra storage and the work of keeping copies in sync. Read-heavy systems at scale denormalise deliberately, often keeping the copies up to date asynchronously via events.

CHECK YOUR UNDERSTANDING

1. Explain each letter of ACID in one sentence.

   See answer →

2. Which database type would you pick for: (a) a shopping cart; (b) friend-of-friend recommendations; (c) billions of IoT sensor readings per day; (d) an accounting ledger?

   See answer →

3. Two concurrent requests each increment a stock counter from 10, and the result is 11 instead of 12. Name this anomaly and two ways to prevent it.

   See answer →

CHAPTER 14

↩ Contents

Indexing and Replication

This chapter covers two techniques every database relies on: **indexes**, which make reads fast, and **replication**, which keeps copies of data for availability and read scale.

14.1 Why indexes?

Without an index, finding rows where `email = 'ana@x.com'` means a **full table scan**: reading every row. With 100 million rows, that takes seconds. An index is an auxiliary data structure that maps column values to row locations, turning a scan into a lookup that touches a handful of pages.

14.2 B-tree indexes

The **B-tree** (strictly, the B+tree) is the default index in nearly every relational database. It is a balanced tree of fixed-size pages, each holding hundreds of sorted keys, so even billions of rows need only 3–4 levels. Leaf pages are linked, which makes **range scans** (`WHERE created_at BETWEEN …`) efficient.

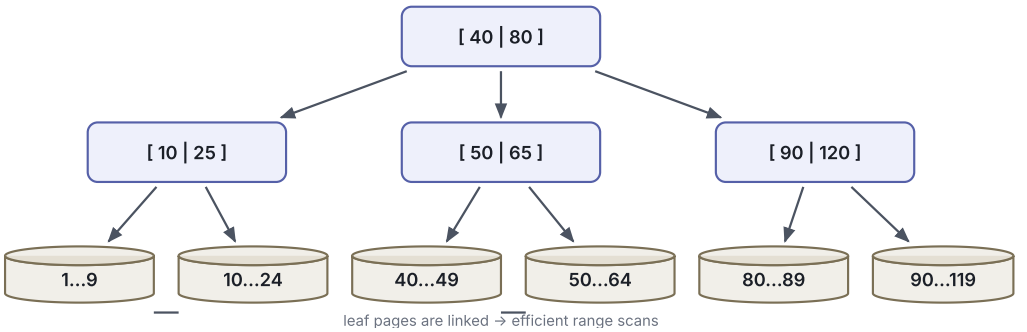


Figure 14.1 A B+tree. Each page holds many keys, so a lookup in a billion rows touches only 3–4 pages. Linked leaves make range scans fast.

14.3 LSM trees

Log-structured merge trees (used by Cassandra, RocksDB, LevelDB, HBase) optimise for *writes*. A write goes to an in-memory sorted buffer (the **memtable**) and to an append-only log for durability. When the memtable fills, it is flushed to disk as an immutable sorted file (an **SSTable**). Background **compaction** merges SSTables and throws away overwritten values. Reads check the memtable, then the SSTables from newest to oldest, helped by a **Bloom filter** per file that skips files which can't contain the key.

	B-tree	LSM tree
Writes	In-place page updates (random I/O)	Sequential appends (fast)
Reads	Fast, predictable	May check several files (Bloom filters help)
Write amplification	Moderate	From compaction
Space	Some fragmentation	Better compression
Typical use	OLTP relational databases	Write-heavy, large-scale stores

14.4 Other index types

- **Hash index:** O(1) equality lookups. No range queries.
- **Composite index** on `(a, b, c)` : serves queries filtering on `a`, `a,b` or `a,b,c` (the *leftmost prefix* rule), but not on `b` alone.
- **Covering index:** contains every column a query needs, so the table itself is never read ("index-only scan").
- **Inverted index:** maps each *term* to the list of documents containing it. It is the core of full-text search.
- **Geospatial index** (R-tree, geohash, S2): "find points near me" (Chapter 21).

TRADE-OFF

Every index taxes every write

Each index must be updated on every insert, update and delete, and it takes disk and memory. Index the columns you actually filter, join and sort on, guided by your real query patterns and the database's `EXPLAIN` plans, not every column "just in case".

14.5 Replication: why keep copies?

1. **High availability:** if one node dies, another has the data.
2. **Read scaling:** spread reads across replicas.
3. **Lower latency:** place replicas near users in several regions.
4. **Disaster recovery:** survive the loss of a whole data centre.

14.6 Single-leader replication

One node, the **leader** (primary), accepts all writes. It sends its change stream to **followers** (replicas), which apply the changes in order. Reads can go to any replica.

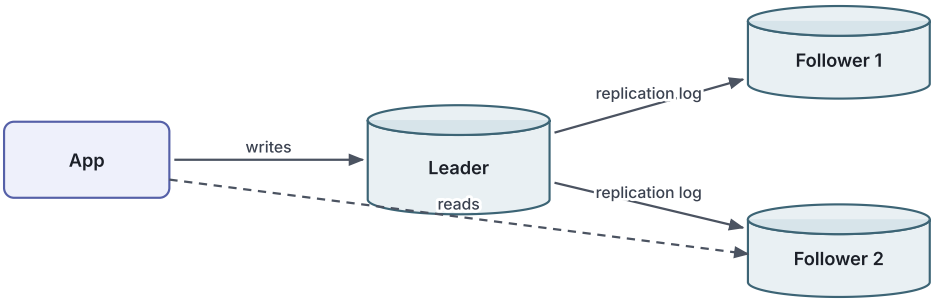


Figure 14.2 Single-leader replication: writes go to the leader and flow to followers, and reads spread across all of them.

Synchronous vs. asynchronous replication:

- **Synchronous:** the leader waits for the follower to confirm before acknowledging the write. The follower is guaranteed up to date, but a slow or dead follower blocks writes.
- **Asynchronous:** the leader acknowledges immediately and sends the change later. Writes are fast, but if the leader dies, recently acknowledged writes that never reached a follower are **lost**.
- **Semi-synchronous:** one follower is synchronous and the rest asynchronous. A common compromise: at least two nodes always have every write.

Replication lag and its anomalies

Asynchronous followers trail the leader, usually by milliseconds, sometimes by seconds or minutes under load. This produces the anomalies from Chapter 4:

- **Reading your own write:** a user saves a comment and then reads from a lagging replica, and the comment is "gone". *Fix:* read the user's own recent data from the leader, or track

the user's last write position and read only from replicas that have caught up to it.

- **Time going backwards:** two successive reads hit replicas with different lag. *Fix:* pin each user to one replica (monotonic reads).

Failover

When the leader fails, a follower must be **promoted**: detect the failure (usually by timeout), choose the most up-to-date follower, and reconfigure clients and the other followers. The dangers: losing asynchronous writes that never replicated, *split brain* if the old leader comes back still thinking it is in charge, and false failovers caused by temporary slowness. Consensus-based systems (Chapter 19) make this much safer.

14.7 Multi-leader replication

Several nodes, typically one per region, accept writes and replicate to each other. Writes are fast and local in every region, and each region keeps working when others are cut off. The great difficulty is **write conflicts**: two regions change the same record at the same time.

Conflict-resolution strategies:

- **Last-write-wins (LWW):** the highest timestamp wins. Simple, but silently drops the other write, and clock skew makes it unreliable.
- **Application-level merging:** keep both versions and resolve with business logic, or ask the user.
- **CRDTs** (conflict-free replicated data types): data structures such as counters, sets and text sequences that merge automatically and deterministically. They are used in collaborative editors and distributed counters.
- **Avoid conflicts:** route all writes for a given record to one "home" region.

14.8 Leaderless replication

Any replica accepts reads and writes (Dynamo-style systems such as Cassandra and Riak). The client, or a coordinator, sends each write to all N replicas and waits for W acknowledgements. Reads query R replicas and take the newest version. With $\mathbf{R + W > N}$, reads and writes overlap on at least one up-to-date node.

Stale replicas catch up through **read repair** (a read that sees an outdated replica writes the newer value back), **anti-entropy** (background comparison, often using Merkle trees, Chapter 21) and **hinted handoff** (writes meant for a down node are held by another node and passed on when it recovers).

	Single-leader	Multi-leader	Leaderless
Writes accepted by	Leader only	Any leader	Any replica
Write conflicts	None	Yes, must resolve	Yes, must resolve
Write availability	Lost during failover	High	High
Consistency	Easiest to make strong	Eventual	Tunable via quorums
Examples	PostgreSQL, MySQL	Multi-region setups, CouchDB	Cassandra, DynamoDB, Riak

CHECK YOUR UNDERSTANDING

1. Why can a composite index on `(country, city)` speed up `WHERE country='FR'` but not `WHERE city='Paris'`?

   See answer →

2. Why are LSM trees better suited to write-heavy workloads than B-trees?

   See answer →

3. A user updates their profile and immediately reloads, but sees the old version. What happened and how do you fix it?

   See answer →

4. What is the risk of asynchronous replication during failover?

   See answer →

5. Give two ways multi-leader systems resolve write conflicts, and a weakness of last-write-wins.

   See answer →

CHAPTER 15

↩ Contents

Partitioning and Sharding

Replication copies the *same* data to several nodes. It helps reads and availability but not *write* capacity or *storage* capacity, since every node still holds everything. When the data or write load outgrows one machine, you must **partition** it: split it so that each node holds only a subset. In databases this is called **sharding**.

15.1 Vertical vs. horizontal partitioning

- **Vertical partitioning** splits by *columns* or *features*: user profiles in one database, orders in another, rarely used large columns in a separate table. It is easy at first, but each piece can still outgrow its machine.
- **Horizontal partitioning (sharding)** splits by *rows*: users 1–10M on shard A, 10–20M on shard B. Every shard has the same schema and holds a fraction of the rows.

15.2 Choosing a shard key

The **shard key** (partition key) decides which shard each row lives on. It is the most important decision in a sharded design, and it is hard to change later. A good shard key:

1. **Spreads data and load evenly** (high cardinality, no dominant values).
2. **Keeps related data together**, so common queries hit **one shard**.
3. **Doesn't produce hotspots** over time.

COMMON PITFALL

Monotonic keys create hotspots

Sharding by a timestamp or auto-increment ID with *range* partitioning sends every new write to the shard holding the latest range. One shard burns while the rest sit idle. Hash the key, or prefix it with something well spread, such as `user_id`.

15.3 Partitioning strategies

Range partitioning

Each shard owns a contiguous key range (A–F, G–M, …). Range scans are efficient and the routing is easy to understand. But hotspots are likely when access is skewed or keys are sequential, and ranges need rebalancing as data grows.

Hash partitioning

The shard is `hash(key) mod N`. Keys spread evenly, and hotspots from sequential keys disappear. Range queries become scatter-gather across all shards. And with naive `mod N`, **changing N moves almost every key**, which is a disaster for caches and databases alike.

Directory (lookup) partitioning

A lookup service maps each key (or key group) to its shard. It is very flexible: individual tenants can be moved and big customers given dedicated shards. The directory is extra infrastructure and must be fast and highly available.

Geographic partitioning

Data is partitioned by user region (EU users in EU shards). This reduces latency and can meet data-residency laws.

15.4 Consistent hashing

Consistent hashing fixes the "change N, move everything" problem. Imagine the hash space as a **ring** (0 to $2^{32}-1$). Each node is hashed to positions on the ring. Each key is hashed onto the ring and belongs to the **first node clockwise** from it.

When a node is **added**, it takes over only the keys between itself and its predecessor, about 1/N of all keys. When a node is **removed**, only its keys move, to the next node clockwise. Everything else stays put.

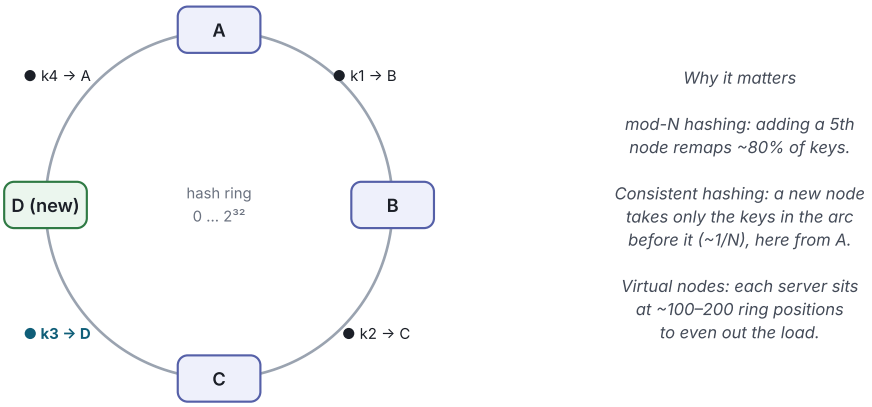


Figure 15.1 A consistent hash ring. Each key belongs to the next node clockwise. Adding node D moves only the keys in the arc between C and D.

Virtual nodes: with only a few physical positions, the arcs are uneven and some nodes get much more data. Giving each physical node many positions on the ring (*virtual nodes*) evens out the distribution. It also lets a node's load spread across *many* neighbours when it fails, rather than all landing on one, and lets more powerful machines take more virtual nodes.

Consistent hashing underpins Dynamo-style databases, distributed caches (memcached client libraries), CDNs, and load balancers that need affinity.

15.5 Rebalancing

As data grows, partitions must move between nodes. Approaches:

- **Fixed number of partitions:** create many more partitions than nodes up front (say 1,000 partitions over 10 nodes) and move *whole partitions* when nodes join. This is simple and widely used (Elasticsearch, Kafka, Riak).
- **Dynamic splitting:** split a partition when it grows too large, merge when it shrinks (HBase, MongoDB, Spanner).
- **Proportional to nodes:** a fixed number of partitions per node (Cassandra's virtual nodes).

Rebalancing moves a lot of data over the network, so throttle it and avoid fully automatic triggers that can make an overload worse.

15.6 The cost of sharding

Sharding solves scale and creates new problems:

- **Cross-shard queries:** a query not filtered by the shard key must **scatter-gather** to every shard and merge the results, which is slow and fragile.
- **Cross-shard joins:** usually impossible. Denormalise instead, so that data needed together lives together.
- **Cross-shard transactions:** need distributed transaction protocols (Chapter 20), or a design that avoids them.
- **Global secondary indexes:** an index on a non-shard-key column is either *local* (one per shard, so queries must scatter) or *global* (partitioned separately and updated asynchronously, so it may lag).
- **Unique IDs:** auto-increment no longer works across shards. Use UUIDs or a distributed ID generator such as Snowflake (Chapter 21).
- **Operational complexity:** backups, schema migrations and monitoring across many shards.

INTERVIEW TIP

Shard as late as you can

Before sharding: optimise queries and indexes, scale vertically, add read replicas, cache aggressively, and archive cold data. Shard when the *write* volume or *data size* truly exceeds what one primary can handle. When you do, explain your shard key choice in terms of the main query patterns.

WORKED EXAMPLE

Choosing a shard key for a chat app

Messages are queried as "the latest 50 messages in conversation X". - `message_id` (hashed): even, but one conversation's messages are scattered, so every read hits every shard. Bad. - `user_id` : a message belongs to a conversation with several users. Ambiguous. - `conversation_id` (hashed): all messages of a conversation sit on one shard, sorted by time within it. Reads hit exactly one shard. Huge group chats could become hot, so mitigate by splitting very large conversations into time buckets: `(conversation_id, month)` .

CHECK YOUR UNDERSTANDING

1. Why doesn't adding read replicas solve a write-throughput bottleneck?

   See answer →

2. With `hash(key) mod 4`, roughly what fraction of keys move when you go to 5 nodes? With consistent hashing?

   See answer →

3. What problem do virtual nodes solve in consistent hashing?

   See answer →

4. An e-commerce orders table is sharded by `order_id`. A common query is "all orders for customer X". What is the problem, and what are two possible solutions?

   See answer →

CHAPTER 16

↩ Contents

Storage Systems and CDNs

Databases store structured records. But systems also need to store **files and blobs**, such as images, videos, backups and logs, often totalling petabytes. This chapter covers the three storage models and the content delivery networks that bring content close to users.

16.1 Block, file and object storage

Block storage presents raw fixed-size blocks, like a virtual hard drive (AWS EBS, SAN volumes). The operating system puts a filesystem on it. It gives the lowest latency and is what databases and VM disks use, but it is usually attached to one machine at a time.

File storage presents a hierarchical filesystem (directories and files) shared over a network (NFS, SMB, EFS). Many machines can mount it at once. It is good for shared content and legacy applications, but scaling and performance limits appear at very large sizes.

Object storage stores immutable **objects** (data + metadata) in a flat namespace of **buckets** and **keys**, accessed over HTTP (S3, GCS, Azure Blob). It scales almost without limit, is extremely durable (often "eleven nines", through replication or erasure coding across facilities), is cheap per GB, and supports lifecycle rules. Latency is higher than block storage, and objects are replaced whole rather than edited in place.

	Block	File	Object
Interface	Disk blocks	POSIX paths	HTTP API (PUT/GET)
Sharing	One host	Many hosts	Global, any client
Scale	TBs per volume	TBs–PBs	Effectively unlimited
Latency	Lowest	Low–medium	Higher (ms)
Typical use	Databases, VM disks	Shared files	Media, backups, data lakes

KEY IDEA

Separate metadata from blobs

Never store large files in your database. Put the bytes in object storage, and store only the metadata (owner, size, content type, object key, checksums) in the database. The database stays small and fast, and object storage does what it is good at.

16.2 Uploading large files directly

Routing uploads through your application servers wastes their bandwidth and CPU. Instead, the server issues a **pre-signed URL**: a time-limited, permission-scoped URL that lets the client upload straight to object storage.

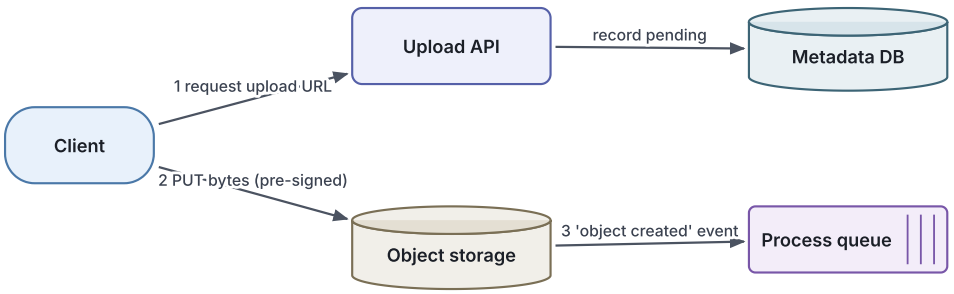


Figure 16.1 Direct-to-storage uploads with pre-signed URLs keep large transfers off your application servers.

For very large files, use **multipart upload**: split the file into chunks (say 8–100 MB), upload them in parallel, retry only failed chunks, and resume after interruptions.

16.3 Durability: replication vs. erasure coding

Storing three full copies triples the storage cost. **Erasure coding** splits data into k data fragments plus m parity fragments (for example, 10 + 4), and *any* k fragments can rebuild the original. That gives similar or better durability than 3x replication with only ~1.4x overhead, at the cost of more CPU and slower rebuilds. Large object stores use erasure coding for most data.

16.4 Distributed file systems

Systems such as **HDFS** and **Google File System** split huge files into large blocks (64–128 MB), replicate each block on several data nodes, and keep the file → block mapping on a **metadata node** (NameNode). They are designed for large sequential reads by batch jobs (Chapter 22), not for many small files or low-latency random access.

16.5 Content Delivery Networks

A **CDN** is a globally distributed network of edge servers (points of presence, PoPs) that cache content close to users. A user in Mumbai fetches an image from a Mumbai PoP in 10 ms instead of from Virginia in 250 ms.

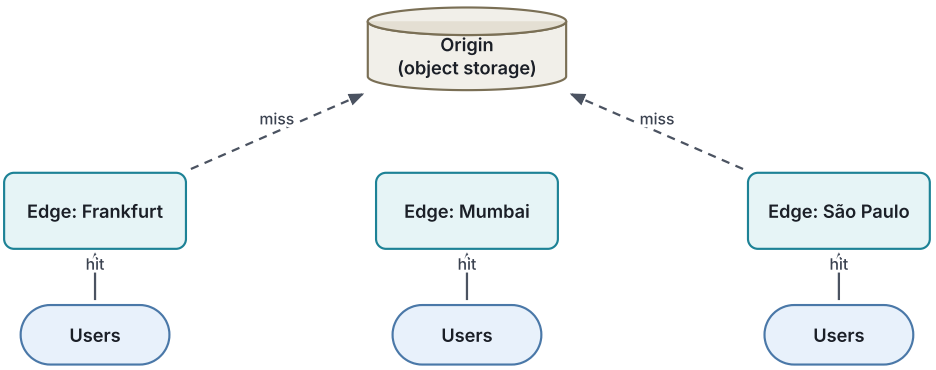


Figure 16.2 A CDN serves cached content from edge locations near users, and only cache misses travel to the origin.

Pull CDN: the edge fetches from the origin on the first request (a miss) and caches according to `Cache-Control` headers. It is easy to set up, and the first request at each edge is slow.

Push CDN: you upload content to the CDN in advance. Better for large, predictable assets such as a game patch or video catalogue, but you manage what is stored.

Cache-busting: rather than purging, publish new versions under new URLs (`app.3f9a2c.js`). The old URLs can then be cached "forever" (`max-age=31536000, immutable`) and updates appear instantly.

Modern CDNs also terminate TLS near users, absorb DDoS attacks, accelerate *dynamic* content by keeping warm connections to the origin, and run code at the edge (edge functions) for personalisation, A/B tests and authentication checks.

16.6 Video streaming: a storage case in miniature

Video platforms **transcode** each upload into several resolutions and bitrates (240p → 4K), split each into short **segments** (2–10 s), and publish a manifest (HLS / DASH). Players use **adaptive bitrate streaming**: they measure bandwidth and switch quality segment by segment. Segments are ordinary immutable objects, so they cache perfectly in CDNs.

CHECK YOUR UNDERSTANDING

1. Why should you store images in object storage rather than as BLOBs in your relational database?

   See answer →

2. What is a pre-signed URL, and why use it for uploads?

   See answer →

3. Compare 3x replication with 10+4 erasure coding in storage overhead and fault tolerance.

   See answer →

4. How does fingerprinting asset filenames (e.g. `app.3f9a2c.js`) simplify CDN caching?

   See answer →

PART V

Architecture

How to carve a system into parts: monoliths, microservices, serverless and event-driven designs, and the patterns that keep distributed services healthy.

CHAPTER 17

↩ Contents

Architectural Styles

Architecture is about where you draw boundaries: what lives together, what lives apart, and how the parts communicate. There is no universally best style. Each fits a particular team size, a stage of the product, and a set of scaling needs.

17.1 The monolith

A **monolith** is a single deployable unit containing all of the application's functionality: user management, orders, payments and notifications, built and deployed together, usually sharing one database.

Strengths: simple to develop, test, debug and deploy; in-process calls are fast and reliable; transactions across features are just database transactions; one thing to monitor. For a small team and a young product, a monolith is usually the *right* choice.

Weaknesses as it grows: the codebase gets tangled; any change means redeploying everything; one bad component (a memory leak in reports) can take down the whole app; all features must scale together; many teams in one codebase step on each other.

KEY IDEA

The modular monolith

A middle path: one deployable application, internally split into well-separated **modules** with explicit interfaces and no reaching into each other's tables. You keep the operational simplicity of a monolith, gain clean boundaries, and later, if needed, can extract a module into a service along a seam that already exists.

17.2 Microservices

Microservices split the system into small, independently deployable services, each owning one business capability *and its own data*, and communicating over the network (HTTP/gRPC or messaging).

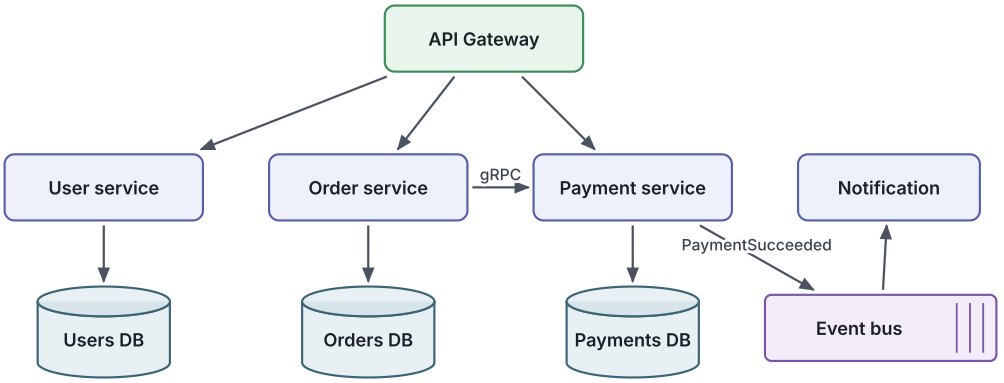


Figure 17.1 Microservices: each service owns its own datastore and can be deployed and scaled independently.

Strengths: teams own services end to end and ship independently; each service scales on its own (scale search, not billing); failures can be isolated; each service can use the technology that suits it; smaller codebases are easier to understand.

Costs: *a distributed system*, with network latency, partial failures, retries and timeouts everywhere; data consistency across services needs sagas and events instead of transactions (Chapter 20); debugging requires distributed tracing; many deployment pipelines; and a large investment in platform tooling (service discovery, observability, CI/CD).

COMMON PITFALL

The distributed monolith

Services that must be deployed together, share a database, or call each other in long synchronous chains give you all the costs of microservices and none of the benefits. If changing one feature means coordinating releases of five services, your boundaries are wrong.

Drawing service boundaries

Use **domain-driven design (DDD)**. Identify *bounded contexts*: areas of the business with their own language and models (Catalog, Ordering, Shipping, Billing). A "Product" in Catalog (descriptions, images) is a different model from a "Product" in Shipping (weight, dimensions). Good service boundaries:

- Follow business capabilities, not technical layers ("Order service", not "Database service").
- Have **high cohesion** inside and **low coupling** outside.
- Own their data. No other service reads their tables directly.
- Align with team ownership (Conway's law: systems mirror the communication structure of the organisations that build them).

17.3 Serverless

Serverless (Functions-as-a-Service, such as AWS Lambda and Cloud Functions, plus managed services) means you deploy functions, and the platform runs them on demand, scales them automatically from zero to thousands of instances, and bills per invocation.

Strengths: no servers to manage; automatic scaling; pay only for use (excellent for spiky or low traffic); fast to build event-driven glue.

Weaknesses: cold starts (the first invocation after an idle period is slow); execution time and memory limits; statelessness forced on you; vendor lock-in; harder local testing; cost can *exceed* servers at high, steady load.

Good fits: image thumbnails on upload, webhooks, scheduled jobs, lightweight APIs with variable traffic, and data pipeline steps.

17.4 Event-driven architecture

In an **event-driven architecture**, services communicate by publishing **events**, facts about something that happened (`OrderPlaced`, `PaymentFailed`), instead of calling each other directly. Interested services subscribe and react.

- **Loose coupling:** the producer doesn't know who consumes its events. New consumers are added without changing it.
- **Resilience:** consumers can be down temporarily, and events wait.
- **Scalability:** asynchronous processing absorbs load.
- **Costs:** eventual consistency; the flow of a business process is spread across services and harder to follow; you need good tooling for tracing, schema evolution and replay.

Event notification vs. event-carried state transfer

- **Event notification:** a thin event ("order 42 changed"). Consumers call back to fetch details, so they stay coupled at runtime.
- **Event-carried state transfer:** the event includes the data consumers need ("order 42 changed: status=SHIPPED, items=[…]"). Consumers keep their own local copies and never call back. More decoupled, but there is more duplicated data.

17.5 Event sourcing and CQRS

Event sourcing: instead of storing the *current state* (balance = $70), store the **full sequence of events** (deposited $100, withdrew $30) as the source of truth. Current state is derived by replaying events, usually with periodic snapshots. You get a complete audit trail, the ability to rebuild state as of any point in time, and new views built later from the history. The costs are complexity, event-schema evolution, and needing projections for queries.

CQRS (Command Query Responsibility Segregation): use *separate models* for writes (commands) and reads (queries). Writes go to a model optimised for validation and consistency, and changes flow (often through events) into one or more **read models** optimised for specific queries: a search index, a denormalised view, a cache.

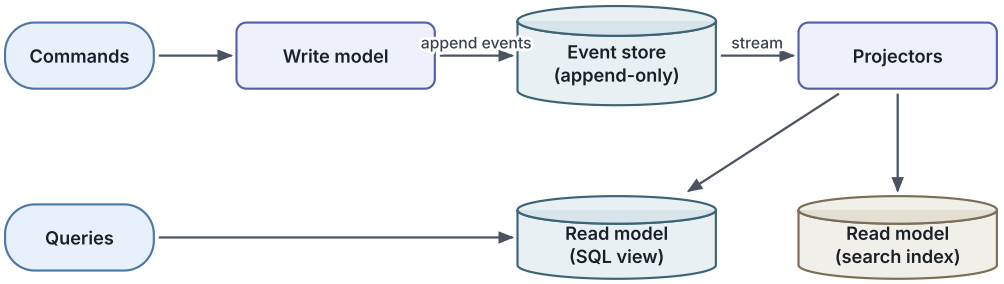


Figure 17.2 CQRS with event sourcing. Commands append events, and projections build read-optimised views.

TRADE-OFF

Powerful but heavy

Event sourcing and CQRS shine in domains with complex rules, strict audit requirements or very different read and write patterns (finance, logistics, booking). For a simple CRUD application they add a lot of machinery and eventual consistency between writes and reads. Use them where they pay for themselves.

17.6 Choosing a style

Situation	Sensible default
New product, small team, finding product–market fit	(Modular) monolith
Several teams blocking each other, parts with very different scaling needs	Extract microservices along domain boundaries
Spiky, event-triggered, short tasks	Serverless functions
Many independent reactions to business events	Event-driven with a message broker
Audit-critical domain, temporal queries	Event sourcing (+ CQRS)

CHECK YOUR UNDERSTANDING

1. Give three reasons a startup with five engineers should usually start with a monolith.

   See answer →

2. What is a "distributed monolith" and what are its symptoms?

   See answer →

3. What is the difference between event notification and event-carried state transfer?

   See answer →

4. In event sourcing, how is the current state of an account obtained, and why are snapshots used?

   See answer →

CHAPTER 18

↩ Contents

Microservices Patterns

Splitting a system into services trades in-process function calls for network calls. Network calls fail, slow down and time out. This chapter covers the patterns that make a fleet of services find each other, talk safely and survive each other's failures.

18.1 Service discovery

Service instances come and go constantly (autoscaling, deploys, crashes), and their IP addresses change. How does the Order service find a healthy Payment instance?

- **Service registry:** instances register themselves (or are registered by the platform) in a registry such as Consul, etcd, ZooKeeper or Kubernetes' API, and send heartbeats. Instances that stop heartbeating are removed.
- **Client-side discovery:** the caller queries the registry and picks an instance itself (load balancing in the client library).
- **Server-side discovery:** the caller talks to a stable address (a load balancer or a DNS name like `payments.internal`), and the platform routes to a healthy instance. Kubernetes Services work this way.

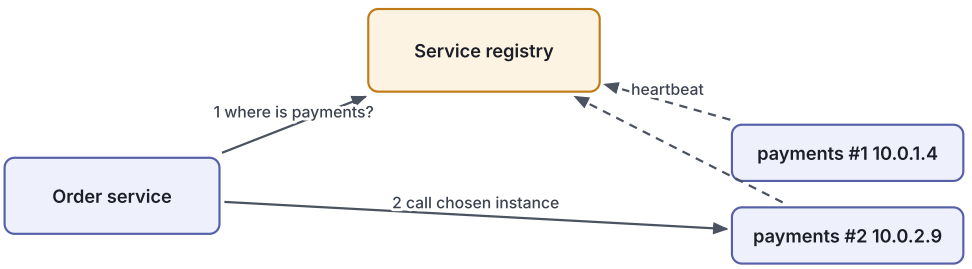


Figure 18.1 Service discovery: instances register with heartbeats, and callers resolve a name to healthy instances.

18.2 Circuit breaker

When a dependency is failing, continuing to call it wastes threads waiting on timeouts, piles load onto a struggling service, and spreads the failure upstream. A **circuit breaker** wraps calls to a dependency and tracks failures.

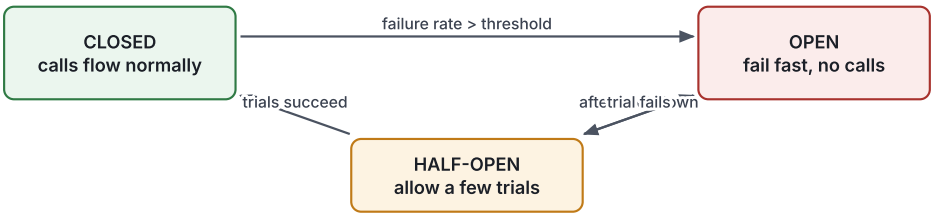


Figure 18.2 Circuit breaker states. An open breaker fails fast, and a half-open breaker sends trial requests to test recovery.

- **Closed:** requests pass through and failures are counted.
- **Open:** after failures pass a threshold (say 50% over 20 calls), the breaker *trips*. Calls fail immediately or return a **fallback** (cached data, a default, a degraded response) without touching the dependency.
- **Half-open:** after a cool-down, a few trial requests go through. If they succeed, the breaker closes. If they fail, it opens again.

The breaker protects *the caller* (threads not wasted) and *the dependency* (given room to recover).

18.3 Bulkhead

Named after a ship's watertight compartments: isolate resources so that one failure can't sink everything. Give each dependency its **own thread pool or connection pool**. If the slow recommendation service fills its 20-thread pool, calls to the payment service, with their own pool, carry on unaffected.

18.4 Timeouts, retries and backoff (revisited)

Every remote call needs a **timeout**. Without one, a hung dependency hangs you too. Retries need **exponential backoff with jitter** and a cap. Retry only **idempotent** operations or those protected by idempotency keys. Set timeouts *end-to-end*: if the user-facing request has a 2-

second budget, pass the remaining **deadline** down the call chain, so that deep services don't keep working on requests the caller has already abandoned.

> **COMMON PITFALL**
>
> **Retry amplification across layers**
>
> If each of four service layers retries a failing call 3 times, one user request can become 3^4 = **81** calls to the bottom service. Retry at one layer (usually closest to the failure, or at the edge), not at all of them.

18.5 Sidecar and service mesh

Cross-cutting networking concerns (mTLS, retries, timeouts, circuit breaking, metrics, tracing) used to be built into every service through libraries, one per language. A **sidecar** moves them into a separate proxy process deployed next to each service instance. All traffic flows through that proxy.

A **service mesh** (Istio, Linkerd, Consul Connect) is a fleet of sidecars (the **data plane**) managed by a central **control plane** that pushes routing rules, security policies and certificates to every proxy.

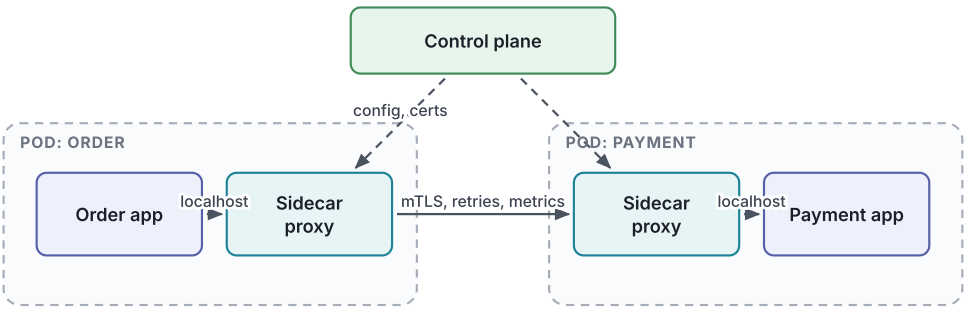


Figure 18.3 A service mesh. Sidecar proxies handle the network concerns, and the control plane configures them all.

Benefits: consistent security (automatic mTLS between all services), traffic control (canary routing, fault injection), and uniform telemetry, all without code changes. **Costs:** an extra network hop and some latency per call, more resource use, and significant operational complexity.

18.6 Other essential patterns

API composition / aggregator: one service (or the gateway) calls several services and combines their responses for the client.

Strangler fig: migrate a monolith gradually. Put a routing layer in front, rebuild one feature at a time as a new service, route its traffic there, and repeat until the monolith can be retired. There is no big-bang rewrite.

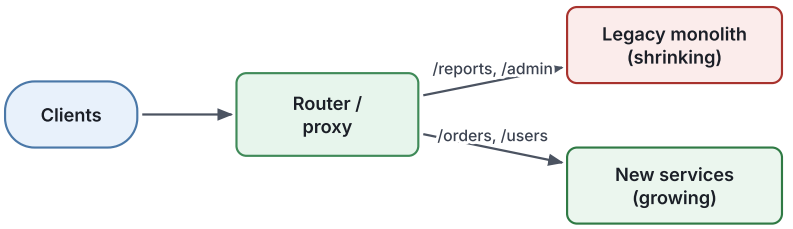


Figure 18.4 The strangler fig pattern: features move one by one from the monolith to new services behind a router.

Database per service: each service owns its data exclusively, and others get it through the service's API or events. This enables independent changes to schemas and technology, but cross-service queries need composition or CQRS views.

Anti-corruption layer: a translation layer between a new service and a legacy system (or external API), so that the legacy model doesn't leak into the clean new design.

Health check API, externalised configuration, and centralised logging/tracing (Chapter 24) are table stakes for operating services.

INTERVIEW TIP

Bring patterns in by need

Name a pattern together with the failure it prevents: "Payment calls can hang when the provider has issues, so I'll add a 2-second timeout, a circuit breaker with a fallback that queues the payment for retry, and a separate connection pool (bulkhead) so checkout browsing isn't affected."

CHECK YOUR UNDERSTANDING

1. Describe the three states of a circuit breaker and the transitions between them.

See answer →

2. How does the bulkhead pattern prevent one slow dependency from causing a full outage?

   See answer →

3. What is a service mesh, and what are its trade-offs?

   See answer →

4. How would you migrate a large monolith to microservices without a risky big-bang rewrite?

   See answer →

PART VI

Distributed Systems

What makes distributed computing hard — unreliable networks, unsynchronised clocks, disagreeing nodes — and the algorithms and data structures that tame it.

CHAPTER 19

↩ Contents

Distributed Systems Fundamentals

A distributed system is one in which a computer you didn't even know existed can make your own computer unusable. Behind that old joke is a real lesson: once a computation spans several machines, a whole new class of failures appears. This chapter covers the core problems and the ideas used to solve them.

19.1 The fallacies of distributed computing

Engineers new to distributed systems tend to assume things that are false:

1. The network is reliable.
2. Latency is zero.
3. Bandwidth is infinite.
4. The network is secure.
5. Topology doesn't change.
6. There is one administrator.
7. Transport cost is zero.
8. The network is homogeneous.

Every pattern in this book exists because one of these assumptions fails in production.

19.2 Partial failure and the unreliable network

On a single machine, things either work or crash. In a distributed system, **some parts fail while others keep working**, and you often *can't tell which*. When you send a request and get no reply, any of these may have happened:

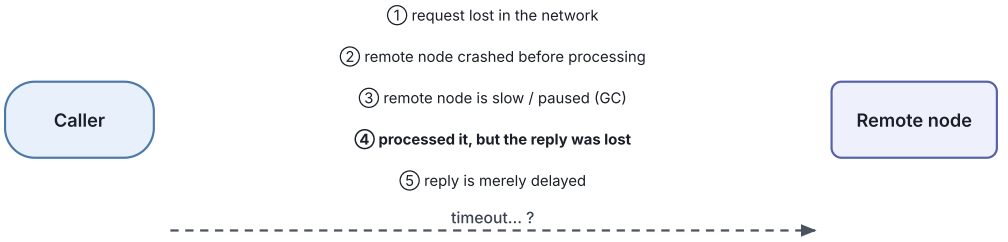


Figure 19.1 When a request times out, the caller cannot tell which of these happened, so a timeout is ambiguous.

Case ④ is why operations must be **idempotent**: you can't know whether to retry, so retrying must be safe. Timeouts are the only failure detector we have, and they are imperfect. Too short, and you declare healthy-but-slow nodes dead. Too long, and you wait ages to react to real failures.

19.3 Time and clocks

Distributed systems often need to know *when* something happened, or at least *in what order*. But physical clocks are unreliable:

- Each machine's quartz clock **drifts** (by tens of parts per million).
- **NTP** synchronisation corrects drift only to within milliseconds, sometimes far worse, and can make a clock *jump backwards*.
- Virtual machines and garbage collection can **pause** a process for seconds without it noticing.

So comparing timestamps from different machines to order events is dangerous. "Last write wins" based on wall-clock time can silently discard the actually-later write.

Logical clocks

Lamport clocks order events without physical time. Each node keeps a counter. It increments the counter on every event, attaches it to every message, and on receiving a message sets its counter to `max(local, received) + 1`. If event A *caused* event B, then `L(A) < L(B)`. The reverse doesn't hold: two concurrent events may get any order.

Vector clocks go further. Each node keeps a vector of counters, one per node. Comparing two vectors tells you whether one event **happened before** the other, or whether they are **concurrent**, which is exactly what is needed to detect write conflicts in multi-leader and leaderless databases.

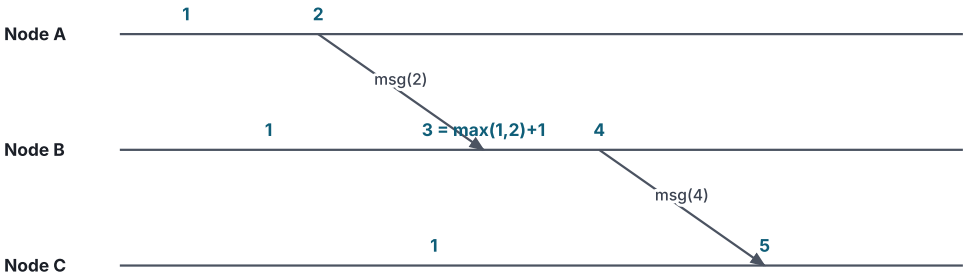


Figure 19.2 Lamport clocks: the counter travels with each message, and a receiver jumps ahead of any timestamp it has seen.

Hybrid and bounded-uncertainty clocks

Hybrid logical clocks combine physical time with a logical counter: they stay close to wall-clock time while preserving causal order. Google Spanner's **TrueTime** exposes clock *uncertainty* explicitly (`now()` returns an interval [earliest, latest] kept small by GPS and atomic clocks) and makes transactions *wait out* the uncertainty before committing. That gives globally consistent ordering.

19.4 Failure detection and membership

Nodes detect failed peers with **heartbeats**: periodic "I'm alive" messages. Missing *k* heartbeats makes a node suspect. At scale, all-to-all heartbeats are too expensive, so **gossip protocols** are used: each node periodically exchanges its view of membership with a few random peers, and information spreads through the cluster like an epidemic, reaching everyone in O(log N) rounds. Cassandra and Consul use gossip. **Phi-accrual detectors** output a suspicion *level* that adapts to observed network conditions, rather than a yes/no answer.

19.5 Leader election

Many systems need exactly one node to coordinate: to accept writes, assign work or hold a lock. **Leader election** chooses one, and a new one when it fails. The danger is a *false* failure detection producing two leaders (split brain).

Leases limit the damage. A leader holds leadership for a bounded time (say 10 s) and must renew it. A new leader can only be elected after the old lease *expires*. Combined with **fencing tokens**, a number that increases with each new leader and is checked by storage on every write, a stale ex-leader's writes are rejected even if it wakes from a long pause still believing it leads.

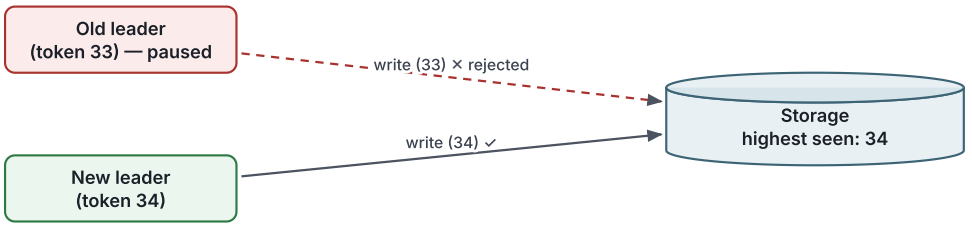


Figure 19.3 Fencing tokens. Storage rejects writes carrying an older token, so a paused ex-leader can't corrupt data.

19.6 Consensus

Consensus is getting several nodes to agree on a value (who the leader is, the next entry in a log, whether a transaction commits) even when some nodes fail and messages are delayed. It is the foundation of reliable coordination.

A consensus algorithm must guarantee:

- **Agreement:** no two nodes decide differently.
- **Validity:** the decided value was actually proposed.
- **Termination:** non-failed nodes eventually decide.

The key insight is the **majority quorum**. With 2f + 1 nodes, a decision needs f + 1 votes. Any two majorities overlap in at least one node, so two conflicting decisions can't both win. Such a cluster survives f failures: 3 nodes tolerate 1, and 5 tolerate 2.

Raft in brief

Raft was designed to be understandable, and it is used by etcd, Consul, CockroachDB and others.

1. **Leader election.** Nodes are followers. If a follower hears no heartbeat for a randomised timeout, it becomes a *candidate*, increments the **term** number, and asks for votes. A candidate that wins a majority becomes leader for that term. Randomised timeouts make split votes rare.
2. **Log replication.** Clients send commands to the leader, which appends them to its log and replicates them to followers. Once a **majority** has stored an entry, it is **committed** and applied to the state machine. Followers apply committed entries in the same order, so all replicas reach the same state (*state machine replication*).

3. **Safety.** A candidate can only win if its log is at least as up to date as a majority's, so committed entries are never lost across leader changes.

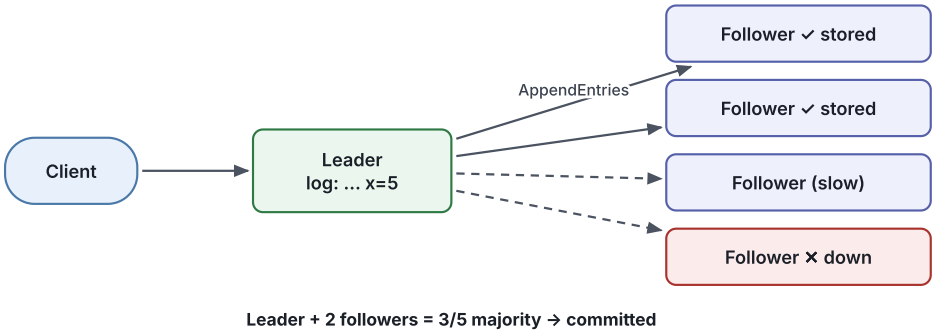


Figure 19.4 Raft log replication. An entry is committed once a majority of nodes (3 of 5) have stored it.

Paxos is the older, foundational consensus algorithm, proven correct and famously hard to understand. Raft and Multi-Paxos are equivalent in practice.

KEY IDEA

You rarely implement consensus; you use it

Production systems put consensus inside a coordination service, such as **etcd**, **ZooKeeper** or **Consul**, and build on top of it: leader election, distributed locks, configuration, service discovery, membership. Keep these clusters small (3 or 5 nodes) and on the critical path only for coordination, never for bulk data.

19.7 Distributed locks

A distributed lock ensures that only one process at a time does something, such as running a nightly job or editing a resource. Implementations use consensus systems (an etcd lease, a ZooKeeper ephemeral node) or Redis (`SET key value NX PX 30000`).

COMMON PITFALL

Locks are leases, and processes pause

A process can take a lock, pause for 40 seconds (GC, VM migration), and wake up after its lock expired and someone else took it, then go ahead and write anyway. Locks used for *correctness* must be paired with **fencing tokens** checked by the resource. Locks used only for *efficiency* (avoiding duplicate work) can tolerate the occasional double execution.

CHECK YOUR UNDERSTANDING

1. A request to a remote service times out. List three different things that might have happened, and explain why this matters for retries.

   See answer →

2. Why is ordering events by wall-clock timestamps across machines unsafe?

   See answer →

3. How many node failures can a 5-node Raft cluster tolerate, and why not more?

   See answer →

4. What is a fencing token and what problem does it solve?

   See answer →

5. What do vector clocks give you that Lamport clocks do not?

   See answer →

CHAPTER 20

↩ Contents

Distributed Transactions

Within one database, a transaction updates many rows atomically. But in a microservices system the steps of one business operation (reserve stock, charge the card, create the shipment) touch *different services with different databases*. How do we keep them consistent when any step can fail?

20.1 The problem

Placing an order requires: (1) the Order service creates an order, (2) the Inventory service reserves the items, (3) the Payment service charges the customer. If the payment fails after the stock was reserved, the reservation must be released. If the Order service crashes halfway through, the system mustn't be left in a half-done state forever.

20.2 Two-phase commit (2PC)

2PC makes several participants commit atomically under a **coordinator**.

1. **Prepare phase.** The coordinator asks every participant: "can you commit?" Each one does all its work, writes it durably, locks the resources, and votes *yes* (a promise that it *can* commit, whatever happens) or *no*.
2. **Commit phase.** If every vote is yes, the coordinator logs the decision and tells everyone to **commit**. If any vote is no, or a participant times out, it tells everyone to **abort**.

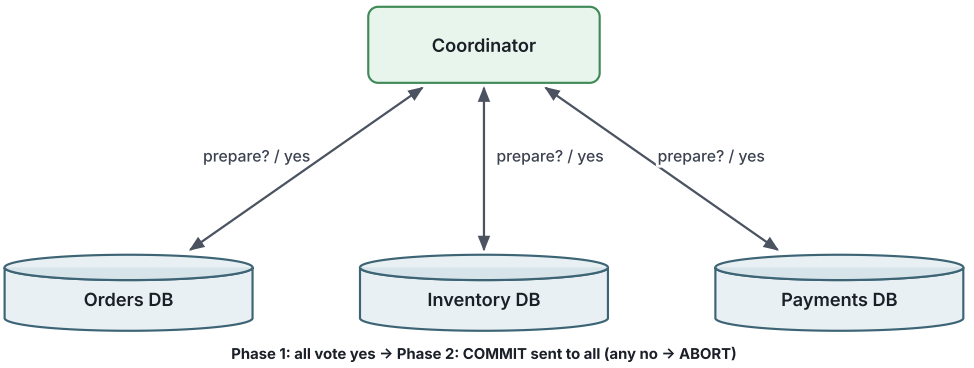


Figure 20.1 Two-phase commit. Every participant must vote yes before anyone commits.

Problems with 2PC:

- **Blocking.** A participant that voted yes must hold its locks until it hears the outcome. If the coordinator crashes after collecting votes, participants are **stuck in doubt**, holding locks, until it recovers.
- **Latency.** Several round trips, plus synchronous durable writes, on every transaction.
- **Availability.** All participants must be up. Any one failure aborts the transaction.
- **Coupling.** Every participant must support the protocol (XA), which most modern services, queues and APIs don't.

2PC works well *inside* a single distributed database (Spanner and CockroachDB run consensus-replicated 2PC internally), but it is a poor fit across independent microservices.

20.3 The Saga pattern

A **saga** breaks a distributed transaction into a sequence of **local transactions**, one per service. Each step commits on its own and triggers the next. If a step fails, the saga runs **compensating transactions** to *semantically undo* the steps already completed.

Step	Action	Compensation
1	Create order (PENDING)	Cancel order
2	Reserve inventory	Release inventory
3	Charge payment	Refund payment
4	Mark order CONFIRMED	(none, final step)

If step 3 fails: run "release inventory", then "cancel order".

Choreography vs. orchestration

Choreography: there is no central controller. Each service listens for events and reacts: `OrderCreated` → Inventory reserves → `InventoryReserved` → Payment charges → `PaymentFailed` → Inventory releases → … It is simple, decoupled and has no single point of control. But the overall flow is implicit, spread across services, hard to follow, and prone to cyclic dependencies as the steps multiply.

Orchestration: a central **saga orchestrator** (a state machine) tells each service what to do and handles replies: "Inventory, reserve" → reply → "Payment, charge" → failure → "Inventory, release". The flow is explicit and in one place, which makes it easier to monitor, test and change. The orchestrator is one more component, and it must be durable (workflow engines such as Temporal, Step Functions or Camunda provide this).

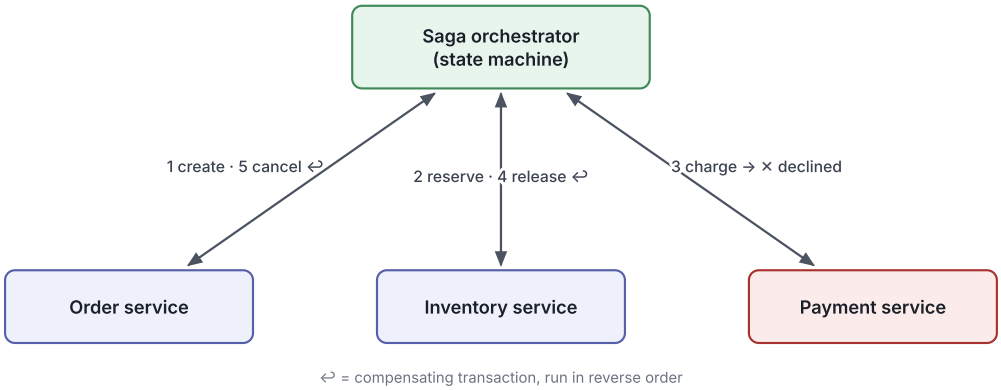


Figure 20.2 Saga orchestration. On payment failure, the orchestrator runs compensations in reverse order.

COMMON PITFALL

Sagas are not isolated

Unlike an ACID transaction, a saga's intermediate states are **visible** to other operations. Another request might read "inventory reserved" for an order that is about to be cancelled. Countermeasures: *semantic locks* (a PENDING status that others respect), designing steps to commute, re-reading values before acting, and ordering steps so that the ones that can't be compensated (sending an email, shipping a parcel) come last.

Compensations must themselves be **idempotent** and **retryable**: a refund that is retried mustn't refund twice.

20.4 The dual-write problem and the transactional outbox

A service often needs to **update its database and publish an event**: save the order, then publish `OrderCreated`. Doing these as two separate operations is a **dual write**, and it is broken. If the service crashes between them, the database has the order but no event was published. If you publish first, the event may describe an order that never got saved.

The **transactional outbox** pattern fixes this. In the **same local transaction** as the business change, insert the event into an `outbox` table. A separate **relay** process reads the outbox and publishes to the broker, marking rows as sent.

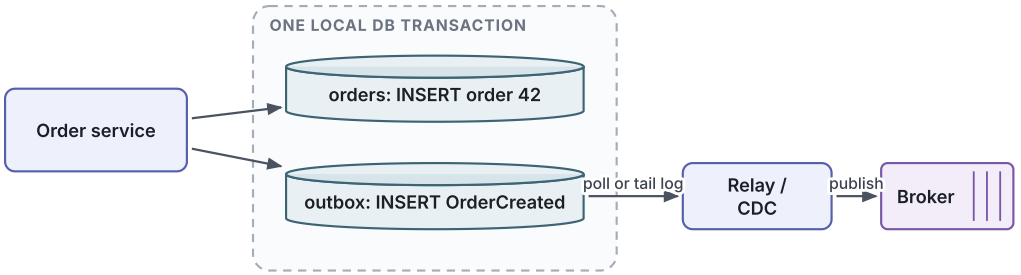


Figure 20.3 The transactional outbox. The business row and the event commit atomically, and a relay publishes events afterwards.

The relay can poll the table, or use **change data capture (CDC)**, tailing the database's replication log (with Debezium, for example). Either way, publishing is *at-least-once*, so consumers must be idempotent, using an event ID to de-duplicate (the **inbox** pattern on the consumer side).

20.5 Other techniques

- **Idempotency keys** across service boundaries make every retried step safe (Chapter 8).
- **Reservation / TCC (Try-Confirm-Cancel):** each service first *tries* (tentatively reserves resources with an expiry), then the coordinator *confirms* or *cancels*. It is like 2PC at the business level, with timeouts instead of indefinite locks.
- **Avoiding the distributed transaction** by redrawing boundaries: if two pieces of data must always change atomically, perhaps they belong in the same service.

INTERVIEW TIP

Default answer for cross-service consistency

"I'll avoid 2PC across services. The order flow is an orchestrated saga with compensating actions; each service updates its DB and emits events through a transactional outbox; and all consumers are idempotent. The system is eventually consistent, and the order shows PENDING until the saga completes."

CHECK YOUR UNDERSTANDING

1. Why is two-phase commit called a "blocking" protocol?

   See answer →

2. In a saga for booking a trip (flight, hotel, car), the car rental fails. What happens?

   See answer →

3. Compare choreography and orchestration for sagas.

   See answer →

4. What is the dual-write problem, and how does the transactional outbox solve it?

   See answer →

CHAPTER 21

↩ Contents

Data Structures for Scale

At large scale, exact answers can be impossibly expensive. Some ingenious data structures trade a tiny, controlled amount of accuracy for huge savings in memory and time. Others organise space or data so that queries remain fast at billions of items. Every senior engineer should know these.

21.1 Bloom filters: "definitely not" or "probably yes"

A **Bloom filter** answers *"is this item in the set?"* using a fraction of the memory of storing the set. It may give **false positives** ("probably yes" when the answer is no) but **never false negatives**: a "no" is certain.

How it works: a bit array of *m* bits, all initially 0, and *k* independent hash functions. To **add** an item, hash it *k* times and set those *k* bits to 1. To **query**, hash it *k* times. If any of those bits is 0, the item is **definitely not** in the set. If all are 1, it is *probably* in the set (the bits may have been set by other items).

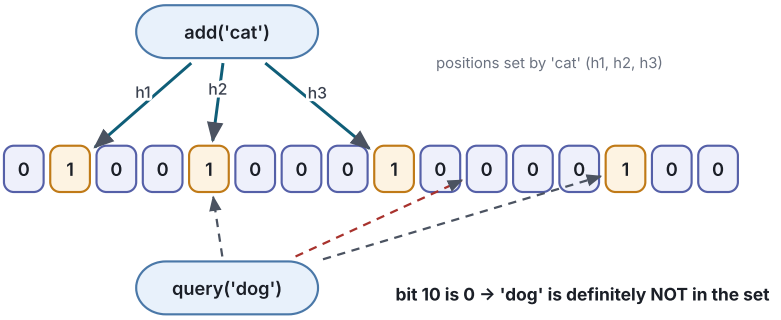


Figure 21.1 A Bloom filter with 3 hash functions. "cat" sets three bits. A query for "dog" finds a 0 bit, so dog is definitely absent.

With about 10 bits per item and 7 hash functions, the false-positive rate is about **1%**, whatever the item size. A billion URLs fit in ~1.2 GB instead of ~100 GB. Standard Bloom filters don't support deletion. *Counting Bloom filters* and *cuckoo filters* do.

Uses: skipping disk lookups for absent keys in LSM-tree databases; blocking cache penetration (Chapter 12); "have we already crawled this URL?"; checking weak passwords; detecting duplicate events.

21.2 Count-Min Sketch: frequencies in fixed memory

Estimates how often each item occurs in a stream, in fixed memory. It is a grid of d rows × w counters, with one hash function per row. To count an item, increment one counter in each row. To estimate its count, take the **minimum** of its d counters. Collisions only *inflate* counters, so the estimate never undercounts, and taking the minimum limits the overcount. **Uses:** heavy hitters or "top K" (trending hashtags, the most-requested URLs), rate-limiting heuristics, network traffic analysis.

21.3 HyperLogLog: counting distinct items

How many *unique* visitors did the site get today? Storing every ID takes gigabytes. **HyperLogLog** estimates cardinality using about **12 KB** for billions of items, with ~0.8% standard error.

Intuition: hash each item to a uniformly random bit string. Long runs of leading zeros are rare: a run of n zeros turns up roughly once every 2^n items. The longest run you have seen tells you roughly how many distinct items there have been. HyperLogLog splits items into thousands of buckets and averages their estimates (with a harmonic mean) to keep the variance low. Sketches can be **merged**, so daily counts combine into weekly ones. Redis has it built in (`PFADD`, `PFCOUNT`).

21.4 Merkle trees: comparing huge datasets cheaply

A **Merkle tree** is a tree of hashes. Each leaf hashes a block of data, and each parent hashes its children's hashes. The root hash summarises the whole dataset. If one block changes, its hash changes and so do all the hashes above it.

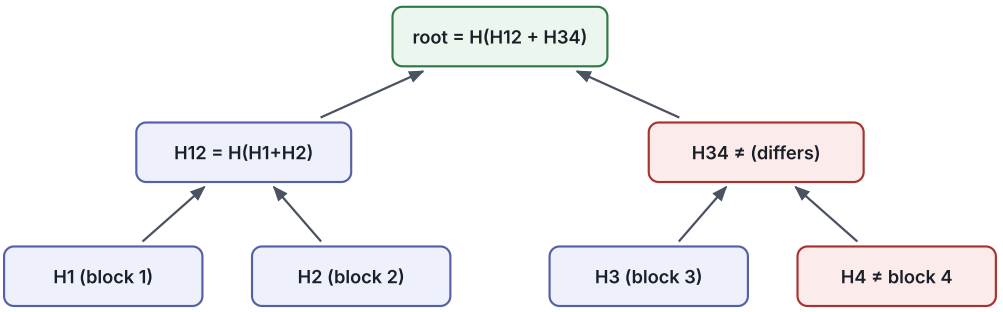


Figure 21.2 A Merkle tree. Comparing root hashes tells two replicas whether they differ, and descending only into mismatched branches finds the differing blocks in O(log n) steps.

Uses: anti-entropy repair between database replicas (Cassandra, DynamoDB), so that only out-of-sync ranges are exchanged; Git (commits are hash trees); blockchains; certificate transparency logs; verifying file downloads in chunks.

21.5 Geospatial indexing: geohash, quadtrees, S2

"Find drivers within 2 km" is hard with ordinary indexes, which sort along one dimension, while location has two.

Geohash interleaves the bits of latitude and longitude and encodes them as a base-32 string. The map is recursively divided into a grid, and each extra character narrows the cell. **Points in the same cell share a prefix**, so a prefix query (`WHERE geohash LIKE '9q8yy%'`) on an ordinary B-tree finds nearby points. Cells at boundaries need the 8 neighbouring cells checked too.

Geohash length	Cell size (approx.)
4	39 km × 20 km
5	4.9 km × 4.9 km
6	1.2 km × 0.6 km
7	153 m × 153 m

Quadtrees recursively split a region into four quadrants *only where points are dense*, so a city centre is finely divided while the ocean is one big cell. They adapt well to uneven distributions and are usually held in memory.

Google S2 and **Uber H3** project the sphere onto cells (squares on a cube for S2, hexagons for H3) with hierarchical IDs, avoiding distortion near the poles. Hexagons have uniform neighbour distances, which suits ride-hailing supply/demand grids.

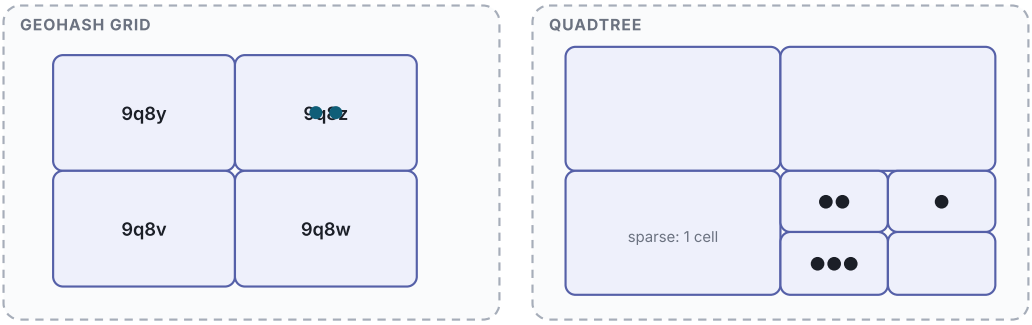


Figure 21.3 Left: geohash cells share prefixes. Right: a quadtree subdivides only where points are dense.

21.6 Skip lists and sorted sets

A **skip list** is a linked list with "express lanes": extra levels of pointers that skip over many elements, giving O(log n) search, insertion and deletion, and efficient range queries, with a simple, lock-friendly implementation. Redis **sorted sets** (leaderboards, rate-limit windows, priority queues) are built on skip lists. LSM memtables often use them too.

21.7 Unique ID generation at scale

Auto-increment IDs need a single counter, which is a bottleneck and a single point of failure across shards. Alternatives:

- **UUID v4:** 128 random bits generated anywhere with no coordination. But IDs are large and random, which scatters B-tree inserts (bad locality) and doesn't sort by time. **UUID v7** puts a timestamp first and fixes the ordering.
- **Snowflake-style IDs:** 64 bits = **41-bit millisecond timestamp** + **10-bit machine ID** + **12-bit per-millisecond sequence**. They sort roughly by time, are compact, and are generated locally (up to 4,096 IDs per millisecond per machine).
- **Ticket servers / range allocation:** a central service hands out *blocks* of IDs (1–10,000 to server A), and servers allocate locally from their block.



Figure 21.4 Snowflake ID layout: 64 bits that sort roughly by time and need no coordination.

Structure	Answers	Memory	Error
Bloom filter	Is X in the set?	~10 bits / item	False positives only
Count-Min Sketch	How often does X occur?	Fixed (KB)	Overestimates only
HyperLogLog	How many distinct items?	~12 KB	~1%
Merkle tree	Where do datasets differ?	O(n) hashes	Exact
Geohash / quadtree / S2	What is near this point?	Index size	Exact (with neighbours)
Skip list	Ordered set operations	O(n)	Exact

CHECK YOUR UNDERSTANDING

1. A Bloom filter reports that an item is present. Can you trust it? What if it reports the item is absent?

   See answer →

2. You must count unique daily visitors across billions of events, with weekly and monthly roll-ups, in minimal memory. Which structure fits, and why?

   See answer →

3. How do Merkle trees make replica repair efficient?

   See answer →

4. Why does a geohash query need to check neighbouring cells?

   See answer →

5. What are the parts of a Snowflake ID, and what properties does that layout give?

   See answer →

PART VII

Operating at Scale

Processing oceans of data, shipping changes safely many times a day, seeing inside running systems, and defending them against attackers.

CHAPTER 22

↩ Contents

Big Data Processing

Online systems answer user requests in milliseconds. But organisations also need to process *all* their data: compute daily reports, train recommendation models, detect fraud patterns, rebuild search indexes. This chapter covers the two great families of data processing, **batch** and **stream**, and the architectures that combine them.

22.1 OLTP vs. OLAP

	OLTP (transactional)	OLAP (analytical)
Typical query	Read/write a few rows by key	Aggregate millions to billions of rows
Users	End users via the application	Analysts, dashboards, ML pipelines
Latency	Milliseconds	Seconds to hours
Data	Current state	Historical, append-mostly
Storage layout	Row-oriented	**Column-oriented**
Examples	PostgreSQL, MySQL, DynamoDB	Snowflake, BigQuery, Redshift, ClickHouse

Column-oriented storage keeps each column's values together. A query like `SELECT avg(price) FROM sales WHERE year = 2026` reads only the `price` and `year` columns instead of every whole row, and similar values packed together compress dramatically (often 10x). This is why analytical warehouses are columnar.

22.2 Batch processing and MapReduce

Batch processing runs a job over a large, *bounded* dataset (yesterday's logs, the full user table) and produces output. Throughput matters more than latency.

MapReduce made large-scale batch processing accessible by expressing jobs in two functions that the framework runs in parallel over thousands of machines:

1. **Map:** read input records and emit key-value pairs. (For word count: each word → `(word, 1)`.)

2. **Shuffle:** the framework groups all values by key, sorting and moving data across the network, so each key's values land on one reducer.
3. **Reduce:** process each key's values. (Sum the 1s → `(word, total)`.)

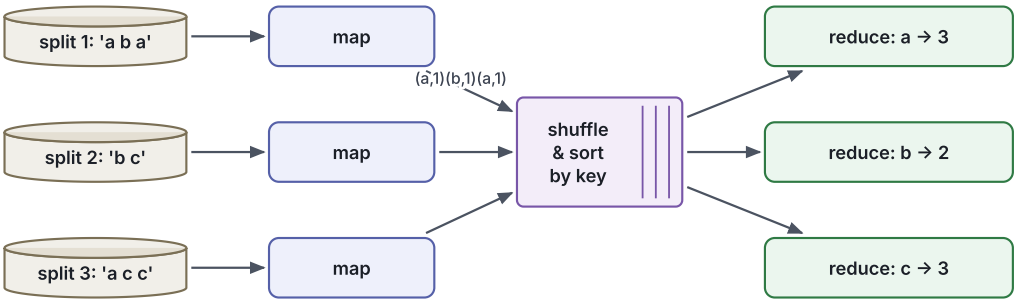


Figure 22.1 MapReduce word count. Mappers run in parallel over input splits, the shuffle groups by key, and reducers aggregate.

The framework takes care of the hard parts: **data locality** (running map tasks on the machines that hold the data), **fault tolerance** (re-running failed tasks, which works because inputs are immutable and tasks are deterministic), and **scheduling**.

MapReduce writes intermediate results to disk between stages, which is robust but slow for multi-step jobs. **Apache Spark** keeps intermediate data in memory and models jobs as a graph of transformations over distributed datasets (RDDs and DataFrames), recovering lost partitions by recomputing them from their lineage. It is often 10–100x faster for iterative and multi-stage work, and it has largely replaced classic MapReduce.

22.3 Data warehouses, lakes and lakehouses

- **Data warehouse:** structured, cleaned, schema-on-write data in a columnar store, optimised for SQL analytics and BI. Reliable and fast to query, but costly and rigid for raw or unstructured data.
- **Data lake:** cheap object storage holding *raw* data of every kind (logs, JSON, images, Parquet files), with schema-on-read. Flexible and cheap, but without discipline it becomes a "data swamp".
- **Lakehouse:** open table formats (Apache Iceberg, Delta Lake, Hudi) add warehouse features (ACID transactions, schema enforcement, time travel, efficient updates) on top of lake storage, so one copy of the data serves both BI and ML.

ETL vs. ELT: traditionally data was *Extracted*, *Transformed*, *then Loaded* into the warehouse. With cheap, scalable warehouses, it is now common to *Extract*, *Load* raw data first, then *Transform* inside the warehouse with SQL.

22.4 Stream processing

Stream processing handles *unbounded* data continuously, as events arrive: fraud detection within seconds of a card swipe, live dashboards, real-time recommendations, alerting. Engines include **Apache Flink**, Kafka Streams and Spark Structured Streaming, usually reading from a log such as Kafka.

Key concepts:

- **Event time vs. processing time.** *Event time* is when something happened (the timestamp on the phone). *Processing time* is when the system sees it. Events arrive late and out of order (a phone offline in a tunnel), so correct results need event time.
- **Windows** group the unbounded stream into finite chunks: *tumbling* (fixed, non-overlapping: every minute), *sliding* (overlapping: the last 5 minutes, updated every minute), *session* (activity separated by gaps of inactivity).
- **Watermarks** declare "we believe all events up to time T have arrived". They let a window close and emit results while bounding how long to wait for latecomers. Events later than that go to a side output or update an earlier result.
- **State and checkpoints.** Operators keep state (counts, joins). Engines like Flink periodically snapshot state together with input offsets, so after a failure they restore and replay, achieving *exactly-once state updates*.

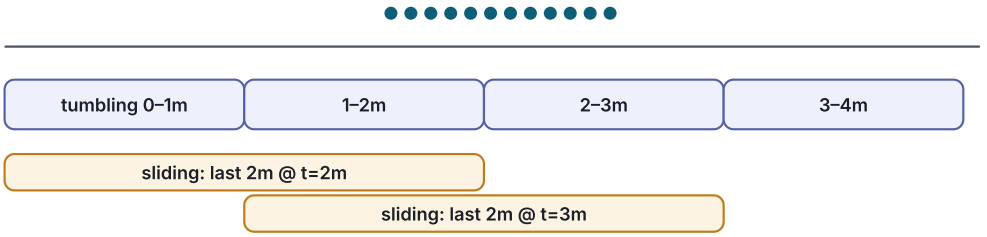


Figure 22.2 Tumbling vs. sliding windows over the same event stream.

22.5 Lambda and Kappa architectures

Lambda architecture runs two paths side by side: a *batch layer* that periodically recomputes accurate views from all historical data, and a *speed layer* that processes recent events in real time for fresh but approximate results. A serving layer merges the two. It is accurate and fresh, but you maintain the same logic in two systems.

Kappa architecture uses *only* a stream processor over a replayable log. To recompute (after fixing a bug, say), you replay the log from the beginning into a new version of the job. One codebase, simpler operations. It depends on long log retention and a capable stream engine.

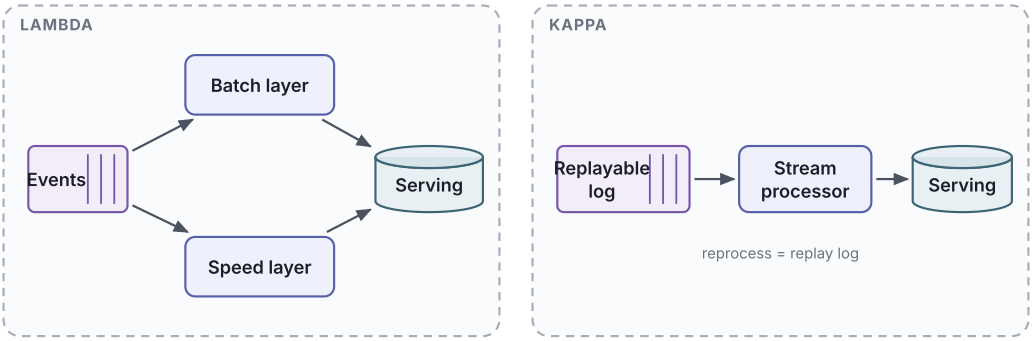


Figure 22.3 Lambda (batch + speed layers) versus Kappa (one streaming path over a replayable log).

CHECK YOUR UNDERSTANDING

1. Why are analytical databases usually column-oriented?

   See answer →

2. Explain the map, shuffle and reduce phases using the example of counting page views per URL from log files.

   See answer →

3. What is the difference between event time and processing time, and why do watermarks exist?

   See answer →

4. What is the main drawback of the Lambda architecture that Kappa tries to fix?

   See answer →

CHAPTER 23

↩ Contents

Deployment and Release Engineering

The fastest-moving engineering organisations deploy hundreds of times a day, *and* have fewer outages than those that deploy monthly. The secret isn't heroics. It is small changes, automation, and release strategies that limit the blast radius when something goes wrong.

23.1 CI/CD pipelines

- **Continuous Integration (CI):** every change is merged to the main branch frequently and automatically built and tested (unit tests, linters, static analysis, security scans). Integration problems are caught within minutes, not at the end of a release cycle.
- **Continuous Delivery:** every change that passes the pipeline is *deployable*, and releasing is a button press.
- **Continuous Deployment:** every passing change goes to production automatically.



Figure 23.1 A typical CI/CD pipeline. Each stage is a gate, and a failure stops the change before production.

Build an **immutable artifact** (a container image tagged with the commit hash) once, and promote *that same artifact* through every environment. Never rebuild for production. Keep environments as similar as possible, and define infrastructure as code (Terraform, Kubernetes manifests) so environments are reproducible and changes are reviewed like code.

23.2 Deployment strategies

Recreate

Stop the old version, start the new one. Simple, but there is downtime. Acceptable only for internal or non-critical tools.

Rolling deployment

Replace instances a few at a time (say 10% per step), waiting for health checks before continuing. There is no downtime and no extra capacity needed. But two versions run side by side during the rollout (so they must be compatible), and rollback is another rolling deploy.

Blue-green deployment

Run two complete environments. **Blue** serves production while **green** gets the new version and is tested. Then switch the router so that all traffic goes to green. Rollback is instant: switch back. It costs double the capacity during the switch, and database changes must work with both versions.

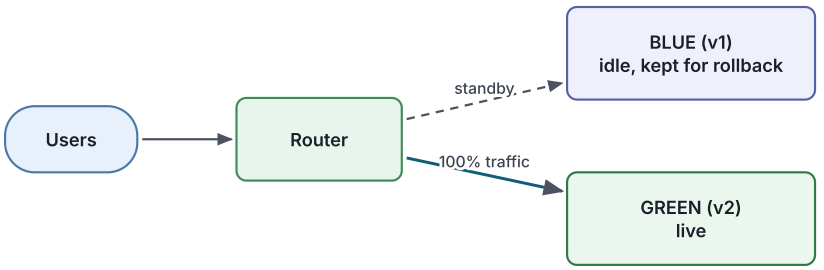


Figure 23.2 Blue-green deployment. Traffic switches all at once, and rollback is flipping it back.

Canary release

Route a *small* percentage of real traffic (1% → 5% → 25% → 100%) to the new version while closely comparing its error rate, latency and business metrics against the old version. If the metrics get worse, roll back automatically. Only a small fraction of users ever see a bad release. It needs good observability and traffic-splitting infrastructure.

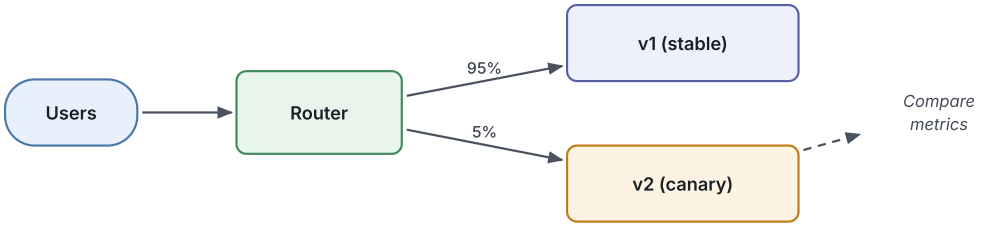


Figure 23.3 Canary release. A small slice of traffic tests the new version, and the share grows only while its metrics stay healthy.

Shadow (dark) launch

Copy real production traffic to the new version, but *throw away its responses*. Users are unaffected, and you see how the new code behaves under real load. Be careful with side effects: shadowed requests must never send emails or charge cards.

Strategy	Downtime	Rollback speed	Extra capacity	Risk exposure
Recreate	Yes	Slow	None	All users
Rolling	No	Medium	Minimal	Grows gradually
Blue-green	No	Instant	2x	All users at the switch
Canary	No	Fast	Small	Small slice first
Shadow	No	N/A	Extra	None (responses discarded)

23.3 Feature flags

Feature flags (toggles) separate *deploying* code from *releasing* features. New code ships turned off behind a flag, and is then switched on gradually for internal users, beta testers, a percentage of users or particular regions, all without a deploy. Flags allow trunk-based development, A/B testing, instant kill switches for misbehaving features, and gradual rollouts.

COMMON PITFALL

Flag debt

Every flag doubles the number of code paths to reason about and test. Flags left in place for months make code tangled and fragile. Give release flags an owner and an expiry date, and delete them once the rollout is complete.

23.4 Database migrations without downtime

Code can roll back in seconds. Schema changes can't. Use the **expand–contract** (parallel change) pattern so that every step is compatible with both the old and new code:

1. **Expand:** add the new column or table (nullable, with no constraints that old code would break).
2. **Dual-write:** deploy code that writes both old and new structures. Backfill existing rows in batches.
3. **Switch reads** to the new structure, and verify.
4. **Contract:** stop writing the old structure, then drop it in a later release.

Never rename or drop a column in the same release that stops using it, and run large backfills in throttled batches so they don't lock tables or overload replication.

INTERVIEW TIP

Talk about the blast radius

"Deploys go out through a canary in one region at 1%, with automatic rollback on SLO regression, then progressively to all regions. Risky features sit behind flags. Schema changes follow expand–contract, so any single deploy can be rolled back on its own."

CHECK YOUR UNDERSTANDING

1. Compare blue-green and canary deployments. When would you prefer each?

   See answer →

2. What does it mean to separate "deploy" from "release"? Which tool enables it?

   See answer →

3. You need to rename the column `fullname` to `display_name` with zero downtime. Outline the steps.

   See answer →

CHAPTER 24

↩ Contents

Observability

You can't fix what you can't see. **Monitoring** tells you *whether* something is wrong. **Observability** is the ability to work out *why*, including for failures you never anticipated, from the signals the system emits. The three classic pillars are **logs**, **metrics** and **traces**.

24.1 Logs

Logs are timestamped records of discrete events: "user 42 logged in", "payment 9f2 failed: card declined".

- **Structured logging:** emit JSON with consistent fields (`timestamp`, `level`, `service`, `trace_id`, `user_id`, `message`) rather than free text, so logs can be searched, filtered and aggregated by machines.
- **Centralised aggregation:** ship logs from every instance to a central system (an ELK/OpenSearch stack, Loki, a managed service). Instances are ephemeral, so local log files disappear with them.
- **Levels and sampling:** DEBUG, INFO, WARN, ERROR. At high volume, sample routine logs, but keep every error.
- **Never log secrets** (passwords, tokens, full card numbers), and be careful with personal data.

24.2 Metrics

Metrics are numeric measurements aggregated over time: request rate, error count, latency histogram, CPU usage, queue depth. They are cheap to store and fast to query, which makes them ideal for dashboards and alerts.

- **Counter:** only goes up (total requests, total errors). Graph its *rate*.
- **Gauge:** goes up and down (memory in use, queue depth, active connections).

- **Histogram:** distribution of values in buckets (request latency), from which percentiles are computed.

Two widely used checklists of what to measure:

- **RED** (for services): **R**ate (requests/s), **E**rrors (failed requests/s), **D**uration (latency distribution).
- **USE** (for resources): **U**tilisation, **S**aturation (queued work), **E**rrors.
- The **four golden signals**: latency, traffic, errors, saturation.

COMMON PITFALL

Cardinality explosions

Each unique combination of metric labels is a separate time series. Labelling a metric with `user_id` or `request_id` creates millions of series and can bring down your metrics system. Keep labels low-cardinality (endpoint, status code, region). High-cardinality detail belongs in logs and traces.

24.3 Distributed tracing

In a microservices system, one user request may touch twenty services. Logs show each service's part separately, but nothing ties them together. **Distributed tracing** follows a request across services.

- A **trace** represents one end-to-end request and has a unique **trace ID**.
- It is made of **spans**, timed operations (an HTTP call, a database query), each with a span ID, a parent span ID, timing and attributes.
- The trace context is **propagated** in request headers (the W3C `traceparent` standard) from service to service.
- **OpenTelemetry** is the vendor-neutral standard for producing traces, metrics and logs, and backends such as Jaeger, Zipkin or Tempo store and visualise them.

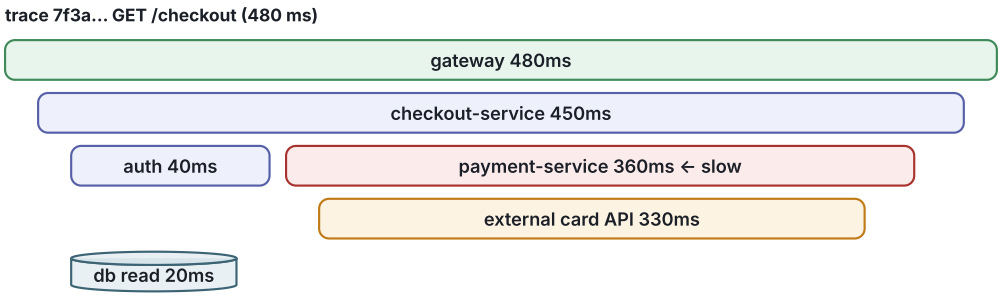


Figure 24.1 A trace as a waterfall of spans. Each bar is an operation, and nesting shows which call caused which, making the slow span obvious.

Traces answer questions like "why is checkout slow for users in Brazil?" by showing exactly which hop takes the time. Because tracing every request is expensive, systems **sample**: *head-based* sampling decides at the start (keep 1%), while *tail-based* sampling decides after the request finishes, keeping all slow or failed traces.

> KEY IDEA
>
> **Correlate the three pillars**
>
> Put the `trace_id` in every log line. Then an alert on a metric (error rate up) leads to example traces (which requests failed, and where), which lead to the exact logs for those requests. Signals that are linked like this are what turn monitoring into observability.

24.4 Alerting

- **Alert on symptoms, not causes.** Page a human when *users are affected* (the error-rate SLO is burning, latency is above target), not because CPU hit 80%, which may be perfectly fine.
- **SLO burn-rate alerts:** alert when the error budget is being consumed fast enough to run out soon, with fast-burn alerts that page and slow-burn alerts that open a ticket.
- **Every page must be actionable and urgent.** Noisy alerts train people to ignore them ("alert fatigue").
- Each alert should link to a **runbook** describing diagnosis and mitigation steps.

24.5 Incidents and learning

When things break: detect, assign an **incident commander**, communicate status, mitigate first (roll back, fail over, shed load), and only then find the root cause. Afterwards, write a **blameless post-mortem**: a timeline, contributing factors, what went well, and concrete action

items. The goal is to improve the *system*, not to blame people.

CHECK YOUR UNDERSTANDING

1. What is the difference between a counter, a gauge and a histogram? Give an example of each.

   See answer →

2. Why shouldn't `user_id` be a label on a metric?

   See answer →

3. How does distributed tracing link work across many services into one picture?

   See answer →

4. Why is "alert on symptoms, not causes" a good principle?

   See answer →

CHAPTER 25

↩ Contents

Security at Scale

Security isn't a feature you add at the end. It is a property of the whole design. This chapter covers the essential concepts that come up in system design: protecting data in transit and at rest, managing secrets, controlling access, and defending against common attacks.

25.1 Principles

- **Defence in depth:** multiple independent layers (network controls, authentication, authorisation, encryption, monitoring), so that one failure isn't a breach.
- **Least privilege:** every user, service and key gets the minimum access it needs, and no more.
- **Zero trust:** don't trust a request because it comes from "inside the network". Authenticate and authorise every request, service-to-service calls included.
- **Secure by default:** safe defaults (encryption on, ports closed, public access off) that must be explicitly relaxed.
- **Minimise the attack surface:** expose as few endpoints, ports and dependencies as possible.

25.2 Encryption in transit: TLS and mTLS

All network traffic, internal included, should be encrypted with **TLS**. During the handshake, the server proves its identity with a **certificate** (its public key, signed by a certificate authority the client trusts). The two sides then agree on symmetric session keys using an ephemeral key exchange, which gives **forward secrecy**: even if the server's private key leaks later, past sessions can't be decrypted.

Mutual TLS (mTLS) has *both* sides present certificates. It is the standard way to authenticate services to each other in a zero-trust network, and service meshes automate it by issuing short-lived certificates to every workload.

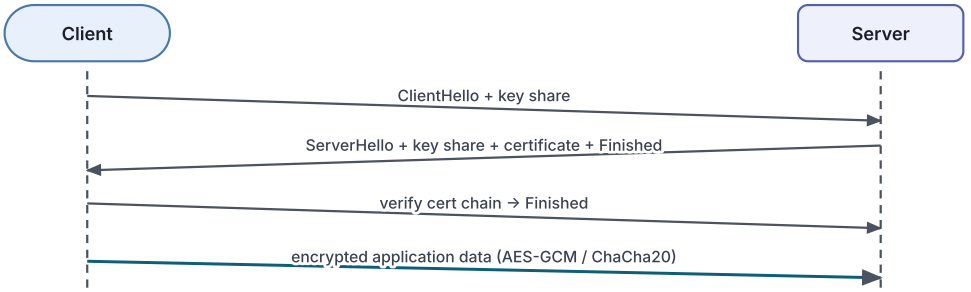


Figure 25.1 A TLS 1.3 handshake in outline. Asymmetric crypto establishes trust and a shared key, then fast symmetric encryption carries the data.

25.3 Encryption at rest

Data on disks, in databases, backups and object storage should be encrypted, so that stolen disks or misconfigured snapshots don't leak it. The standard approach is **envelope encryption**: data is encrypted with a *data key*, and the data key is itself encrypted with a *master key* held in a **key management service (KMS)** or hardware security module (HSM) that never exports it. Rotating the master key re-wraps the data keys without re-encrypting all the data.

For particularly sensitive fields (national IDs, card numbers), add **application-level** or field-level encryption, or **tokenisation**: replace the value with a meaningless token and keep the real value in a separate, tightly controlled vault.

25.4 Secrets management

Passwords, API keys, database credentials and private keys must **never** sit in source code, container images or plain config files.

- Store them in a **secrets manager** (HashiCorp Vault, AWS Secrets Manager, Kubernetes secrets with encryption).
- Inject them at runtime, scoped to the services that need them, with every access audited.
- **Rotate** them regularly and automatically. Prefer **short-lived, dynamically generated credentials** (Vault issuing a database login valid for 1 hour) over long-lived static ones.
- Scan repositories for leaked secrets, and revoke immediately on exposure.

25.5 Identity and access control

Chapter 9 covered authentication (sessions, JWTs, OAuth 2.0/OIDC) and authorisation models (RBAC, ABAC, ReBAC). At organisational scale:

- **Single sign-on (SSO):** users authenticate once with a central identity provider and reach many applications. Enterprise SSO uses **SAML** (XML-based assertions, common in older enterprise apps) or **OIDC** (JSON/JWT-based, modern).
- **Multi-factor authentication (MFA):** something you know + something you have (a TOTP app, a hardware key) + something you are. Phishing-resistant factors (FIDO2/WebAuthn passkeys) stop most account-takeover attacks.
- **Workload identity:** services get cryptographic identities (SPIFFE IDs, cloud IAM roles) instead of shared passwords.

25.6 Common attacks and defences

Attack	What happens	Defences
SQL / command injection	Untrusted input executed as code	Parameterised queries, input validation, least-privilege DB users
Cross-site scripting (XSS)	Attacker's script runs in victims' browsers	Output encoding, Content Security Policy, HttpOnly cookies
Cross-site request forgery (CSRF)	Victim's browser makes unwanted authenticated requests	SameSite cookies, CSRF tokens
Broken object-level authorisation	User reads others' data by changing an ID	Check ownership on every object access
Credential stuffing	Leaked passwords tried at scale	Rate limiting, MFA, breached-password checks, bot detection
SSRF	Server tricked into calling internal URLs	Allow-lists for outbound requests, block metadata endpoints
DDoS	Traffic floods exhaust resources	CDN/edge absorption, anycast, rate limiting, autoscaling, WAF
Supply chain	Malicious or vulnerable dependency	Pinning, signature verification, SBOMs, dependency scanning

25.7 DDoS protection

Distributed denial-of-service attacks flood a system with traffic from many sources. **Volumetric** attacks (hundreds of Gbps of UDP floods) are absorbed by the huge capacity of CDN and anycast networks. **Protocol** attacks (SYN floods) are mitigated by edge devices and SYN cookies. **Application-layer** attacks (floods of expensive-looking HTTP requests) need a **web application firewall (WAF)**, rate limiting, bot detection and caching. Design so that the origin is never directly reachable, and all traffic passes through the protective edge.

25.8 Auditing, privacy and compliance

- Keep **immutable audit logs** of security-relevant actions (who accessed what, when, from where).
- Practise **data minimisation:** don't collect or keep what you don't need, and set retention limits.
- Regulations (GDPR, HIPAA, PCI-DSS, SOC 2) affect architecture: data residency (keeping EU data in the EU), the right to deletion (you must be able to find and delete a user's data, including derived copies and backups within set timelines), and strict separation of cardholder data.

INTERVIEW TIP

Security in a design interview

Mention it briefly but concretely: "TLS everywhere with mTLS between services; secrets in a vault with rotation; encryption at rest with KMS; authentication at the gateway with short-lived JWTs; object-level authorisation inside each service; WAF and rate limiting at the edge; audit logs for admin actions."

CHECK YOUR UNDERSTANDING

1. What is mutual TLS, and why is it used between microservices?

   See answer →

2. Explain envelope encryption and its advantage.

   See answer →

3. Why are short-lived dynamic credentials preferable to long-lived static secrets?

   See answer →

4. A user changes `/api/orders/1001` to `/api/orders/1002` and sees someone else's order. What vulnerability is this, and how do you fix it?

   See answer →

PART VIII

Case Studies

Four complete designs, worked step by step the way you would in a design review or interview — each one exercising a different set of the ideas from Parts I–VII.

CHAPTER 26

↩ Contents

Case Study: A URL Shortener

A URL shortener turns a long URL into a short link (`sho.rt/x7Kp2Q`) and redirects visitors to the original. It looks trivial, but at scale it exercises ID generation, read-heavy caching, and database choice.

26.1 Step 1: Requirements

Functional

- Create a short link for a long URL, optionally with a custom alias and expiry.
- Redirect a short link to its long URL.
- Basic click analytics (count per link, by country and day).

Non-functional

- Very high availability for redirects, which are on the critical path of every click.
- Low redirect latency (p99 < 50 ms at the server).
- Short codes must be unique and not easily guessable.
- Scale: 100M new links/month and a 100:1 read/write ratio.

26.2 Step 2: Estimates

- Writes: 100M / month ÷ 2.5M s ≈ **40 links/s** (peak ~200/s).
- Reads: 100 × 40 = **4,000 redirects/s** average, ~20,000/s at peak.
- Storage: 100M/month × 12 × 5 years = 6B links × ~500 bytes ≈ **3 TB** (plus replicas).
- Code length: with base62 (`a–z A–Z 0–9`), $62^7 \approx 3.5$ trillion codes, far more than 6B. **7 characters** is plenty.

26.3 Step 3: API

```
POST /v1/links        {long_url, custom_alias?, expires_at?}  -> 201 {code,
short_url}
GET  /{code}          -> 301/302 Location: long_url
GET  /v1/links/{code}/stats  -> {clicks, by_country, by_day}
```

TRADE-OFF

301 or 302?

A **301 (permanent)** redirect is cached by browsers, which cuts load but means you never see repeat clicks, so analytics undercount. A **302 (temporary)** sends every click through you: accurate analytics, more load. Most commercial shorteners use 302 (or 307) for analytics.

26.4 Step 4: Generating short codes

Option A: hash the URL (e.g. the first 7 base62 characters of SHA-256). This is deterministic, but collisions must be detected and resolved, and the same URL always produces the same code (sometimes undesirable).

Option B: a unique counter + base62 encoding. Each new link gets a unique 64-bit integer, encoded in base62. There are no collisions by construction. To avoid a central counter bottleneck, each app server **leases ranges** of IDs from a coordinator (for example, blocks of 10,000 recorded in a tiny ZooKeeper/etcd or database table), or uses Snowflake-style IDs (Chapter 21). Sequential codes are guessable, so if enumeration is a concern, permute the integer with a reversible bijective scramble before encoding.

Option C: pre-generated random codes. A key-generation service fills a table of unused random codes, and app servers take batches. The codes are unguessable and handed out with no collision checks at request time.

We choose **B with range leasing**: simple, fast, collision-free, with a scramble step for unguessability.

26.5 Step 5: Data model and storage

```
links(code PK, long_url, user_id, created_at, expires_at)
```

Access is pure key-value by `code`, with no joins, enormous read volume and modest writes. A **key-value / wide-column store** (DynamoDB, Cassandra) partitioned by `code` fits well. A sharded relational database also works. Analytics events go to a separate pipeline, never into

the hot path.

26.6 Step 6: High-level design

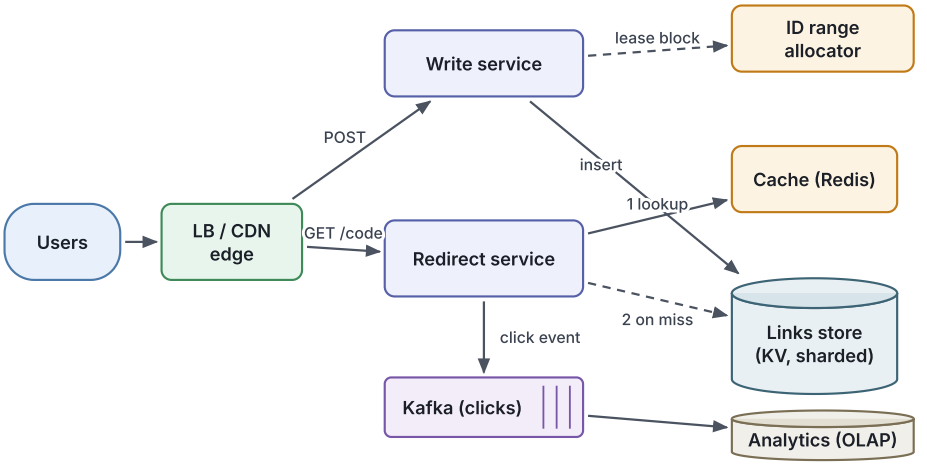


Figure 26.1 URL shortener architecture. Redirects are served mostly from cache, and click events flow asynchronously to analytics.

Redirect path: look up `code` in the cache. On a miss, read from the store, fill the cache, and return a 302. Emit a click event asynchronously and **never block the redirect** on analytics.

26.7 Step 7: Deep dives

- **Caching:** link popularity is extremely skewed, so a cache holding the hottest ~20% of links absorbs most reads. Use LRU, cache "not found" results briefly to block enumeration (penetration), and consider caching hot redirects at the CDN edge.
- **Expiry:** store `expires_at`, check it on read, and run a background job to purge expired links.
- **Custom aliases:** insert with a conditional write ("only if not exists") to enforce uniqueness.
- **Abuse:** check submitted URLs against malware and phishing lists, rate limit creation per user and IP, and support takedowns.
- **Availability:** redirects depend only on the cache and a replicated store, with multi-AZ deployment and stateless services. Even if analytics is down, redirects keep working.

CHECK YOUR UNDERSTANDING

1. Why is a 7-character base62 code sufficient for 6 billion links?

   See answer →

2. Why should click analytics be published asynchronously rather than written synchronously during the redirect?

   See answer →

3. How do you generate unique IDs across many app servers without a central counter on every request?

   See answer →

CHAPTER 27

↩ Contents

Case Study: A Social News Feed

Users follow others, and their home feed shows recent posts from the people they follow, ranked by relevance. The core challenge is **fan-out**: how a post reaches millions of feeds quickly, when follower counts range from 10 to 100 million.

27.1 Step 1: Requirements

Functional: create posts (text and media); follow and unfollow; view a home feed of followed accounts' posts, ranked, with infinite scroll.

Non-functional: feed loads in < 200 ms (p99); new posts appear in followers' feeds within seconds (eventual consistency is fine); 300M DAU; highly available.

27.2 Step 2: Estimates

- Posts: 300M DAU × 0.5 posts/day = 150M posts/day ≈ **1,750 writes/s** (peak ~5k/s).
- Feed reads: 300M × 10 feed loads/day = 3B/day ≈ **35,000 reads/s** (peak ~100k/s).
- Average 200 followers per user, so 150M posts × 200 = **30B feed insertions/day** if we push to every follower.

27.3 Step 3: Fan-out on write vs. fan-out on read

Fan-out on write (push): when a user posts, insert the post ID into each follower's precomputed feed (a list in Redis, for example). Reading a feed is then a single fast lookup. The problem: a celebrity with 100M followers creates 100M writes per post, which is slow, expensive, and mostly wasted on inactive users.

Fan-out on read (pull): store posts only with their author. When a user opens their feed, fetch recent posts from everyone they follow and merge them. Writes are cheap and celebrities aren't a problem, but reads are expensive (following 1,000 accounts means 1,000 lookups per feed load), which is too slow at 100k reads/s.

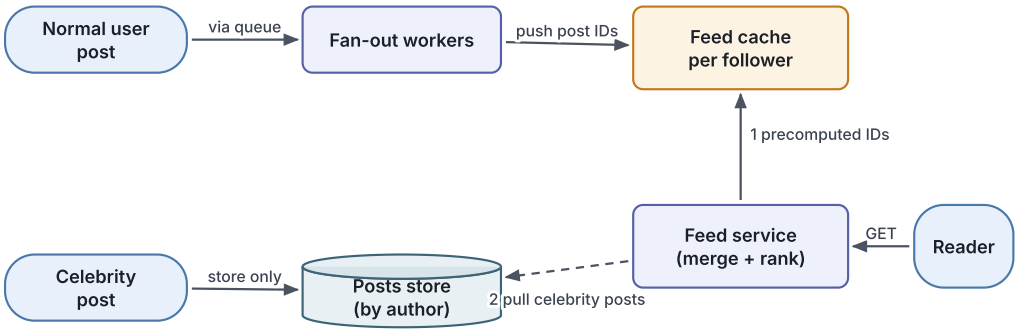


Figure 27.1 Hybrid fan-out. Ordinary users' posts are pushed into followers' feed caches; celebrity posts are pulled and merged at read time.

Hybrid (the standard answer): push for normal users, pull for accounts above a follower threshold (say 100k), merging those posts in at read time. Also skip pushing to users who haven't been active for weeks, and build their feed on demand when they return.

27.4 Step 4: Data model

```
users(user_id PK, name, ...)
follows(follower_id, followee_id, created_at)   -- two tables/indexes for both
directions
posts(post_id PK (Snowflake, time-sortable), author_id, body, media_keys,
created_at)
feed:{user_id} -> Redis sorted set of (post_id, score), trimmed to ~800 entries
```

Posts are sharded by `post_id` (or `author_id` for author timelines). The social graph is stored in a sharded key-value or graph store, with both the "followers of X" and "following of Y" directions indexed. Media lives in object storage behind a CDN.

27.5 Step 5: High-level design

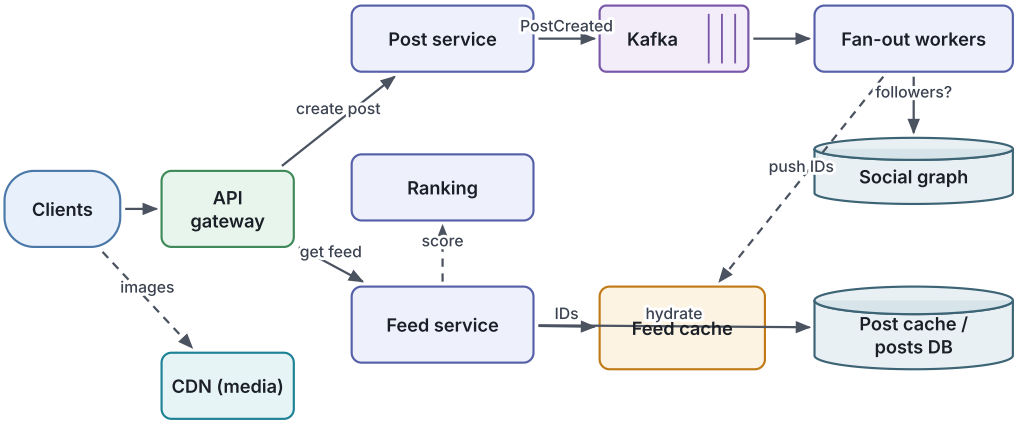


Figure 27.2 News feed architecture. The write path fans out asynchronously, and the read path hydrates cached post IDs into full posts.

27.6 Step 6: Deep dives

- **Ranking:** a pure reverse-chronological order is simple. Ranked feeds score candidates (recency, affinity to the author, engagement, content type) with an ML model. Two stages: *candidate generation* (a few hundred post IDs), then *ranking* (scoring them), with cached features.
- **Pagination:** cursor-based on `(score, post_id)` for stable infinite scroll (Chapter 8).
- **Hydration:** feed caches hold only IDs. Post bodies, author info and counts are fetched in batches from caches, with *multi-get*.
- **Counters (likes, comments):** very write-heavy, and a few seconds stale is fine. Increment in Redis or sharded counters and flush to durable storage asynchronously.
- **Deletes and privacy:** a deleted post is filtered at hydration time (a tombstone), which avoids having to remove it from millions of feed caches immediately.
- **Hot posts:** viral posts are replicated across cache nodes and served from local in-process caches.

CHECK YOUR UNDERSTANDING

1. Why does pure fan-out-on-write break for celebrity accounts?

See answer →

2. Describe the hybrid fan-out approach.

See answer →

3. Why store only post IDs in the feed cache rather than full posts?

See answer →

CHAPTER 28

↩ Contents

Case Study: A Chat System

Design a messaging service like a modern chat app: one-to-one and group conversations, delivery and read receipts, online presence, and message history on every device.

28.1 Step 1: Requirements

Functional: 1:1 and group chats (up to 500 members); real-time delivery; message history synced across devices; sent/delivered/read receipts; online/last-seen presence; push notifications when offline.

Non-functional: low delivery latency (< 300 ms within a region); messages are **never lost** and appear **in order** within a conversation; 500M DAU, 50M concurrent connections; highly available.

28.2 Step 2: Estimates

- Messages: 500M DAU × 40/day = 20B/day ≈ **230k messages/s** average (peak ~700k/s).
- Storage: 20B × 200 bytes ≈ 4 TB/day ≈ **1.5 PB/year** before replication.
- Connections: 50M concurrent WebSockets ÷ ~100k per gateway ≈ **500+ gateway servers**.

28.3 Step 3: Connection layer

Clients hold a persistent **WebSocket** to a **chat gateway** (Chapter 10). Gateways are stateful (they hold connections) but contain no business logic. A **session registry** (Redis) maps `user_id → {gateway_id, device_id}` for every connected device.

28.4 Step 4: Sending a message

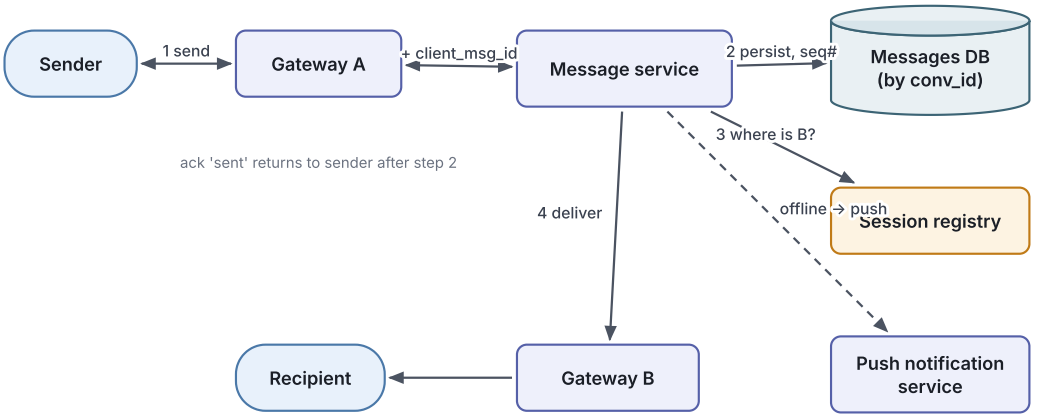


Figure 28.1 Message flow: persist first, acknowledge the sender, then route to recipients' gateways or to push notifications.

1. The client sends the message with a client-generated `client_msg_id` (for **idempotent retries**).
2. The message service assigns a **per-conversation sequence number** and persists the message *before* anything else (durability first).
3. The sender gets an acknowledgement and shows a single tick ("sent").
4. The service looks up each recipient's devices in the session registry and forwards the message to their gateways, which push it down the sockets. The recipient's client acks, and the sender is shown "delivered".
5. Offline recipients get a push notification. When they reconnect, their client **syncs** by asking for all messages after the last sequence number it has seen in each conversation.

28.5 Step 5: Ordering and delivery guarantees

- **Ordering:** a per-conversation sequence number (assigned by the conversation's owner shard, or by a counter in the message store) gives a total order within each chat. Clients sort by it and spot gaps.
- **At-least-once + dedup:** retries may duplicate sends, so the server de-duplicates on `client_msg_id`, and clients ignore sequence numbers they already have.
- **Multi-device sync:** each device tracks its own last-synced sequence number per conversation, so a newly signed-in laptop pulls history incrementally.

28.6 Step 6: Storage

The access pattern is "the latest N messages in conversation X, then page backwards", plus an enormous write volume. A **wide-column store** (Cassandra/ScyllaDB/HBase) fits:

```
messages: partition key = conversation_id (+ time bucket for huge groups)
          clustering key = seq_no DESC
          columns: sender_id, body (encrypted), type, created_at
```

An `inbox` table per user (`user_id → conversations` sorted by last activity) powers the chat list screen. Media goes to object storage, and messages carry only references.

28.7 Step 7: Group chats

For small groups, fan out on write: the message service delivers to each member's online devices. For large groups (hundreds of members, or broadcast channels), store the message once and let members fetch it, or route through a pub/sub topic per group that the relevant gateways subscribe to, rather than looking up every member.

28.8 Step 8: Presence

Presence (online/offline, last seen) is high-churn, low-value data:

- Clients send **heartbeats** every ~30 s through the WebSocket. If heartbeats stop for a timeout, the user is marked offline.
- Store presence in an in-memory store with TTLs. Publish changes only to users who currently have the relevant chat open, or to contacts, and **batch and throttle** updates. Exact precision isn't worth the fan-out cost.

28.9 Step 9: End-to-end encryption

With **end-to-end encryption** (E2EE), messages are encrypted on the sender's device with keys only recipients' devices hold, so the server stores and routes only ciphertext. Modern protocols use per-message keys derived through a ratchet, giving forward secrecy. Architecturally, the server can't search or moderate content, multi-device support needs per-device keys, and backups must be encrypted client-side.

CHECK YOUR UNDERSTANDING

1. Why is the message persisted before it is delivered to recipients?

   See answer →

2. How does the system keep messages in order and avoid duplicates when clients retry?

   See answer →

3. Why is `conversation_id` a good partition key for message storage?

   See answer →

4. How would you handle a recipient who is offline?

   See answer →

CHAPTER 29

↩ Contents

Case Study: A Ride-Hailing Platform

Riders request trips, nearby drivers are matched in seconds, both see each other's live location, and the trip is priced and paid. This design brings together real-time location ingestion, geospatial indexing, matching and consistent state transitions.

29.1 Step 1: Requirements

Functional: drivers publish their location while online; riders see nearby cars and request a ride; the system matches a nearby available driver who can accept or decline; both see live locations during the trip; fare estimation, surge pricing and payment.

Non-functional: matching within a few seconds; location updates reflected within ~5 s; a driver must **never be assigned two trips at once**, and a trip never gets two drivers; highly available, especially matching; global, multi-city.

29.2 Step 2: Estimates

- 5M drivers online at peak, each sending a location every 4 s, gives **~1.25M location updates/s**. This write load dominates everything else.
- 20M trips/day ≈ 230 trip requests/s on average, with heavy local peaks (rush hour, concerts).
- Each update ~100 bytes, so ~125 MB/s of ingest. Only the *latest* location needs to be kept hot. History goes to cheap storage for analytics.

29.3 Step 3: High-level design

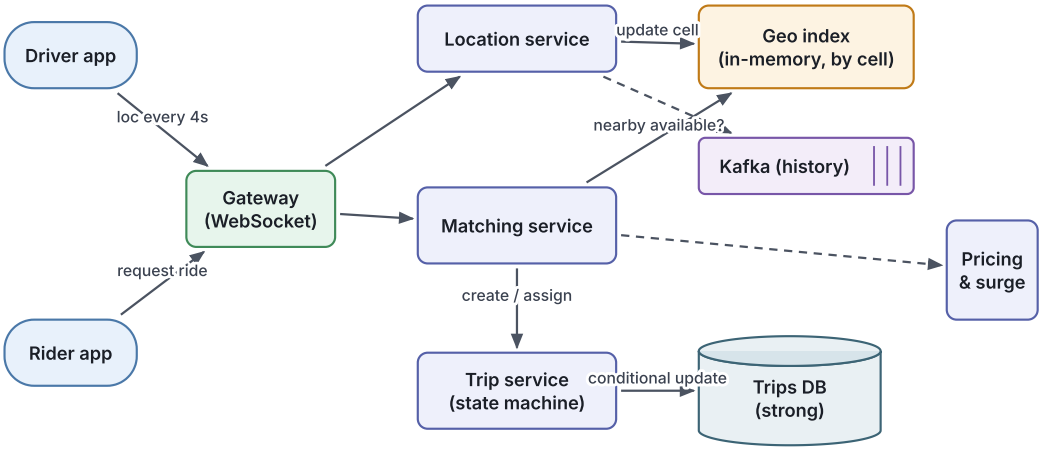


Figure 29.1 Ride-hailing architecture. Location updates flow into an in-memory geospatial index, and matching queries it and claims drivers atomically.

29.4 Step 4: Location ingestion and geospatial index

Drivers send updates over a persistent connection to the **location service**, which updates an **in-memory geospatial index** that maps each cell (geohash prefix, S2 cell or H3 hexagon, Chapter 21) to the available drivers in it. The index is partitioned **by city or region and cell**, so that each partition handles a manageable area, with replicas for availability. Every update is also appended to Kafka for trip tracking, analytics, ETA model training and fraud detection.

Why not write every update to a database? 1.25M writes/s of data that is obsolete in 4 seconds doesn't need durability. Losing one update costs nothing, because the next arrives shortly. Keep the latest positions in memory (Redis GEO, or custom services) with TTLs, so that drivers who go silent drop out of the index automatically.

29.5 Step 5: Matching

1. A rider requests a trip. The matching service finds the rider's cell and queries it and its **neighbouring cells** for available drivers, widening the radius if too few are found.
2. Candidates are ranked by **estimated time of arrival** (using the road network, not straight-line distance), driver rating, vehicle type, and so on.
3. The best driver gets an offer with a short timeout (~10 s). If they decline or time out, the next candidate gets one.

Preventing double assignment is a consistency problem (Chapters 4 and 13). Claim the driver with an **atomic conditional update**: `UPDATE drivers SET state='ON_OFFER', trip=:t WHERE id=:d AND state='AVAILABLE'`, or the equivalent compare-and-set in the store. Only one matcher can win, and any others move on to the next candidate. A lease with an expiry returns the driver to AVAILABLE if the offer times out.

29.6 Step 6: The trip state machine

Each trip is a **state machine** stored in a strongly consistent database, with each transition applied as a conditional write:

`REQUESTED → DRIVER_ASSIGNED → DRIVER_ARRIVED → IN_PROGRESS → COMPLETED → PAID` (plus `CANCELLED` from several states).

State changes emit events through a transactional outbox (Chapter 20) to notifications, pricing, payments and analytics. Payment runs **after** completion as a saga, with idempotency keys, retries and compensation (refunds) when needed.

29.7 Step 7: Surge pricing and ETAs

- **Surge:** a stream processor (Chapter 22) computes demand (open requests) and supply (available drivers) per hexagon over sliding windows of a minute or two, and publishes multipliers that the pricing service reads. Smoothing and caps keep prices from swinging wildly.
- **ETA:** routing engines over the road graph with live traffic, refined by ML models trained on historical trips, and cached per cell pair for speed.

29.8 Step 8: Reliability and scale

- **Partition by geography:** cities are largely independent, so each region's services and data can scale and fail in isolation, and an outage in one city doesn't affect others.
- **Graceful degradation:** if ETA services are slow, fall back to distance-based estimates. If surge computation lags, use the last known multiplier.
- **Hot spots:** an airport or stadium cell can get thousands of drivers. Subdivide hot cells (adaptive resolution) and queue drivers fairly (an airport FIFO).

CHECK YOUR UNDERSTANDING

1. Why are live driver locations kept in memory rather than written to a durable database on every update?

   See answer →

2. How do you make sure two matchers can't assign the same driver to two riders at the same time?

   See answer →

3. Why does the matcher search neighbouring cells, and why rank by ETA rather than straight-line distance?

   See answer →

4. Which parts of this system need strong consistency, and which can be eventual?

   See answer →

APPENDIX

↩ Contents

The Design Interview Playbook

A one-page method for any design question, with time budgets for a 45-minute session.

Phase	Time	What to do	Output
1. Requirements	5 min	Ask about users, core features, scale, consistency and latency needs. Agree on what is out of scope.	3–5 functional + 3–5 non-functional requirements
2. Estimates	3–5 min	QPS (avg/peak), read/write ratio, storage, bandwidth.	Numbers that justify the design
3. API	3–5 min	Endpoints/messages with key parameters.	A contract
4. Data model	3–5 min	Entities, keys, access patterns; choose stores.	Schema + store choice
5. High-level design	10 min	Draw the boxes; walk the read and write paths end to end.	A working architecture
6. Deep dives	10–15 min	Bottlenecks at 10x: caching, sharding, async, consistency, hot spots.	Justified refinements
7. Wrap-up	3–5 min	Failures, monitoring, security, trade-offs, future work.	Evidence of senior judgement

Questions to ask in every design

- What are the read and write paths, and their ratio?
- What is the single most important non-functional requirement?
- Where does state live, and what is the source of truth?
- What is the shard key, and does the main query hit one shard?
- What gets cached, how is it invalidated, and what happens on a stampede?
- What is synchronous, and what can be asynchronous?
- Which data needs strong consistency, and which can be eventual?
- What happens when each component fails? Where are the single points of failure?

- How do retries stay safe (idempotency)?
- What would you monitor and alert on?

Phrases that show judgement

- "Let me state my assumptions so you can correct them…"
- "The bottleneck here will be ___, because ___."
- "I'm choosing ___ over ___ because our priority is ___; the cost is ___."
- "This part can be eventually consistent; this part cannot."
- "If this component fails, the user sees ___, and we recover by ___."

APPENDIX

↩ Contents

Reference Numbers

Latency

Operation	Time
L1 cache / branch mispredict	~1–5 ns
Main memory access	~100 ns
Mutex lock/unlock (uncontended)	~20 ns
SSD random read	~16–100 µs
Same-datacentre round trip	~0.5 ms
Redis GET over the network	~0.2–1 ms
Simple indexed DB query	~1–5 ms
HDD seek	~10 ms
Cross-continent round trip	~70–150 ms
TCP + TLS 1.3 setup at 100 ms RTT	~200 ms

Throughput rules of thumb (single node, order of magnitude)

Component	Capacity
Stateless web/API server	1k–10k req/s
Relational DB (simple queries)	5k–50k queries/s
Redis / Memcached	100k–1M ops/s
Kafka partition	~10–50 MB/s
WebSocket gateway (mostly idle)	100k–1M connections
1 Gbps network link	~125 MB/s
10 Gbps network link	~1.25 GB/s

Time and availability

Availability	Downtime/year	Downtime/day
99%	3.65 days	14.4 min
99.9%	8.77 h	1.44 min
99.99%	52.6 min	8.6 s
99.999%	5.26 min	0.86 s

$1 \text{ day} \approx 10^5 \text{ s} \cdot 1\text{M/day} \approx 12\text{/s} \cdot 1\text{B/day} \approx 12\text{k/s} \cdot 1 \text{ KB} \times 1\text{B} = 1 \text{ TB}$

APPENDIX

↩ Contents

Glossary

ACID

Atomicity, Consistency, Isolation, Durability: the guarantees of database transactions (Ch. 13).

Anycast

Announcing one IP address from many locations, so the network routes each client to the nearest one (Ch. 7).

Availability zone

An isolated data centre (or group of data centres) within a cloud region, with independent power and networking.

Backpressure

A way for an overloaded consumer to slow its producers instead of being overwhelmed (Ch. 11).

Bloom filter

A compact probabilistic set that can say "definitely not present" or "probably present" (Ch. 21).

Bulkhead

Isolating resources per dependency so that one failure can't exhaust everything (Ch. 18).

CAP theorem

During a network partition, a replicated system must choose between consistency and availability (Ch. 4).

CDC

Change data capture: streaming a database's committed changes by reading its replication log (Ch. 20).

CDN

Content delivery network: edge servers that cache content close to users (Ch. 16).

Circuit breaker

A wrapper that stops calling a failing dependency, failing fast until it recovers (Ch. 18).

Consensus

Getting distributed nodes to agree on a value despite failures; e.g. Raft, Paxos (Ch. 19).

Consistent hashing

Hashing keys and nodes onto a ring, so adding or removing a node moves only ~1/N of the keys (Ch. 15).

CQRS

Command Query Responsibility Segregation: separate models for writes and reads (Ch. 17).

CRDT

Conflict-free replicated data type: a structure whose replicas merge deterministically without coordination (Ch. 14).

Dead-letter queue

Where messages go after repeatedly failing processing (Ch. 11).

Denormalisation

Duplicating data where it is read, to avoid joins at the cost of extra writes (Ch. 13).

Error budget

The amount of unreliability an SLO allows; spent on risk and releases (Ch. 3).

Eventual consistency

Replicas converge if writes stop, with no guarantee about when (Ch. 4).

Fan-out

One event producing many downstream writes or messages, e.g. a post delivered to followers' feeds (Ch. 27).

Fencing token

An increasing number that lets storage reject requests from a stale leader or lock holder (Ch. 19).

Gossip protocol

Peer-to-peer, epidemic-style spreading of state such as cluster membership (Ch. 19).

HyperLogLog

A probabilistic structure estimating distinct counts in ~12 KB (Ch. 21).

Idempotency

Doing an operation several times has the same effect as doing it once; essential for safe retries (Ch. 3, 8).

Leader election

Choosing one node to coordinate, and replacing it on failure (Ch. 19).

Linearizability

The strongest single-object consistency: the system behaves as if there were one copy and operations are instantaneous (Ch. 4).

LSM tree

A write-optimised storage structure built from memtables and immutable sorted files merged by compaction (Ch. 14).

mTLS

Mutual TLS: both sides authenticate with certificates (Ch. 25).

Outbox pattern

Writing events to an outbox table in the same transaction as the data change, then publishing them (Ch. 20).

p99

The 99th-percentile latency: 99% of requests are faster than this (Ch. 2).

PACELC

If Partitioned, choose Availability or Consistency; Else, choose Latency or Consistency (Ch. 4).

Quorum

The minimum number of nodes that must take part for an operation to succeed; with R + W > N, reads see the latest write (Ch. 4).

Rate limiting

Restricting how many requests a client may make in a period (Ch. 9).

Replication lag

The delay before a follower reflects the leader's writes (Ch. 14).

Saga

A sequence of local transactions with compensating actions, replacing a distributed transaction (Ch. 20).

Sharding

Splitting data horizontally across nodes by a shard key (Ch. 15).

Sidecar

A helper process deployed next to each service instance, e.g. a mesh proxy (Ch. 18).

SLI / SLO / SLA

Indicator (what you measure) / Objective (internal target) / Agreement (contract) (Ch. 3).

SPOF

Single point of failure: a component whose failure takes down the whole system (Ch. 3).

Split brain

Two nodes both acting as leader after a partition, so their data diverges (Ch. 3, 19).

Stateless service

A service that keeps no client state between requests, so any instance can serve any request (Ch. 2).

Strangler fig

Migrating a legacy system by gradually routing features to new services (Ch. 18).

Thundering herd

Many clients doing the same expensive thing at once, e.g. a cache stampede (Ch. 12).

Two-phase commit

An atomic commit protocol with prepare and commit phases under a coordinator (Ch. 20).

Watermark

In stream processing, the estimate that all events before time T have arrived (Ch. 22).

APPENDIX

↩ Contents

Answer Key

Every answer links back to its question. Try each question honestly before reading the answer — retrieval is what makes it stick.

Chapter 1: What System Design Is

1. (a) Functional. (b) Non-functional (latency). (c) Non-functional (durability and availability). (d) Functional, although *how fast* search must be would be non-functional.

   ↩ Back to question

2. If no server holds per-user state between requests, any server can handle any request. The load balancer can then route freely, servers can be added or removed at will (horizontal scaling), and a server crash loses nothing. State goes into dedicated systems built to manage it: databases, caches and session stores.

   ↩ Back to question

3. Scale decides the shape of the design. At 10 requests per second a single server and database are plenty. At 100,000 per second you need load balancing, caching, sharding and probably asynchronous processing. Without numbers you can't justify adding or leaving out any component.

   ↩ Back to question

Chapter 2: Scalability and Performance

1. An average mixes fast and slow requests into one number that no individual user actually experiences, and a long tail can hide behind it. You need percentiles (p50, p95, p99, p99.9) to see how the slowest requests behave, and the slowest ones often belong to your heaviest users.

   ↩ Back to question

2. $1 - 0.99^{20} \approx 1 - 0.818 \approx$ **18%** of page loads. This is tail latency amplification. Fixes include reducing fan-out, hedged requests, tighter timeouts with partial results, and improving the backends' p99.

   ↩ Back to question

3. Advantages: practically no capacity ceiling; fault tolerance, since losing one node of many is survivable; and cheaper commodity hardware. Disadvantages: more complexity (load balancing, data partitioning, distributed coordination); data consistency gets harder; and more machines to operate and monitor.

↩ Back to question

4. By Little's Law, $L = \lambda W = 5{,}000 \times 0.04 =$ **200** concurrent requests.

↩ Back to question

Chapter 3: Availability, Reliability and Fault Tolerance

1. $0.999^4 \approx 0.996$, or **99.6%**, which is roughly 35 hours of downtime per year. Every dependency in series lowers availability.

↩ Back to question

2. $1 - (0.005)^2 = 1 - 0.000025 =$ **99.9975%**. Note that this assumes the failures are truly independent. Replicas sharing a rack, power supply or bad deploy are *not* independent.

↩ Back to question

3. It is the allowed unreliability implied by the SLO. A 99.9% SLO gives a 0.1% budget of failed requests or downtime per period. While budget remains, teams can ship features quickly and take risks. When it is used up, they prioritise reliability work and freeze risky launches. It gives product and operations an objective, shared rule instead of an argument.

↩ Back to question

4. When a service is overloaded, failing requests get retried immediately, which multiplies the load (each original request may become 3–4 attempts). The extra load causes more failures and so more retries: a positive feedback loop called a retry storm. Exponential backoff, jitter, retry caps and retry budgets break the loop.

↩ Back to question

5. Split brain happens when two nodes both believe they are the primary, usually after a network partition or a slow-but-alive leader, and both accept writes, so the data diverges. It is prevented by quorum-based leader election (a leader needs a majority), fencing tokens or STONITH ("shoot the other node in the head"), and leases that expire.

↩ Back to question

Chapter 4: Consistency, CAP and PACELC

1. Because network partitions can't be prevented in a distributed system, so partition tolerance isn't optional. CAP describes what you must give up *during* a partition: consistency or availability. A system that "gives up P" is effectively a single node.

↩ Back to question

2. The trade-off during *normal* operation, with no partition: latency vs. consistency. Strong consistency needs coordination (waiting for a quorum or leader acknowledgement), which adds latency even when the network is healthy.

↩ Back to question

3. Any R + W > 5 guarantees overlap, for example W = 3, R = 3 (balanced), or W = 5, R = 1. To favour write latency while keeping overlap, use W = 1, R = 5. Fast writes with no guarantee would be W = 1, R = 1.

↩ Back to question

4. (a) Read-your-writes, so the user must see their own edit immediately. (b) Eventual, since minutes of staleness is fine. (c) Strong (linearizable) for the actual booking, to prevent overselling. Browsing availability can be slightly stale as long as the final purchase is checked strongly.

↩ Back to question

Chapter 5: Back-of-the-Envelope Estimation

1. 300M ÷ 10^5 ≈ 3,000 req/s average; × 4 ≈ **12,000 req/s** at peak.

↩ Back to question

2. 2 × 10^9 × 200 B = 400 GB/day. × 365 ≈ 146 TB/year. × 5 years ≈ 730 TB. × 3 replicas ≈ **2.2 PB**.

↩ Back to question

3. It explains why caching and in-memory data structures transform performance. Serving hot data from RAM rather than from disk-backed storage cuts latency by orders of magnitude and lets one machine serve far more requests. It also guides choices like keeping indexes and working sets in memory.

↩ Back to question

4. 20,000 × 50 KB = 1,000,000 KB/s = 1 GB/s. × 8 = **8 Gbps**.

↩ Back to question

Chapter 6: Networking Essentials

1. Resolvers and clients cache records until the TTL expires, so after you change a record to point away from a failed server, some clients keep using the old address until their cache expires. Short TTLs mean faster failover and more flexible routing, at the cost of more DNS queries and slightly slower lookups. Long TTLs mean fewer queries but slow changes.

↩ Back to question

2. (1) Real-time voice and video: a late packet is useless, so retransmitting it only adds delay, and it is better to skip it. (2) Online games sending frequent position updates, where only the newest update matters. Others: DNS lookups (one small request/response), metrics collection, and QUIC, which builds its own reliability on UDP.

   ↩ Back to question

3. HTTP/2 multiplexes many requests over one connection, removing application-level head-of-line blocking and the need for many parallel connections. It still runs on TCP, though, so one lost packet stalls every stream on that connection (transport-level head-of-line blocking). HTTP/3 runs over QUIC, whose streams are independent at the transport layer.

   ↩ Back to question

4. GET, PUT, DELETE (as well as HEAD and OPTIONS). Repeating an idempotent request has the same effect as sending it once, so clients and proxies can safely retry it after a timeout. POST isn't idempotent, so retrying it can create duplicates unless the API uses idempotency keys.

   ↩ Back to question

Chapter 7: Load Balancing

1. When routing must depend on request content: path-based routing to different services, header- or cookie-based canary splits, authentication at the edge, request retries, gRPC-aware balancing, or HTTP rewriting. L4 is preferred for raw speed, non-HTTP protocols, or as a front tier that spreads traffic across L7 proxies.

   ↩ Back to question

2. Round robin ignores how busy each server is, so a server stuck with several 30-second requests keeps receiving new ones while others sit idle. **Least connections** (or power-of-two-choices on active requests) sends new work to the least busy server.

   ↩ Back to question

3. Liveness answers "is the process alive, or should it be restarted?" Readiness answers "should this instance receive traffic right now?" An instance can be alive but not ready: it may be starting up, warming caches or temporarily overloaded. Load balancers should route by readiness. Orchestrators restart instances based on liveness.

   ↩ Back to question

4. Run several load balancers: an active-passive pair sharing a floating virtual IP, active-active load balancers behind several DNS records or anycast, or a managed cloud load balancer that is distributed internally. Spread them across availability zones.

   ↩ Back to question

Chapter 8: Designing APIs

1. Offset pagination gets slower at deep offsets (the database must skip N rows), and it produces duplicates or gaps when items are inserted or deleted between page requests. Cursor pagination seeks directly to the last-seen key using an index, so it is fast at any depth and stable while new items arrive.

   ↩ Back to question

2. Have the client send a unique idempotency key with the request. The server records the key with the outcome, atomically with creating the order. On a retry with the same key, it returns the original response instead of creating a second order. Keys expire after a reasonable window, such as 24 hours.

   ↩ Back to question

3. Resolving a list of N items, then running a separate query for each item's related field, makes 1 + N database queries. It is solved by batching and caching within a request (the DataLoader pattern), which collects all the IDs requested in one tick and fetches them with a single query.

   ↩ Back to question

4. Any four of: request routing; authentication and token validation; rate limiting and quotas; TLS termination; request and response transformation or protocol translation; aggregating several backend calls; caching; logging, metrics and tracing; CORS handling.

   ↩ Back to question

Chapter 9: Rate Limiting and Authentication

1. Up to **200**: 100 in the last moments of one window and 100 at the start of the next. Fixed windows reset abruptly at boundaries. Sliding windows or token buckets prevent this.

   ↩ Back to question

2. It enforces an average rate while allowing short bursts (up to the bucket's capacity), which matches real client behaviour. It needs only two values per client (token count and last refill time), is O(1) per request, and is easy to implement atomically in a shared store like Redis.

   ↩ Back to question

3. They are self-contained and verified without a lookup, so they can't easily be revoked before they expire. A stolen token works until `exp`. The mitigation is short-lived access tokens (minutes) with revocable, server-stored refresh tokens. Sometimes a denylist of revoked token IDs is added for emergencies.

   ↩ Back to question

4. Authentication verifies identity: Alice logs in and proves she is Alice. Authorisation decides what that identity can do: Alice can view her own invoices but not Bob's, and only users with the "admin" role can delete accounts.

↩ Back to question

Chapter 10: Real-Time Communication

1. The feed only needs server → client updates. SSE runs over plain HTTP, so it works with existing proxies, load balancers and auth. It reconnects and resumes from the last event ID automatically, and it is simpler to operate. WebSockets' bidirectional ability would go unused.

↩ Back to question

2. Server 12 publishes the message to a pub/sub system (on a channel for the recipient or the conversation), and server 140, which subscribed to that channel when the recipient connected, receives it and pushes it down the WebSocket. Alternatively, a presence registry tells server 12 that the recipient is on server 140, and it forwards the message directly. The message is also saved durably so offline or reconnecting users can fetch it.

↩ Back to question

3. STUN lets a peer discover its public IP and port as seen from outside its NAT, so peers can try to connect directly. TURN relays media between peers when a direct connection is impossible (for example, behind symmetric NATs or strict firewalls). It always works, but it costs server bandwidth.

↩ Back to question

Chapter 11: Message Queues and Publish/Subscribe

1. When several independent consumers each need to react to the same event, such as an `OrderPlaced` event used by inventory, email, analytics and fraud services. Pub/sub delivers a copy to each subscriber, which decouples the producer from the growing list of consumers. Point-to-point suits splitting work among identical workers, where each task must be done once.

↩ Back to question

2. The producer uses the `order_id` as the message key. Kafka hashes the key to pick a partition, so all events for that order land in the same partition, which is strictly ordered and read by one consumer in the group at a time. Different orders spread across partitions and are processed in parallel.

↩ Back to question

3. At-least-once delivery: the consumer probably crashed or timed out after processing but before acknowledging, so the message was redelivered. Make the consumer idempotent. Record the message or payment ID in the same transaction as the charge and skip IDs already processed, or use an idempotency key with the payment provider.

↩ Back to question

4. It holds messages that failed processing a set number of times (poison messages), so they stop blocking or endlessly consuming the main queue. Engineers can inspect them, fix the bug or data, and replay them. DLQ depth should be monitored and alerted on.

↩ Back to question

Chapter 12: Caching

1. 90%: 0.9 × 2 + 0.1 × 80 = 1.8 + 8 = **9.8 ms**. 99%: 0.99 × 2 + 0.01 × 80 = 1.98 + 0.8 ≈ **2.8 ms**. Going from 90% to 99% cuts latency by about 3.5x and database load by 10x. Hit rate matters enormously.

↩ Back to question

2. Deleting is simpler and avoids races in which concurrent updates write values into the cache in the wrong order, leaving it stale. After a delete, the next read loads the current value from the database. It also avoids computing and caching values that may never be read.

↩ Back to question

3. When a popular key expires, many simultaneous requests miss and all hit the backend to recompute the same value, which can overload it. Mitigations: request coalescing or a lock so only one request recomputes; serving the stale value while refreshing in the background; early probabilistic refresh; and TTL jitter.

↩ Back to question

4. For write-heavy workloads where low write latency matters and losing a little recent data is tolerable or can be mitigated: counters, analytics, activity tracking, game state. The main risk is data loss, since writes acknowledged by the cache but not yet flushed to the database disappear if the cache node fails.

↩ Back to question

5. Cache penetration. Defences: cache "not found" results briefly; use a Bloom filter of existing IDs to reject impossible keys before they touch the cache or database; validate the ID format; and rate limit abusive clients.

↩ Back to question

Chapter 13: Choosing a Database

1. Atomicity: all of a transaction's changes happen or none do. Consistency: transactions keep the database valid with respect to its constraints. Isolation: concurrent transactions don't see each other's partial work and behave as if run in some order. Durability: committed data survives crashes because it is persisted (usually through a write-ahead log) before the commit is acknowledged.

   ↩ Back to question

2. (a) Key-value or document store (accessed by user ID; simple and fast). (b) Graph database (relationship traversal). (c) Wide-column or time-series store (huge write throughput, queries by device and time range). (d) Relational database (multi-row ACID transactions, constraints, auditability).

   ↩ Back to question

3. A lost update. Prevent it with an atomic in-database update (`SET n = n + 1`), pessimistic locking (`SELECT … FOR UPDATE`), optimistic concurrency control (a version column with a conditional update and retry), or a stronger isolation level.

   ↩ Back to question

Chapter 14: Indexing and Replication

1. The index is sorted by country first, then by city within each country. Filtering on country uses a prefix of the sort order, so the database can seek straight to France. Paris entries are scattered across every country's section, so a city-only filter can't use the index's order (the leftmost prefix rule).

   ↩ Back to question

2. LSM trees turn random writes into sequential ones: writes go to an in-memory buffer and an append-only log, and are flushed as sorted immutable files, all of which is sequential I/O. B-trees update pages in place, which causes random I/O and page splits. The cost moves to reads, which may check several files, and to background compaction.

   ↩ Back to question

3. Replication lag: the write went to the leader, but the reload was served by an asynchronous follower that hadn't applied it yet. To guarantee read-your-writes: read a user's own profile from the leader (perhaps for a short time after a write), or record the write's log position and serve reads only from replicas that have reached it.

   ↩ Back to question

4. Writes the old leader acknowledged but hadn't yet sent to any follower are lost when a follower is promoted. If the old leader comes back and still accepts writes, split brain can follow. Semi-synchronous replication or consensus-based replication reduces this risk.

   ↩ Back to question

5. Last-write-wins using timestamps; application-defined merge logic; CRDTs that merge deterministically; or avoiding conflicts by giving each record one home region. LWW silently throws away concurrent writes, and clock skew between nodes can make an older write "win".

↩ Back to question

Chapter 15: Partitioning and Sharding

1. Every replica must apply *every* write, so all of them carry the full write load. Replicas spread reads but not writes. Write capacity grows only by spreading writes across nodes that each own a subset of the data, which is partitioning or sharding.

↩ Back to question

2. With mod-N, about 80% of keys change location (a key stays only if `hash mod 4 == hash mod 5`). With consistent hashing, only about 1/5 (20%) move: the keys the new node takes from its neighbours.

↩ Back to question

3. With few ring positions per node, the arcs are uneven, so the load is unbalanced, and a failed node's load shifts entirely to its single successor. Many virtual positions per physical node even out the distribution, spread a failed node's load across many nodes, and allow weighting by capacity.

↩ Back to question

4. Customer X's orders are spread across all shards, so the query must scatter-gather to every shard. Solutions: shard by `customer_id` instead, so a customer's orders live together; or keep a global secondary index (or a separate customer → orders table, sharded by customer) maintained asynchronously.

↩ Back to question

Chapter 16: Storage Systems and CDNs

1. Large binaries bloat the database: slower backups and replication, wasted buffer-pool memory, and costly storage. Object storage is cheaper per GB, scales without limit, is extremely durable, serves content over HTTP, and works directly with CDNs. Keep only metadata and the object key in the database.

↩ Back to question

2. A URL signed with your credentials that grants temporary, limited permission (for example, PUT one specific key for 15 minutes). Clients can upload straight to object storage without going through your servers, which saves your bandwidth and CPU, while you still control who may upload what.

↩ Back to question

3. 3x replication costs 3.0x storage and survives the loss of 2 copies. 10+4 erasure coding costs 1.4x and survives the loss of any 4 of 14 fragments. Erasure coding is far more storage-efficient with equal or better durability, but uses more CPU and network for encoding and rebuilding, and gives slower degraded reads.

↩ Back to question

4. Each content version gets a unique URL, so files can be cached for a very long time and marked immutable. Deploying a new version changes the URL referenced by your HTML, so clients pick it up immediately with no purge needed and no risk of mixing stale and fresh assets.

↩ Back to question

Chapter 17: Architectural Styles

1. (1) It is simpler to build, test, debug and deploy, so the team moves faster. (2) Requirements and domain boundaries are still changing, and wrong service boundaries are expensive to fix. (3) It avoids distributed-systems costs: network failures, distributed transactions, tracing, and lots of infrastructure. A modular monolith keeps future extraction possible.

↩ Back to question

2. A system split into services that are still tightly coupled, so it has the operational costs of microservices without the independence. Symptoms: services must be deployed together; shared databases; long chains of synchronous calls where one failure breaks everything; changing one feature touches many services.

↩ Back to question

3. Event notification sends a minimal "something changed" message, and consumers call the source to get details, so they stay coupled at runtime. Event-carried state transfer includes the relevant data in the event, so consumers can keep local copies and never call back. That is more decoupled and resilient, at the cost of larger events and duplicated data.

↩ Back to question

4. By replaying all of the account's events in order (deposits, withdrawals and so on) to compute the balance. Snapshots periodically save the computed state, so a rebuild starts from the latest snapshot and replays only later events instead of the entire history.

↩ Back to question

Chapter 18: Microservices Patterns

1. Closed: calls go through and failures are tracked. When failures pass a threshold, it moves to Open: calls fail fast or use a fallback without contacting the dependency. After a cool-down it moves to Half-open: a limited number of trial calls go through. Success returns it to Closed; failure sends it back to Open.

↩ Back to question

2. Each dependency gets its own limited pool of resources (threads, connections). If one dependency becomes slow and fills its pool, only calls to that dependency are affected. Other dependencies keep their own pools, so the rest of the service keeps working instead of every thread getting stuck.

↩ Back to question

3. An infrastructure layer of sidecar proxies, one per service instance, that handle service-to-service networking: mTLS, retries, timeouts, load balancing, metrics and tracing. A central control plane configures them. Benefits: consistent security and observability without code changes, and fine-grained traffic control. Costs: added latency per hop, resource overhead, and substantial operational complexity.

↩ Back to question

4. Use the strangler fig pattern. Put a proxy or router in front of the monolith, pick one well-bounded capability, build it as a new service (with its own data, synchronised as needed), and route that capability's traffic to it. Repeat feature by feature, using anti-corruption layers at the boundaries, until the monolith is gone or reduced to a small core.

↩ Back to question

Chapter 19: Distributed Systems Fundamentals

1. The request was lost before arriving; the remote node crashed or is paused; or the remote node processed the request but the reply was lost or delayed. Because the caller can't tell whether the operation happened, a blind retry may duplicate it, so retried operations must be idempotent (or carry idempotency keys).

↩ Back to question

2. Clocks on different machines drift and are only loosely synchronised (milliseconds or worse), NTP can make a clock jump backwards, and processes can pause. An event that truly happened later may get an earlier timestamp, so last-write-wins can silently discard the newer write. Logical clocks, hybrid clocks or bounded-uncertainty clocks avoid this.

↩ Back to question

3. Two. Commits and elections need a majority (3 of 5). With 3 nodes down, the remaining 2 can't form a majority, so the cluster stops accepting writes to preserve consistency (it chooses C over A).

↩ Back to question

4. A monotonically increasing number issued with each lock acquisition or leadership term. The protected resource remembers the highest token it has seen and rejects requests carrying older ones. This stops a stale lock holder or ex-leader (for example, one that paused and lost its lease without noticing) from corrupting data.

↩ Back to question

5. Vector clocks can tell whether two events are causally related (one happened before the other) or **concurrent**. Lamport clocks give a total order consistent with causality but can't distinguish concurrency from causation. Detecting concurrency is essential for spotting conflicting writes.

↩ Back to question

Chapter 20: Distributed Transactions

1. After voting "yes", a participant has promised to commit if told to and must hold its locks until it learns the outcome. If the coordinator crashes after collecting votes but before announcing a decision, participants can't decide on their own and stay blocked, holding locks, until the coordinator recovers.

↩ Back to question

2. The saga runs compensating transactions for the steps already completed, in reverse order: cancel the hotel booking, then cancel the flight booking. Each compensation is a local transaction in its service, and must be idempotent and retried until it succeeds. The trip ends in a "failed / cancelled" state.

↩ Back to question

3. Choreography: services react to each other's events with no central coordinator. It is loosely coupled and simple for a few steps, but the flow is implicit, hard to see and debug, and can get tangled. Orchestration: a central orchestrator tells services what to do and tracks state. The flow is explicit, easier to monitor, change and test, and good for complex processes, at the cost of an extra component and some centralisation.

↩ Back to question

4. Updating a database and publishing a message as two independent operations can't be atomic. A crash between them leaves one done and the other not, so you get lost or phantom events. The outbox writes the event into an outbox table in the same local transaction as the data change, so both commit or neither does. A relay (polling or CDC) then publishes the outbox rows to the broker, at least once.

↩ Back to question

Chapter 21: Data Structures for Scale

1. "Present" means *probably* present: it could be a false positive, because the bits may have been set by other items, so confirm with the real data store if it matters. "Absent" is certain: Bloom filters have no false negatives, because an added item always sets all its bits.

↩ Back to question

2. HyperLogLog. It estimates the number of distinct items with ~1% error using about 12 KB per counter, whatever the cardinality. Daily sketches can be merged into weekly or monthly ones without double-counting visitors who appear on several days.

↩ Back to question

3. Replicas compare root hashes. If they match, the data is identical and nothing needs to move. If not, they compare child hashes and descend only into mismatched branches, finding the differing blocks in O(log n) comparisons. Only those blocks are exchanged, not the whole dataset.

↩ Back to question

4. Two points can be very close together but on opposite sides of a cell boundary, so they don't share a prefix. Searching only the target cell would miss them. Querying the cell plus its 8 neighbours covers everything within the search radius, and you then filter by exact distance.

↩ Back to question

5. A sign bit, a 41-bit millisecond timestamp, a 10-bit machine/worker ID and a 12-bit sequence number. IDs are unique without coordination (machine ID + sequence), sort roughly by creation time (timestamp first, which also helps index locality), fit in 64 bits, and can be generated at up to 4,096 per millisecond per worker.

↩ Back to question

Chapter 22: Big Data Processing

1. Analytical queries typically aggregate a few columns across huge numbers of rows. Columnar storage reads only the columns a query needs (far less I/O), and values of one type stored together compress extremely well. That allows vectorised processing and large speed-ups over row stores for these workloads.

↩ Back to question

2. Map: each mapper reads part of the logs and emits `(url, 1)` for each page view. Shuffle: the framework groups all pairs by URL and sends each URL's values to one reducer (sorting by key). Reduce: each reducer sums the values for its URLs and outputs `(url, total_views)`.

↩ Back to question

3. Event time is when the event actually happened. Processing time is when the system processes it. Network delays and offline devices make events arrive late and out of order, so windows based on processing time give wrong results. A watermark estimates how far event time has progressed ("all events before T have probably arrived"), so the engine knows when to close a window while bounding how long it waits for late data.

↩ Back to question

4. Lambda needs the same business logic implemented and maintained in two different systems (batch and streaming), and the two must give consistent results, which doubles the complexity. Kappa uses a single stream-processing codebase and recomputes by replaying the log.

↩ Back to question

Chapter 23: Deployment and Release Engineering

1. Blue-green switches all traffic at once between two full environments. Rollback is instant, but every user is exposed at the switch and it needs double capacity. Canary sends a small, growing percentage of traffic to the new version while comparing metrics, which limits how many users a bad release reaches, but it needs traffic splitting and solid observability. Prefer canary for high-traffic user-facing services. Blue-green suits simpler setups, or cases where both versions can't serve side by side.

↩ Back to question

2. Deploying puts code on production servers. Releasing exposes a feature to users. Feature flags let new code be deployed switched off and then turned on gradually (or turned off instantly) without another deploy.

↩ Back to question

3. Expand–contract: (1) add the `display_name` column; (2) deploy code that writes both columns; (3) backfill `display_name` from `fullname` in batches; (4) deploy code that reads `display_name`, still writing both; (5) deploy code that writes only `display_name`; (6) in a later release, drop `fullname`. Each step is compatible with the code before and after it, so any deploy can be rolled back.

↩ Back to question

Chapter 24: Observability

1. A counter only increases (total HTTP requests served); you look at its rate. A gauge rises and falls (current queue depth, memory usage). A histogram records the distribution of observations in buckets (request latency), allowing percentiles like p99.

↩ Back to question

2. Each distinct label value creates a separate time series. Millions of users would mean millions of series, which explodes the metrics system's memory, storage and query cost (a cardinality explosion). Per-user detail belongs in logs and traces.

↩ Back to question

3. The first service creates a trace ID and a root span, and each downstream call carries the trace context in headers. Every service records spans (timed operations with parent span IDs) tagged with that trace ID. The tracing backend assembles all spans with the same trace ID into a tree, or waterfall, showing the whole request path and where the time went.

↩ Back to question

4. Symptoms (high error rate, high latency, SLO burn) directly reflect user impact and are always worth waking someone up for. Causes (high CPU, one node down) often have no user impact in a redundant system, which creates noisy pages and alert fatigue. Cause-level signals are better on dashboards and for diagnosis.

↩ Back to question

Chapter 25: Security at Scale

1. In mTLS, both client and server present and verify certificates, so each side cryptographically authenticates the other, and the traffic is encrypted. Between microservices it provides zero-trust service identity: a service accepts calls only from authenticated workloads, not just from anything on the internal network. Service meshes usually automate certificate issuance and rotation.

↩ Back to question

2. Data is encrypted with a data encryption key (DEK). The DEK is encrypted ("wrapped") with a key encryption key (master key) held in a KMS or HSM, and only the wrapped DEK is stored with the data. The master key never leaves the KMS, access can be tightly controlled and audited, and rotating the master key only means re-wrapping the small DEKs, not re-encrypting all the data.

↩ Back to question

3. If they leak, they expire quickly, which limits the damage. Each workload can get unique credentials, so access is attributable and revocable individually. Rotation is built in rather than a risky manual chore. Static secrets tend to get copied, shared and forgotten, and stay valid for years.

↩ Back to question

4. Broken object-level authorisation (also called IDOR, insecure direct object reference). Fix it by checking, on every request in the service that owns the data, that the authenticated user is allowed to access that specific object (for example, that the order belongs to them or that they have an appropriate role). Unguessable IDs alone are not a fix.

↩ Back to question

Chapter 26: Case Study: A URL Shortener

1. $62^7 \approx 3.5 \times 10^{12}$ possible codes, about 580 times more than 6×10^9 links. That leaves plenty of room for growth and keeps the space sparse, which helps make codes hard to guess.

↩ Back to question

2. The redirect is latency-critical and must be highly available. Writing analytics synchronously adds latency and makes redirects depend on the analytics store's availability. Publishing an event to a queue (fire-and-forget, buffered) keeps the redirect fast and isolated, and consumers aggregate clicks in bulk.

↩ Back to question

3. Lease ranges: each server takes a block of IDs (e.g. 10,000) from a coordinator with one atomic operation and hands them out locally from memory, asking for a new block when it runs low. Alternatives are Snowflake-style IDs (timestamp + machine ID + sequence) or a pre-generated pool of random codes.

↩ Back to question

Chapter 27: Case Study: A Social News Feed

1. One post from an account with 100M followers needs 100M feed insertions, which takes a long time, costs a lot of write capacity, delays the post for most followers, and mostly writes to feeds of inactive users. Repeated posts multiply the cost.

↩ Back to question

2. Posts from ordinary accounts are pushed asynchronously into followers' precomputed feed caches. Posts from accounts above a follower threshold are not pushed, and are fetched from the author's timeline at read time and merged with the precomputed feed before ranking. Inactive users are also skipped during push, with their feed built on demand.

↩ Back to question

3. IDs are tiny, so feeds of hundreds of entries fit cheaply in memory for hundreds of millions of users. Post content can change (edits, like counts, deletions) and would otherwise have to be updated in millions of copies. Fetching content at read time from a shared post cache keeps one source of truth.

↩ Back to question

Chapter 28: Case Study: A Chat System

1. Durability: once the sender sees "sent", the message must never be lost, even if the recipient's gateway or the delivery path fails. Persisting first means delivery can always be retried and offline or new devices can sync it from storage later.

↩ Back to question

2. The server assigns a monotonically increasing sequence number per conversation when it persists each message, and clients display messages by sequence number. Clients attach a unique `client_msg_id` to each send, and the server de-duplicates retries by that ID. Recipients ignore sequence numbers they already have.

↩ Back to question

3. The dominant query is "recent messages in conversation X, paged backwards". Partitioning by conversation puts all of a conversation's messages on one partition, sorted by sequence number, so the read is one sequential partition read. Very large or old conversations can be split further by adding a time bucket to the key.

↩ Back to question

4. The message is already stored durably. The session registry shows no active connection, so the service sends a push notification through the platform's push service (APNs/FCM). When the user reconnects, the client syncs every conversation from its last known sequence number and acknowledges, which triggers "delivered" receipts to the senders.

↩ Back to question

Chapter 29: Case Study: A Ride-Hailing Platform

1. At ~1M+ updates per second, durable writes would be very expensive, and each value is obsolete within seconds, so losing one update is harmless because the next one replaces it. An in-memory geospatial index with TTLs handles the write rate cheaply and answers proximity queries fast. History is streamed to Kafka and cheap storage for analytics.

↩ Back to question

2. Claim the driver with an atomic conditional update (compare-and-set) that succeeds only if the driver's state is still AVAILABLE, moving them to ON_OFFER with that trip ID. Only one claim can succeed, and the losers try other candidates. An offer timeout or lease releases the driver if they don't accept.

↩ Back to question

3. A rider near a cell boundary may have the closest drivers in adjacent cells, so searching only one cell would miss them. Straight-line distance ignores roads, rivers, one-way streets and traffic: a driver 300 m away across a river may be 15 minutes away by road, so ETA reflects real pickup time.

↩ Back to question

4. Strong: driver assignment (no double-booking), trip state transitions, payments and the ledger. Eventual: driver positions on the rider's map, the nearby-cars display, surge multipliers, ETAs, analytics, and ratings aggregation.

↩ Back to question